



Enterprise .NET Architecture Mastery

Contents

Contents	2
How to use this guide	4
A one-page mental model	4
Part A. Architectural Principles and Patterns	5
A1. Quality attributes: the real job of an architect	5
A2. Architecture decision records and fitness functions	6
A3. Core design principles	6
A4. The architecture style spectrum	7
A5. Microservices in depth	8
A6. Event-driven architecture (EDA)	10
A7. Service-Oriented Architecture (SOA)	12
A8. Domain-Driven Design (DDD)	13
A9. CQRS and Event Sourcing	15
A10. Distributed transactions: sagas	16
A11. Evolving a legacy system: the strangler fig	17
A12. Architectural anti-patterns and mitigations	17
Part B. Technology Stack Selection	19
B1. A repeatable selection method	19
B2. Languages and frameworks: trade-offs	19
B3. Data store selection	20
B4. Cloud vendor selection	21
B5. Containerization with Docker	22
B6. Orchestration: Kubernetes and alternatives	23
B7. Frontend, mobile and hybrid	24
B8. Optimising for cloud-native, mobile and hybrid	25
B9. Cost, performance and scalability trade-off cheat sheet	25
Part C. System Design and Modeling	26
C1. High-level versus low-level design	26
C2. Core components, data flows and integration points	26
C3. API design	27
C4. UML essentials	28
C5. The C4 model	29
C6. Capacity planning and performance modeling	30

C7. Containers and orchestration for operational efficiency	32
C8. Observability and operability design	33

Part D. Non-Functional Requirements as Architecture 34

D1. Security architecture	34
D2. Reliability and availability	38
D3. Performance engineering	40
D4. Cost optimisation and resource efficiency (FinOps)	41
D5. Compliance-aware design (regulated data)	41

Part E. The Question Bank 42

E1. Principles and quality attributes	42
E2. Microservices	44
E3. Event-driven architecture and messaging	46
E4. SOA	48
E5. Domain-Driven Design	49
E6. CQRS, Event Sourcing and Sagas	51
E7. Anti-patterns and system health	52
E8. Technology stack selection	53
E9. System design and modeling	56
E10. Capacity planning and performance modeling	58
E11. Containers and orchestration	59
E12. Security, RBAC, encryption and threat modeling	61
E13. Reliability, availability, HA and DR	64
E14. Performance, caching and cost	65
E15. .NET deep-dive for senior architects	67
E16. Scenario and case-study questions	69
E17. Rapid-fire round (one-liners)	71

Part F. Cheat Sheets, Drills and Self-Assessment 73

F1. Pattern selection cheat sheet	73
F2. NFR to mechanism cheat sheet	73
F3. The 10-step answer structure for any design question	74
F4. Questions you should ask the evaluator (and yourself)	74
F5. Self-assessment rubric	74
F6. Four-week preparation plan	75
F7. Glossary	75

How to use this guide

Part A. Architectural Principles and Patterns

A1. Quality attributes: the real job of an architect

Functional requirements say what the system does. **Architecture is shaped by quality attributes** (non-functional requirements): how fast, how safe, how available, how changeable. A good architect converts vague wishes ("make it scalable") into measurable, testable statements called **quality attribute scenarios**.

Attribute	Vague wish	Testable scenario (stimulus, response, measure)
Scalability	Must scale	When load rises from 200 to 2,000 requests/s over 10 min, p95 latency stays under 300 ms and cost per request rises less than 20%
Availability	Always on	When one availability zone is lost, 99.95% monthly availability is kept; failover completes in under 60 s
Performance	Fast	A study search returns the first 50 rows in under 400 ms at p95 with 50 concurrent users
Security	Secure	An unauthenticated caller cannot read any patient record; every access to PHI is audit-logged within 5 s
Maintainability	Easy to change	A new tax rule is added by one team, in one service, and deployed without coordinating with others, in under 2 days
Modifiability / Deployability	Fast releases	Any service can be deployed to production independently, with zero downtime, 10+ times a day
Recoverability	No data loss	After region loss, RPO is under 5 min and RTO under 30 min, proven by a quarterly drill

Tactics: the vocabulary of design

A **tactic** is a design decision that improves one attribute. Knowing tactics lets you justify choices instead of reciting patterns.

Attribute	Typical tactics	.NET / cloud examples
Performance	Caching, async I/O, batching, indexing, pagination, CQRS read models, compression	<code>IMemoryCache</code> , HybridCache, Redis, <code>AsNoTracking</code> , response compression, <code>System.Threading.Channels</code>
Scalability	Statelessness, horizontal scale, partitioning/sharding, queue-based load leveling, autoscaling	AKS HPA/KEDA, Service Bus, Cosmos partition keys, stateless ASP.NET Core pods
Availability	Redundancy, health checks, failover, retries, circuit breaker, bulkhead, graceful degradation	Availability Zones, Front Door, Polly / <code>Microsoft.Extensions.Http.Resilience</code> , health endpoints
Security	AuthN/AuthZ, least privilege, defence in depth, input validation, encryption, audit, secrets management	Entra ID, JWT bearer, policies, Data Protection API, Key Vault, managed identity
Modifiability	Encapsulation, loose coupling, contracts, feature flags, bounded contexts, dependency inversion	Clean architecture projects, OpenAPI, Protobuf, <code>Microsoft.FeatureManagement</code>
Testability	Dependency injection, ports and adapters, contract tests, test doubles, deterministic time	<code>TimeProvider</code> , Testcontainers, WebApplicationFactory, Pact

Key idea

ATAM in one breath. The Architecture Tradeoff Analysis Method: list the top quality attribute scenarios, map them to architectural approaches, find **sensitivity points** (one decision strongly affecting one attribute), **trade-off points** (a decision affecting several attributes in opposite directions) and **risks**. Even a 2-hour lightweight version beats debating technology preferences.

A2. Architecture decision records and fitness functions

Decisions decay when they live only in heads. An **Architecture Decision Record (ADR)** is a short, immutable document: context, options, decision, consequences. Store it in the repo next to the code so it is versioned and reviewed.

```
# ADR-014: Use Azure Service Bus for inter-service events
Status: Accepted   Date: 2026-03-02
Context: Orders must notify Inventory and Billing; peak 1,500 msg/s; ordering per order id.
Options: (1) Service Bus topics (2) Kafka (3) HTTP callbacks
Decision: Service Bus topics with sessions (session id = order id).
Consequences: + managed, DLQ, sessions - 256 KB message cap (premium 100 MB), vendor lock,
              no log replay; revisit if analytics replay becomes a requirement.
```

A **fitness function** turns an architectural rule into an automated check, so the architecture cannot silently erode. In .NET, **NetArchTest** or **ArchUnitNET** can enforce layering in a unit test:

```
[Fact]
public void Domain_must_not_depend_on_infrastructure()
{
    var result = Types.InAssembly(typeof(Order).Assembly)
        .That().ResideInNamespace("Shop.Domain")
        .ShouldNot().HaveDependencyOnAny("Shop.Infrastructure", "Microsoft.EntityFrameworkCore")
        .GetResult();
    Assert.True(result.IsSuccessful, string.Join(", ", result.FailingTypeNames ?? []));
}
```

Other useful fitness functions: no cyclic references between modules, p95 latency budget in a load-test gate, container image has no critical CVEs, public API diff has no breaking change.

A3. Core design principles

Principle	Meaning	Typical violation
SOLID	Single responsibility, open/closed, Liskov, interface segregation, dependency inversion	God services; if (type == ...) chains; fat interfaces
Separation of concerns	Each unit changes for one reason	Business rules inside controllers or SQL
High cohesion, low coupling	Things that change together live together; modules talk through narrow contracts	Shared database tables between services
KISS / YAGNI	Do not build for imagined futures	Microservices for a 4-person team and one deployable
DRY (carefully)	Duplicate knowledge is bad; duplicated code across bounded contexts is often fine	Shared "Common" library coupling every service
Postel / robustness	Be liberal in what you accept, conservative in what you send (tolerant reader)	Strict deserialization that breaks on a new optional field
Design for failure	Everything remote fails; make failure explicit	Calling a service with no timeout
Evolutionary design	Keep options open; defer irreversible choices to the last responsible moment	Big up-front vendor lock-in decisions

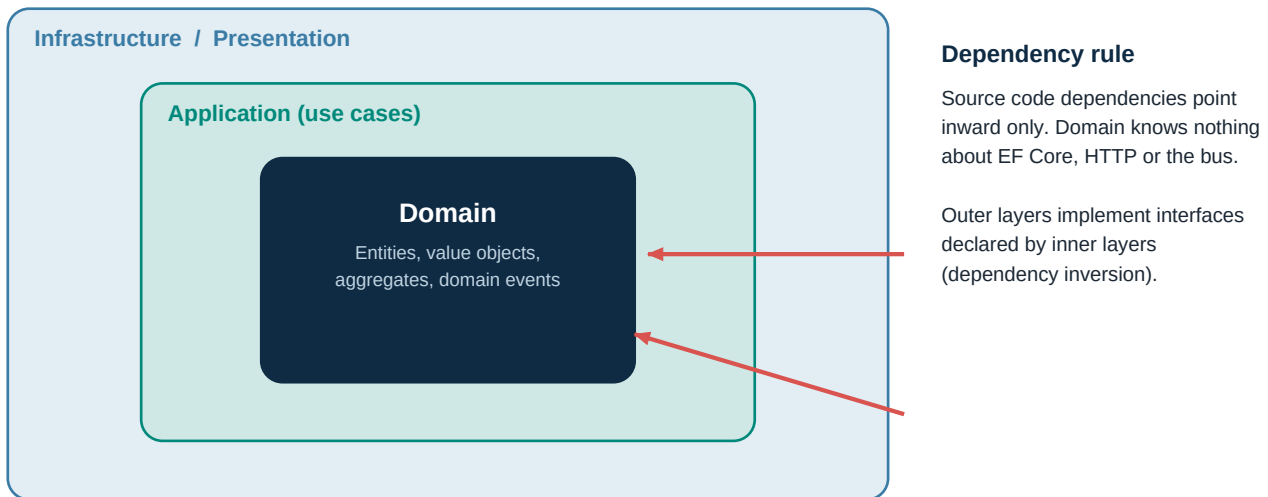
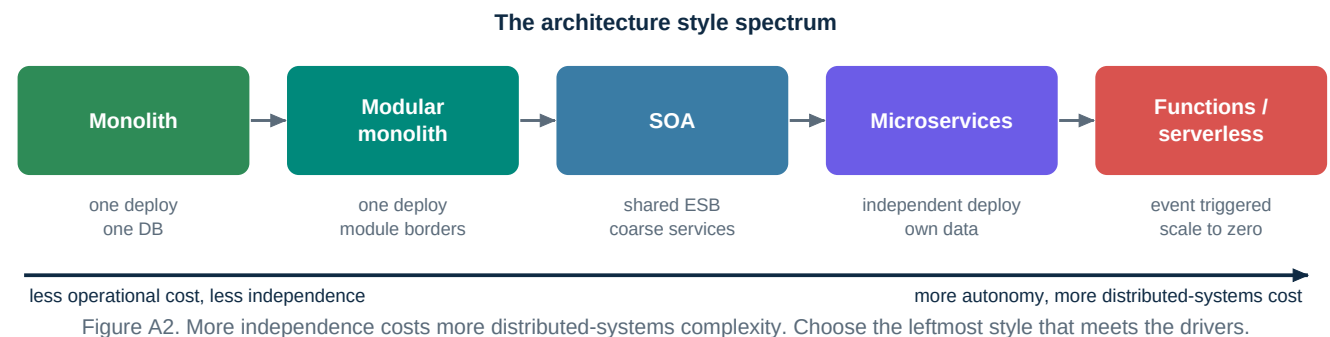


Figure A1. Clean / onion architecture: dependencies point inward toward the domain.

.NET note

A typical .NET solution for one service: **Domain** (no package references), **Application** (use cases, interfaces, MediatR-style handlers or plain handlers), **Infrastructure** (EF Core, bus, HTTP clients), **Api** (composition root: wires DI, endpoints, auth). The **composition root** is the only place that knows every concrete type. Note: MediatR moved to a commercial licence in 2025; many teams use plain handler interfaces, Wolverine, or source-generated mediators. Verify licensing before standardising.

A4. The architecture style spectrum



Style	Strengths	Weaknesses	Choose when
Monolith	Simple deploy and debug; ACID transactions; lowest latency; cheap	Whole-system scaling; release coupling; big-ball-of-mud risk	Small team, unclear domain, startup, tight deadline
Modular monolith	Strong boundaries inside one deployable; easy refactor; one DB transaction	Needs discipline; a single runtime and scaling unit	Default starting point for most new systems; precursor to extraction
SOA	Reuse of coarse business services; central governance; standard contracts	ESB becomes a bottleneck and a monolith of logic; slow change	Large enterprises with many legacy systems to integrate
Microservices	Independent deploy and scale; team autonomy; tech heterogeneity; fault isolation	Distributed data, latency, operational load, testing and debugging cost	Many teams, clear bounded contexts, mature DevOps, uneven scaling needs
Serverless / functions	Scale to zero; pay per use; low ops	Cold starts, limits, vendor lock-in, harder local dev, cost at steady high load	Spiky, event-driven, glue workloads

Watch out

Distributed monolith is the most common failure of POC-to-production microservices: services deploy together, share a database, call each other synchronously in long chains, and one outage cascades. You pay microservice costs and keep monolith coupling. Test: "Can I deploy service A today without touching B?" If not, it is not independent.

Key idea

Conway's Law says systems mirror the communication structure of the organisation that builds them. Use the **inverse Conway manoeuvre**: shape teams around the bounded contexts you want, then let the architecture follow. Team Topologies adds stream-aligned, platform, enabling and complicated-subsystem teams, and a cognitive-load limit per team.

A5. Microservices in depth

Identifying boundaries

Wrong boundaries are the costliest mistake because they cannot be fixed by refactoring one codebase. Use these heuristics together:

- **Business capability** (what the business does: Ordering, Billing, Catalogue), not technical layer.
- **Bounded context** from DDD: a place where one model and one language hold. "Customer" in Sales is not "Customer" in Support.
- **Change cadence and ownership**: things that change together belong together; one team should own a service end to end.
- **Data ownership**: if two services must update the same rows in one transaction, the boundary is wrong.
- **Event storming** workshops reveal aggregates and hot-spots quickly.

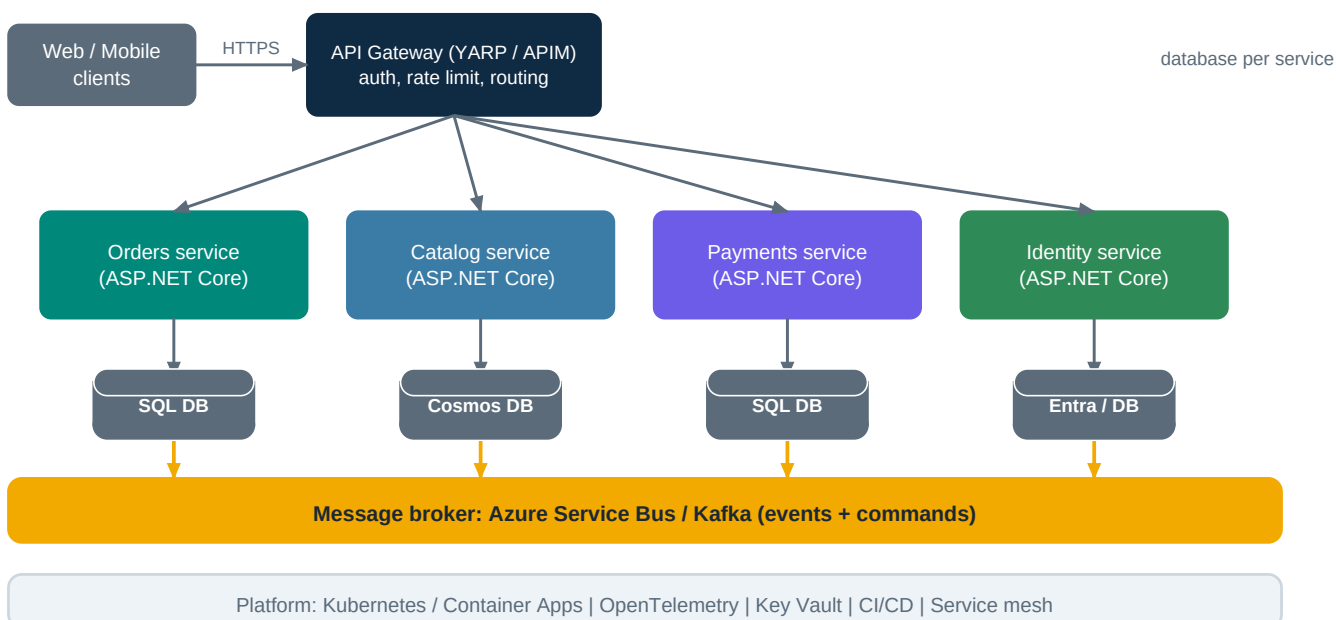


Figure A3. Reference microservices topology: gateway, database per service, asynchronous backbone, platform services.

Communication styles

Style	Protocol	Coupling	Use for	Risks
Synchronous request/response	REST/JSON, gRPC	Temporal (callee must be up)	Queries needing an immediate answer; user-facing reads	Cascading failure, latency stacking, tight availability coupling
Asynchronous messaging (commands)	Queue (Service Bus, RabbitMQ)	Weaker; sender knows receiver type	Work hand-off, load leveling	Ordering, duplicates, poison messages
Asynchronous events (pub/sub)	Topic, Kafka, Event Grid	Lowest; producer unaware of consumers	Notifications of facts, data propagation, integration	Eventual consistency, schema evolution, hard-to-trace flows
Streaming	gRPC streams, SignalR, WebSocket	Session	Live updates, large transfers	Backpressure, reconnect logic

Data management

Database per service means only the owning service reads or writes its data; others use its API or consume its events. This is what gives independent deployability, and it removes cross-service joins and ACID transactions. Replacements:

- **Local transactions plus outbox** for reliable event publishing.
- **Sagas** (chained local transactions with compensations) for multi-service workflows.
- **Read models / API composition** for queries across services (compose in a BFF or build a denormalised view fed by events).
- **Reference data replication**: keep a local, read-only copy of data you need often (e.g. product name) and update it from events.

Cross-cutting platform concerns

Concern	Typical solution	Note
Edge / routing	API gateway (YARP, Azure API Management, Ocelot), BFF per client type	Keep business logic out of the gateway
Service discovery	Kubernetes DNS / Services; Aspire service discovery in dev	No Consul needed on K8s
Configuration & secrets	Environment variables, Azure App Configuration, Key Vault with managed identity	One image, many environments
Observability	OpenTelemetry traces + metrics + logs; correlation by W3C traceparent	Logs alone cannot debug distributed flows
Resilience	Timeouts, retries with jitter, circuit breaker, bulkhead, fallback	AddStandardResilienceHandler()
Identity	OIDC / OAuth2 via Entra ID, Keycloak, Duende, OpenIddict	Validate tokens in every service, not only the gateway
Contract management	OpenAPI, Protobuf, AsyncAPI, consumer-driven contract tests (Pact)	Version deliberately

.NET note

Minimal service skeleton on current .NET with health checks, OpenTelemetry and resilience wired in:

```
var b = WebApplication.CreateBuilder(args);
b.Services.AddHealthChecks().AddSqlServer(b.Configuration["Sql"]!, tags: ["ready"]);
b.Services.AddOpenTelemetry()
    .WithTracing(t => t.AddAspNetCoreInstrumentation().AddHttpClientInstrumentation().AddOtlpExporter())
    .WithMetrics(m => m.AddAspNetCoreInstrumentation().AddRuntimeInstrumentation().AddOtlpExporter());
b.Services.AddHttpClient<IPaymentsClient, PaymentsClient>(c => c.BaseAddress = new("http://payments"))
    .AddStandardResilienceHandler();
var app = b.Build();
app.MapHealthChecks("/health/live", new() { Predicate = _ => false }); // process only
app.MapHealthChecks("/health/ready", new() { Predicate = r => r.Tags.Contains("ready") });
app.MapPost("/orders", async (CreateOrder cmd, IOrderHandler h, CancellationToken ct) =>
    Results.Created($"{orders}/{await h.HandleAsync(cmd, ct)}", null));
app.Run();
```

Deployment unit and testing strategy

Test pyramid for services: many fast **unit tests** on domain logic; **component tests** per service using **WebApplicationFactory** plus Testcontainers (real SQL, real broker); **contract tests** between services (Pact or schema compatibility checks); a few **end-to-end** journeys; plus **synthetic probes** in production. Avoid large shared test environments: they become a queue for the whole organisation.

A6. Event-driven architecture (EDA)

In EDA, components communicate by publishing **events** (immutable facts that something happened, named in the past tense: **OrderPlaced**) which other components react to. This decouples producers from consumers in time, space and knowledge.

Producers publish facts. They do not know who listens.

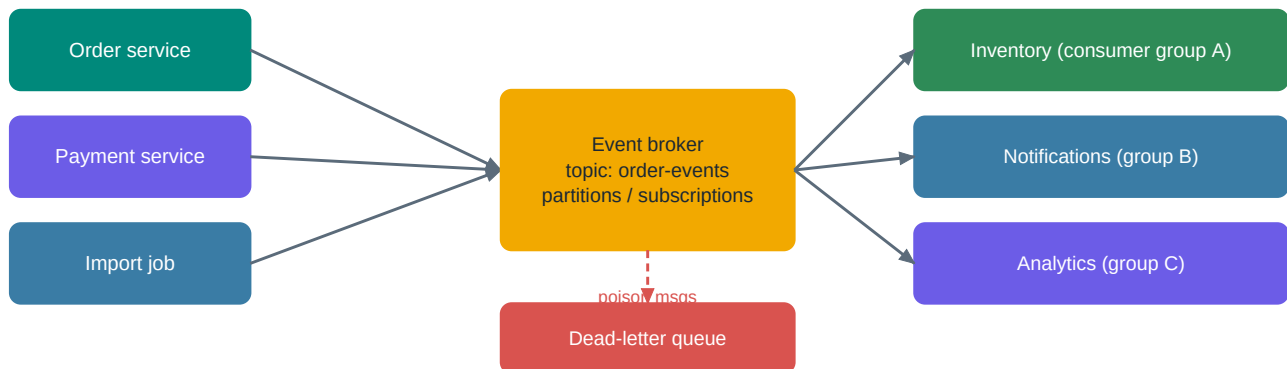


Figure A4. Pub/sub with consumer groups and a dead-letter queue.

Message kind	Intent	Example	Addressing
Command	Ask one receiver to do something	ReserveStock	Point-to-point queue
Event	Announce a fact	StockReserved	Publish/subscribe topic
Query / reply	Ask and wait for data	GetPrice / Price	Request-reply (sync or async)
Document / event-carried state transfer	Event contains the new state, so consumers need not call back	CustomerChanged{ full snapshot }	Topic; larger payload, less coupling

Delivery semantics and idempotency

Brokers offer **at-most-once** (may lose), **at-least-once** (may duplicate; the practical default) and, in limited scopes, **effectively-once** (transactions in Kafka within the Kafka ecosystem). End-to-end exactly-once across a database and a broker does not exist; you build it from **at-least-once delivery + idempotent consumers**.

- Give every message a stable **MessageId** and store processed ids (**inbox** table) in the same transaction as the side effect.
- Prefer naturally idempotent operations (**SET status = Shipped** rather than **ADD 1**).
- Use **optimistic concurrency** (version/ETag) to reject stale writers.

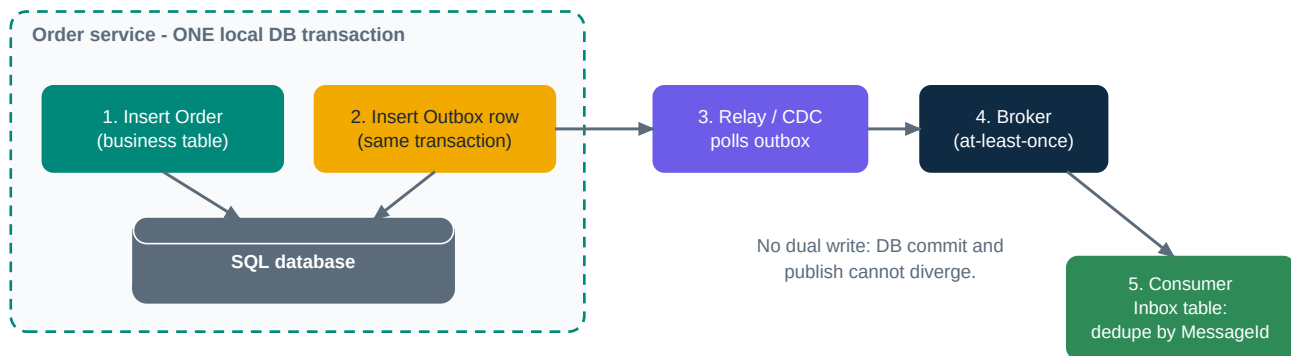


Figure A5. Transactional outbox: the only way to avoid the dual-write problem without distributed transactions.

```

// Order handler: business change and outbox row commit atomically.
await using var tx = await db.Database.BeginTransactionAsync(ct);
db.Orders.Add(order);
db.Outbox.Add(new OutboxMessage {
    Id = Guid.NewGuid(), Type = nameof(OrderPlaced), OccurredOn = clock.UtcNow(),
    Payload = JsonSerializer.Serialize(new OrderPlaced(order.Id, order.Total)) });
await db.SaveChangesAsync(ct);
await tx.CommitAsync(ct); // a separate relay publishes, marks as sent, retries on failure

// Consumer: idempotent inbox
if (await db.Inbox.AnyAsync(i => i.MessageId == msg.Id, ct)) return; // duplicate, ignore
db.Inbox.Add(new InboxMessage(msg.Id)); /* ...apply effect... */ await db.SaveChangesAsync(ct);
  
```

Ordering, partitions and scale

Global ordering does not scale. Guarantee ordering **per key** (partition key or session id = aggregate id) and design consumers to tolerate reordering across keys. In Kafka, a partition is read by one consumer of a group, so parallelism is limited by partition count; in Service Bus use sessions to get per-entity FIFO.

Schema evolution

Events outlive code. Follow **additive-only** changes (new optional fields), keep old fields until all consumers have moved, use a **schema registry** (Avro/Protobuf/JSON Schema) with compatibility modes (backward, forward, full), and apply the **tolerant reader** pattern. For breaking changes publish a new event type (**OrderPlacedV2**) and run both during migration; use **upcasters** when replaying history.

Watch out

EDA hazards: invisible control flow (hard to know who reacts), event storms, "event-carried everything" leaking internals, chatty choreography with no owner, and consumers silently stuck on a poison message. Mitigate with a published event catalogue (AsyncAPI), end-to-end tracing, DLQ alerts, and a clear owner per business process.

Broker	Model	Strengths	Watch for
Azure Service Bus	Queues, topics, sessions, DLQ, transactions	Rich enterprise features, managed, AMQP, scheduled messages	Message size limits, no replay, cost at premium tier
Azure Event Hubs / Kafka	Partitioned log, consumer groups	Very high throughput, replay, streaming, retention	Ordering only per partition, consumer lag ops, no per-message ack semantics
Azure Event Grid	Push event routing, reactive	Serverless, native Azure events, filtering	Not for high-volume ordered streams
RabbitMQ	Exchanges, queues, routing keys	Flexible routing, mature, on-prem friendly	Self-operated clustering, no native log replay (streams plugin helps)
AWS SQS/SNS, GCP Pub/Sub	Managed queue and topic	Cloud-native, scalable	Vendor specifics, semantic differences

A7. Service-Oriented Architecture (SOA)

SOA structures an enterprise as **coarse-grained, contract-first services** (classically SOAP/WSDL, XML schemas, WS-Security) governed centrally and often integrated by an **Enterprise Service Bus (ESB)** that does routing, transformation and orchestration. WCF, BizTalk and early Azure Service Bus belong to this era.

Aspect	SOA (classic)	Microservices
Service size	Large, business-process level	Small, single bounded context
Integration	Smart pipes: ESB with routing, transformation, orchestration	Smart endpoints, dumb pipes
Data	Often shared enterprise DB / canonical model	Database per service
Governance	Central, standards-driven, slower	Decentralised, platform-guided
Reuse	Maximise reuse of services	Prefer autonomy over reuse; duplicate when cheaper
Protocols	SOAP, WS-*, XML	REST, gRPC, messaging, JSON/Protobuf
Deploy	Application servers, release trains	Containers, CI/CD per service
Failure mode	ESB as central bottleneck / single point of failure	Distributed monolith, operational sprawl

Example

Migrating WCF to modern .NET. WCF server is not supported on .NET 8+. Options: (1) keep it on .NET Framework and wrap with a gateway; (2) CoreWCF (community, server-side) as a stop-gap; (3) re-implement as gRPC (contract-first Protobuf, closest in spirit and much faster) or REST with OpenAPI; (4) strangle gradually (see A9). Choose by consumer constraints: external SOAP clients push to CoreWCF; internal callers push to gRPC.

A8. Domain-Driven Design (DDD)

Strategic design

Concept	Meaning	Practical use
Ubiquitous language	One precise vocabulary shared by developers and domain experts, used in code	Class names match business terms; glossary per context
Bounded context	Explicit boundary within which a model is consistent	Candidate service / module boundary
Context map	Relationships between contexts	Shows where integration and translation are needed
Subdomains	Core (competitive advantage), supporting, generic	Invest custom design in core; buy or simplify the rest
Anti-corruption layer (ACL)	Translation layer protecting your model from a foreign one	Wrap legacy or vendor APIs
Shared kernel / customer-supplier / conformist / open host / published language	Context map relationship patterns	Pick deliberately; shared kernel creates coupling

Tactical design

Building block	Rule	C# idiom
Entity	Identity-based equality, has a lifecycle	class with Id ; behaviour methods, private setters
Value object	Immutable, equality by value, no identity	record / readonly record struct , validated in constructor
Aggregate	Cluster of objects with one root ; a transactional and invariant boundary	Root exposes methods; children reachable only through the root
Domain event	Something that happened inside the domain	record OrderConfirmed (OrderId Id); collected in root, dispatched after save
Repository	Collection-like persistence for aggregate roots	IOrderRepository in Domain; EF Core implementation in Infrastructure
Domain service	Domain logic not belonging to one entity	Stateless class in Domain
Application service	Orchestrates use case; no business rules	Handler: load, call domain, save, publish
Factory	Complex creation with invariants	Static Create method on the aggregate

Only the aggregate root is loaded/saved as one unit; invariants live inside it.

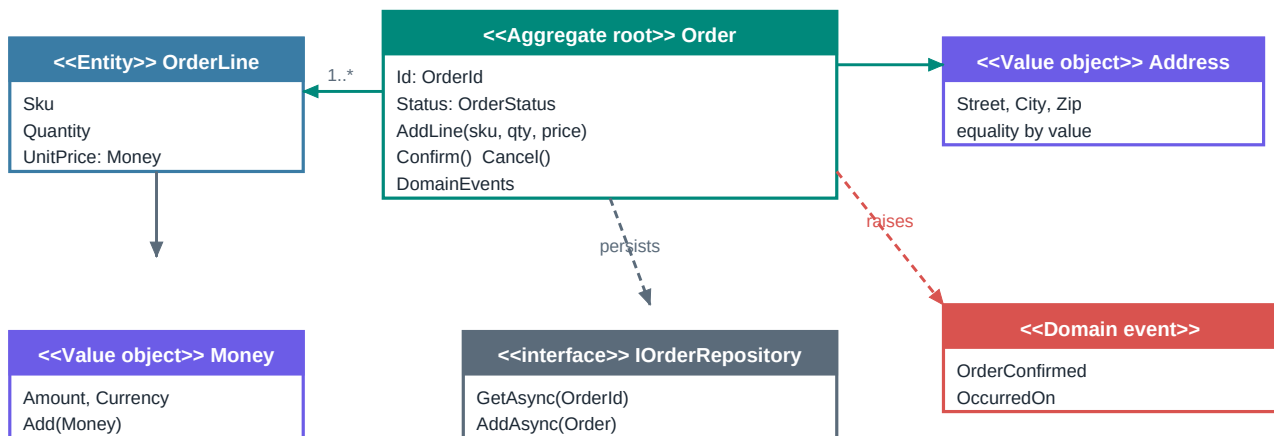


Figure A6. Order aggregate: root, child entity, value objects, repository abstraction and domain event.

```

public readonly record struct Money(decimal Amount, string Currency)
{
    public static Money Zero(string c) => new(0m, c);
    public Money Add(Money o) => o.Currency == Currency ? this with { Amount = Amount + o.Amount }
        : throw new DomainException("Currency mismatch");
}

public sealed class Order : AggregateRoot<OrderId>
{
    private readonly List<OrderLine> _lines = new();
    public OrderStatus Status { get; private set; } = OrderStatus.Draft;
    public IReadOnlyList<OrderLine> Lines => _lines;
    private Order() {} // EF Core

    public static Order Create(OrderId id, CustomerId cust) => new() { Id = id };

    public void AddLine(Sku sku, int qty, Money price)
    {
        if (Status != OrderStatus.Draft) throw new DomainException("Order is locked");
        if (qty is < 1 or > 100) throw new DomainException("Quantity out of range"); // invariant
        _lines.Add(new OrderLine(sku, qty, price));
    }

    public void Confirm()
    {
        if (_lines.Count == 0) throw new DomainException("Empty order");
        Status = OrderStatus.Confirmed;
        Raise(new OrderConfirmed(Id, _lines.Sum(l => l.Total.Amount))); // collected, dispatched after commit
    }
}
  
```

Key idea

Aggregate design rules (Vernon): (1) protect true invariants inside the boundary; (2) design small aggregates; (3) reference other aggregates by identity only, never by object reference; (4) update one aggregate per transaction, use eventual consistency (domain events) between aggregates.

Watch out

Anemic domain model: entities as property bags and all logic in "services". It looks like DDD but loses the point (invariants enforced in one place). Also avoid applying full tactical DDD to CRUD subdomains; a transaction script with EF Core is fine there.

A9. CQRS and Event Sourcing

CQRS separates the model that changes state (commands) from the model that reads it (queries). It can be applied at three strengths: (1) same DB, separate handlers/DTOs; (2) same DB, separate read tables/views; (3) separate stores with asynchronous projections. Most systems only need level 1 or 2; level 3 buys independent read scaling at the price of eventual consistency.

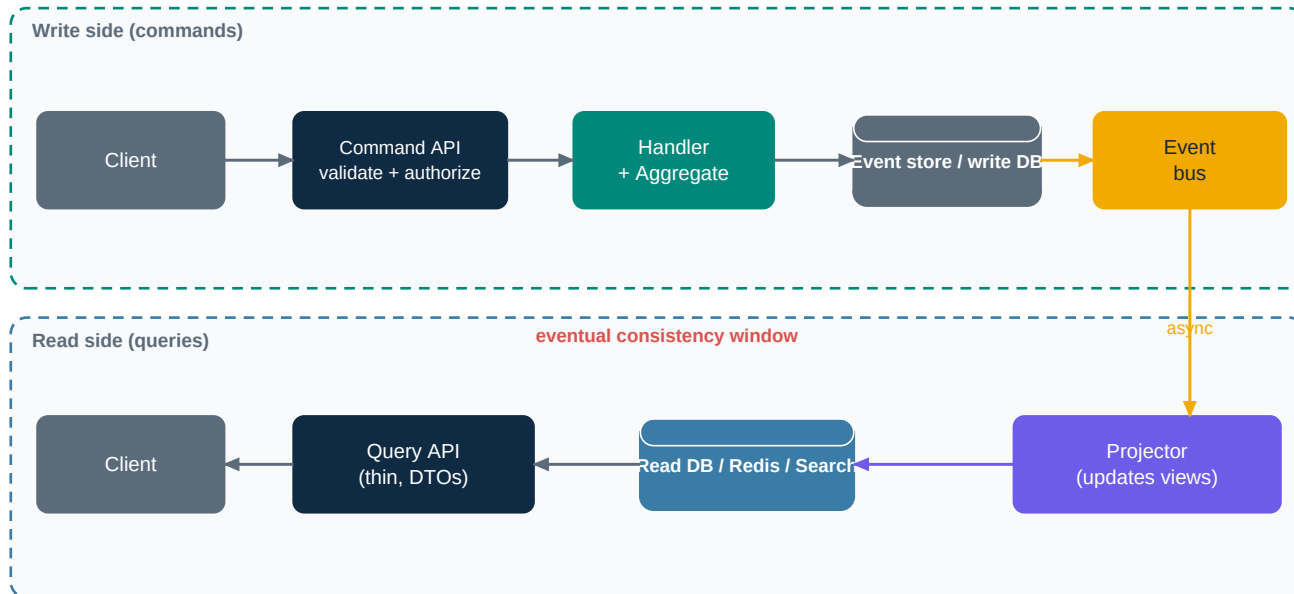


Figure A7. Full CQRS: write model is authoritative; projections feed denormalised read stores.

```
public record PlaceOrder(Guid CustomerId, List<Line> Lines); // command: intent, may be rejected
public record OrderSummary(Guid Id, string Status, decimal Total); // query result: DTO, no behaviour
```

```
// Query side: bypass the domain, project straight to DTO, no tracking.
app.MapGet("/orders/{id:guid}", async (Guid id, ReadDb db, CancellationToken ct) =>
    await db.Orders.AsNoTracking().Where(o => o.Id == id)
        .Select(o => new OrderSummary(o.Id, o.Status, o.Total)).FirstOrDefaultAsync(ct)
    is { } dto ? Results.Ok(dto) : Results.NotFound());
```

Event Sourcing (ES)

Instead of storing current state, store the **sequence of events** that produced it. State is derived by replaying events (**fold**). Benefits: complete audit trail, temporal queries ("what did we know on 1 March?"), natural integration events, ability to build new projections from history. Costs: steep learning curve, event schema versioning, eventual consistency in read models, replay time, privacy/erasure complexity.

Event stream for Order-42 (append-only)

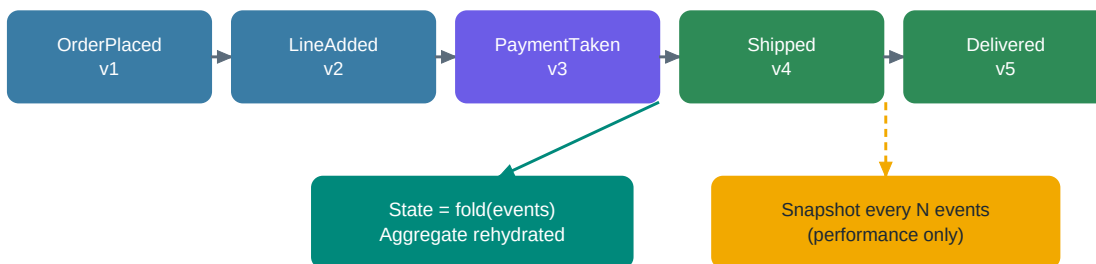


Figure A8. An event stream and rehydration; snapshots optimise load time only.

```
public sealed class Account : EventSourcedAggregate
{
    public decimal Balance { get; private set; }
    public void Withdraw(decimal amt)
    {
        if (amt > Balance) throw new DomainException("Insufficient funds");
    }
}
```

```

    Apply(new MoneyWithdrawn(amt));           // decide, then record the fact
  }
  private void On(MoneyDeposited e) => Balance += e.Amount; // evolve state: no logic, no throwing
  private void On(MoneyWithdrawn e) => Balance -= e.Amount;
}
// Append with optimistic concurrency: expectedVersion prevents lost updates.
await store.AppendAsync(streamId, expectedVersion: acct.Version, acct.UncommittedEvents);

```

Topic	Guidance
Snapshots	Optimisation only, never source of truth; take every N events or by time
Versioning	Upcast old events on read; never rewrite history; keep event types explicit and stable
GDPR / right to erasure	Crypto-shredding: encrypt personal data per subject with a key you can delete; or keep PII outside the stream with references
Projections	Idempotent, rebuildable, track checkpoint; blue/green rebuild for changes
Tooling	Marten (PostgreSQL), EventStoreDB / Kurrent, Axon-like frameworks, or hand-rolled on SQL with (StreamId, Version) unique key
When NOT to use	Simple CRUD, reporting-only domains, teams without appetite for the learning curve

Interviewer lens

Evaluators love: "CQRS does not require event sourcing, and event sourcing almost always implies CQRS. Why?" Answer: ES stores events, which are poor for querying, so you need separate read models; but CQRS with a normal relational write model is perfectly valid and far more common.

A10. Distributed transactions: sagas

Two-phase commit (2PC/MSDTC) blocks, couples availability and is unsupported across most cloud brokers and databases. Use a **saga**: a sequence of local transactions where each step publishes an event or command that triggers the next, and each step has a **compensating action** for rollback semantics.

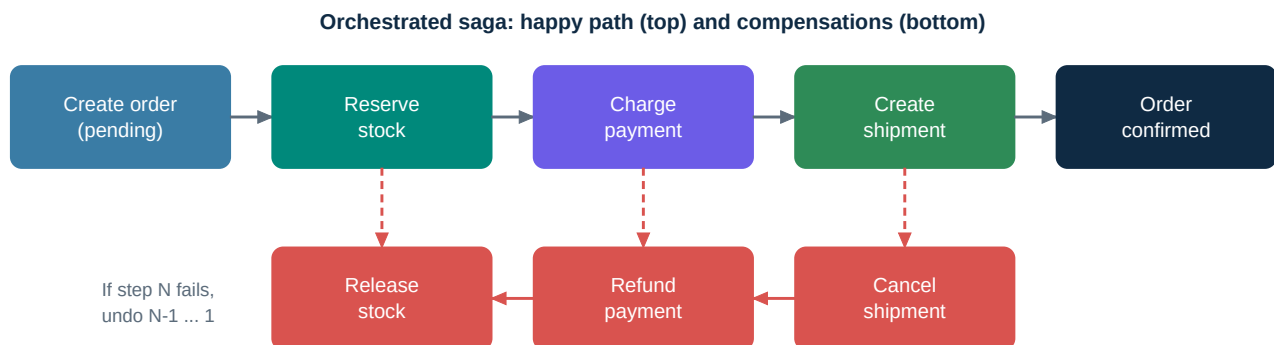


Figure A9. Orchestrated saga with compensations.

	Choreography	Orchestration
Control	Each service reacts to events	A central saga coordinator sends commands
Pros	No central point; loose coupling; simple for 2-3 steps	Process is explicit, observable and testable; easier to add steps
Cons	Flow is implicit and hard to follow; cyclic dependencies creep in	Orchestrator can attract logic; extra component
.NET tools	Plain events via MassTransit / Wolverine / NServiceBus	MassTransit state machines, NServiceBus sagas, Durable Functions, Dapr Workflow, Temporal

Watch out

Compensation is **semantic undo**, not rollback. You cannot unsend an email; you send a correction. Some steps are **non-compensable**, so place them last (the "pivot" step) or make them retryable only. Sagas lack isolation: other transactions can observe intermediate states. Countermeasures: semantic locks (**Pending** status), commutative updates, re-reading values, version checks.

A11. Evolving a legacy system: the strangler fig

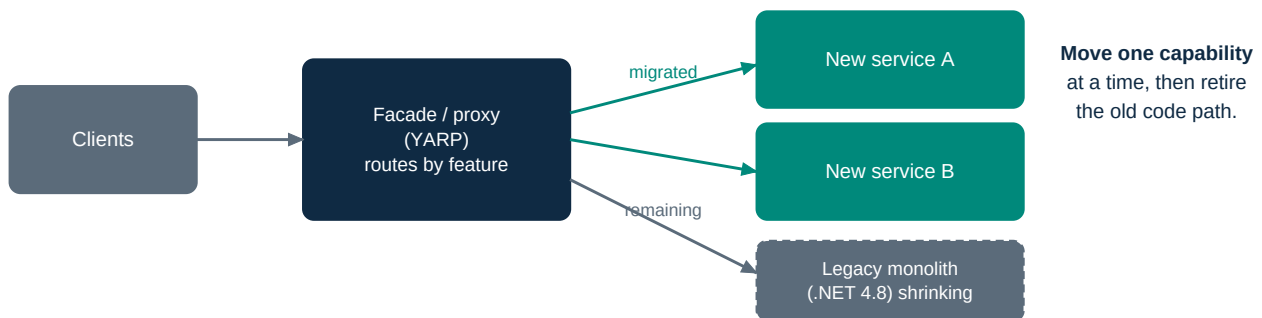


Figure A10. Incremental replacement behind a routing facade.

Never "big-bang rewrite". Put a facade (YARP) in front, route one capability at a time to new services, keep data in sync with an anti-corruption layer or change-data-capture while both exist, and delete the old code path when traffic is zero. Measure progress by traffic share and by lines of legacy code removed.

A12. Architectural anti-patterns and mitigations

Anti-pattern	Symptoms	Mitigation
Distributed monolith	Lockstep deploys; shared DB; chain of sync calls	Redraw boundaries by business capability; one DB per service; async events; independent pipelines
Big ball of mud	No structure; everything references everything	Modular monolith, enforce layering with fitness functions, strangler
God service / God class	One service everyone depends on; endless merge conflicts	Split by bounded context; extract capabilities
Nano-services	Hundreds of tiny services; every feature touches five	Merge services that always change together
Chatty services	N+1 remote calls; high latency	Coarser APIs, batching, read models, BFF aggregation
Shared database / integration DB	Schema change breaks others silently	Ownership, expose via API/events, views as temporary bridge
Golden hammer	Same tech for everything (Kafka, K8s, microservices)	Decision records with alternatives; fit-for-purpose
Anemic domain model	Logic in services; entities are DTOs	Behaviour on aggregates; enforce invariants in root
Distributed transaction coupling	2PC across services; timeouts	Sagas, outbox, idempotency
Retry storm	Immediate retries multiply load during outage	Exponential backoff with jitter, retry budgets, circuit breaker
Sync everywhere	Availability = product of all dependencies	Queue-based load leveling, async messaging, timeouts, fallbacks

Anti-pattern	Symptoms	Mitigation
Shared "Common" library	Every change forces all services to upgrade	Share only contracts/primitives; allow duplication; version carefully
Cargo-cult CQRS / ES	Complexity with no scaling or audit need	Start with simple CRUD; adopt where the driver exists
Resume-driven design	Tech chosen for novelty	Tie each choice to a quality attribute scenario
Config / secrets in code	Connection strings in repos	Key Vault, managed identity, config providers
Hidden single point of failure	One gateway, one queue, one region, one person	Redundancy review; chaos testing; bus-factor review

Part B. Technology Stack Selection

B1. A repeatable selection method

Technology choice is a decision under uncertainty. Use a **weighted decision matrix** tied to the quality attribute scenarios, then validate the top two with a time-boxed spike (a proof of concept with success criteria, not a demo).

Criterion	Weight	Option A: AKS	Option B: Container Apps	Option C: App Service
Low operational effort (5 = least effort)	25%	2	4	5
Scaling flexibility (KEDA, custom)	20%	5	4	3
Ecosystem / portability	15%	5	3	2
Cost at our expected load	15%	3	4	4
Team skills today	15%	2	3	5
Compliance / network control	10%	5	3	3
Weighted score (out of 5)		3.50	3.60	3.80

Key idea

The scores are illustrative. The value is in the **argument**: change the weights (say compliance to 30%) and the winner flips. Always run the sensitivity check and record the result in an ADR. Include **exit cost** (how hard to leave?) as an explicit criterion.

B2. Languages and frameworks: trade-offs

Stack	Performance	Dev speed	Ecosystem	Cost / ops	Best fit
.NET (C#, ASP.NET Core)	Excellent (top-tier web benchmarks), AOT option	High; strong tooling, one language front to back with Blazor	Rich, enterprise, Azure-first	Moderate memory; great container story	Enterprise APIs, Windows heritage, regulated domains, high throughput services
Java / Kotlin (Spring Boot, Quarkus)	Excellent, mature JIT, GraalVM native	High	Largest enterprise ecosystem	Higher memory unless native	Large orgs, big-data adjacency, existing JVM skills
Node.js / TypeScript	Good for I/O; single-thread CPU limits	Very high; shared types with UI	Huge npm; supply-chain risk	Cheap to start; many small deps	BFFs, real-time, startups
Go	Excellent, tiny footprint, fast start	Medium; simple language	Cloud-native tooling	Very cheap	Infra tools, high-concurrency network services
Python	Slower for CPU; fine with native libs	Very high	Dominant in ML/data	Needs more instances for CPU-bound	AI/ML services, scripting, data pipelines
Rust	Best-in-class, no GC	Slow ramp; steep learning curve	Growing	Lowest runtime cost	Latency-critical, safety-critical components

.NET note

Why .NET for a Microsoft-centred enterprise: one skill set from desktop (WPF) to cloud; first-class Azure SDKs and managed identity; very good async I/O; analyzers and strong typing; LTS cadence (even-numbered releases supported for 3 years: .NET 8 LTS, .NET 10 LTS). **Polyglot is fine** if each service has a defensible reason and the platform (CI, observability, security scanning) supports it; otherwise the cognitive load outweighs the benefit.

.NET-specific choices that regularly appear in evaluations

Decision	Options	Guidance
API style in ASP.NET Core	Controllers vs Minimal APIs	Minimal APIs: less ceremony, lower overhead, great for microservices (route groups, endpoint filters, typed results). Controllers: mature conventions, large teams, filters/model binding features. Both share the pipeline; mixing is allowed.
Inter-service protocol	REST/JSON, gRPC, GraphQL	gRPC for internal low-latency/strongly typed calls; REST for public/partner APIs and cacheability; GraphQL for flexible client-driven aggregation (BFF), watch N+1 and cost control.
Data access	EF Core, Dapper, ADO.NET	EF Core for productivity, change tracking, migrations; Dapper/raw SQL for hot paths and reporting; mix via CQRS (EF writes, Dapper reads).
Mediator / messaging libs	MediatR, Wolverine, MassTransit, NServiceBus, Rebus, Brighter	Check licensing (MediatR and recent MassTransit lines moved to commercial terms); evaluate maintainability and exit cost. Plain interfaces are often enough.
Resilience	Polly v8 / Microsoft.Extensions.Http.Resilience	Use the standard handler for HttpClient; add custom pipelines for non-HTTP.
Identity server	Entra ID, Duende IdentityServer, OpenIddict, Keycloak	Buy where possible (Entra / External ID); Duende is commercial; OpenIddict is a library; Keycloak is self-hosted ops.
Hosting model	Self-contained, framework-dependent, Native AOT	AOT cuts startup/memory (great for serverless) but restricts reflection; check library support.
Local dev orchestration	Aspire (formerly .NET Aspire), Docker Compose, Tilt	Aspire gives an app model, service discovery and telemetry dashboard for dev; deployment targets include Azure Container Apps and Kubernetes manifests.
Distributed app runtime	Dapr, none	Dapr abstracts state, pub/sub, bindings and workflow via sidecars; adds a moving part; useful for polyglot or portability goals.

B3. Data store selection

Need	Pick	Why	Trade-off
Transactional business data, relations	Azure SQL / SQL Server / PostgreSQL	ACID, joins, mature tooling, EF Core	Vertical scaling limits; sharding is hard
Global scale, flexible schema, single-digit ms	Cosmos DB	Partitioned, multi-region writes, tunable consistency	Partition key design is critical; RU cost model; limited joins
Cache, sessions, rate limits, locks	Redis (Azure Cache for Redis / Managed Redis)	Sub-ms, rich structures	Volatile; cache invalidation; not a system of record
Full-text and faceted search	Azure AI Search / Elasticsearch	Relevance, facets, vectors	Eventually consistent copy; reindexing
Large binaries (images, documents)	Blob storage (+ CDN)	Cheap, durable, tiered	No query; metadata lives elsewhere

Need	Pick	Why	Trade-off
Time series / telemetry	Azure Data Explorer, Timescale, InfluxDB	Compression, time-window queries	Different query model
Analytics	Synapse / Fabric / Databricks / Snowflake	Columnar scale	Latency; keep OLTP separate (no analytics on OLTP)
Event log / streaming	Kafka / Event Hubs	Replay, throughput	Not a query store

Key idea

Polyglot persistence is justified per service by access pattern, not by fashion. Each extra datastore costs backup, security, monitoring and skills. Default to one relational engine, add others only on evidence.

CAP, PACELC and consistency models

CAP: under a network partition a system must choose consistency or availability. **PACELC** adds: even without partition (Else) you trade latency for consistency. Practical models, strongest to weakest: linearizable, sequential, causal, session, bounded staleness, eventual. Cosmos DB exposes five levels (strong, bounded staleness, session, consistent prefix, eventual); session is the common default because users see their own writes.

B4. Cloud vendor selection

Capability	Azure	AWS	GCP
Managed Kubernetes	AKS	EKS	GKE (very polished)
Serverless containers	Container Apps	ECS Fargate / App Runner	Cloud Run
Functions	Azure Functions (Durable)	Lambda (Step Functions)	Cloud Functions
Messaging	Service Bus, Event Hubs, Event Grid	SQS, SNS, Kinesis, MSK, EventBridge	Pub/Sub
Relational	Azure SQL, PostgreSQL Flexible	RDS, Aurora	Cloud SQL, AlloyDB, Spanner
NoSQL	Cosmos DB	DynamoDB	Firestore, Bigtable
API gateway	API Management, Application Gateway, Front Door	API Gateway, CloudFront	Apigee, API Gateway
Identity	Entra ID	IAM, Cognito	Cloud Identity
Secrets / keys	Key Vault + managed identity	Secrets Manager, KMS	Secret Manager, KMS
Observability	Azure Monitor, App Insights	CloudWatch, X-Ray	Cloud Operations

Selection driver	Usually favours
Existing Microsoft licences (Hybrid Benefit), Entra ID, .NET shop, Windows workloads	Azure
Broadest service catalogue, mature ecosystem, many regions	AWS
Data/AI analytics, Kubernetes-first, global network	GCP
Regulatory data residency / sovereign requirements	Whichever has compliant regions; consider sovereign clouds
Avoid lock-in	Kubernetes + open standards (PostgreSQL, Kafka, OpenTelemetry) but accept that "portable" still costs effort

Watch out

Multi-cloud is not free resilience. It multiplies skills, tooling, security surface and egress cost. Use it for a specific reason (regulatory, acquisition, negotiated risk) and keep a **cloud-agnostic core** only where portability is worth the abstraction cost. Many teams get more reliability from multi-region in one cloud than from multi-cloud.

.NET note

Lift-and-shift reality check. Moving a VM-hosted ASP.NET 4.8 / WPF estate straight to Azure VMs buys datacentre exit but none of the cloud-native benefits. A usual staged path: rehost (VMs) -> replatform (App Service, Azure SQL MI, Service Bus) -> refactor (containers, .NET 8+/10, modular decomposition). Tools: Azure Migrate, .NET Upgrade Assistant, Application Insights dependency maps.

B5. Containerization with Docker

A **container** packages the app and its user-space dependencies and runs as an isolated process sharing the host kernel (namespaces for isolation, cgroups for limits). It is lighter than a VM and gives environment parity. An **image** is an immutable, layered artefact; a **registry** (ACR) stores it; a **tag/digest** identifies it. Deploy by digest for immutability.

```
# Multi-stage Dockerfile: small, cached, non-root
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY ["Orders.Api/Orders.Api.csproj", "Orders.Api/"]
RUN dotnet restore "Orders.Api/Orders.Api.csproj" # layer cached until csproj changes
COPY . .
RUN dotnet publish "Orders.Api/Orders.Api.csproj" -c Release -o /app --no-restore /p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final # or the -chiseled variant: no shell, smaller
WORKDIR /app
COPY --from=build /app .
USER $APP_UID # non-root (built-in app user)
EXPOSE 8080
ENTRYPOINT ["dotnet", "Orders.Api.dll"]
```

Practice	Reason
Multi-stage builds, chiseled / distroless / Alpine bases	Smaller attack surface and pull time; fewer CVEs
Run as non-root, read-only filesystem, drop capabilities	Limits blast radius of a compromise
One process per container, logs to stdout	Orchestrator and log agent handle the rest
Never bake secrets into images	Images are copied and cached widely; inject at runtime
Scan images (Defender for Containers, Trivy, Gype), sign (Notation, Cosign), generate SBOM	Supply-chain integrity
Set memory/CPU requests and limits; tune .NET GC for containers	.NET honours cgroup limits; Server GC vs Workstation GC and DOTNET_GCHeapHardLimit matter
Graceful shutdown (SIGTERM) and HostOptions.ShutdownTimeout	Drain in-flight work during rolling updates

B6. Orchestration: Kubernetes and alternatives

An orchestrator schedules containers across machines, restarts failures, scales, rolls out versions and wires networking. **Kubernetes (K8s)** is the de facto standard: you declare desired state in YAML and controllers continuously reconcile reality toward it.

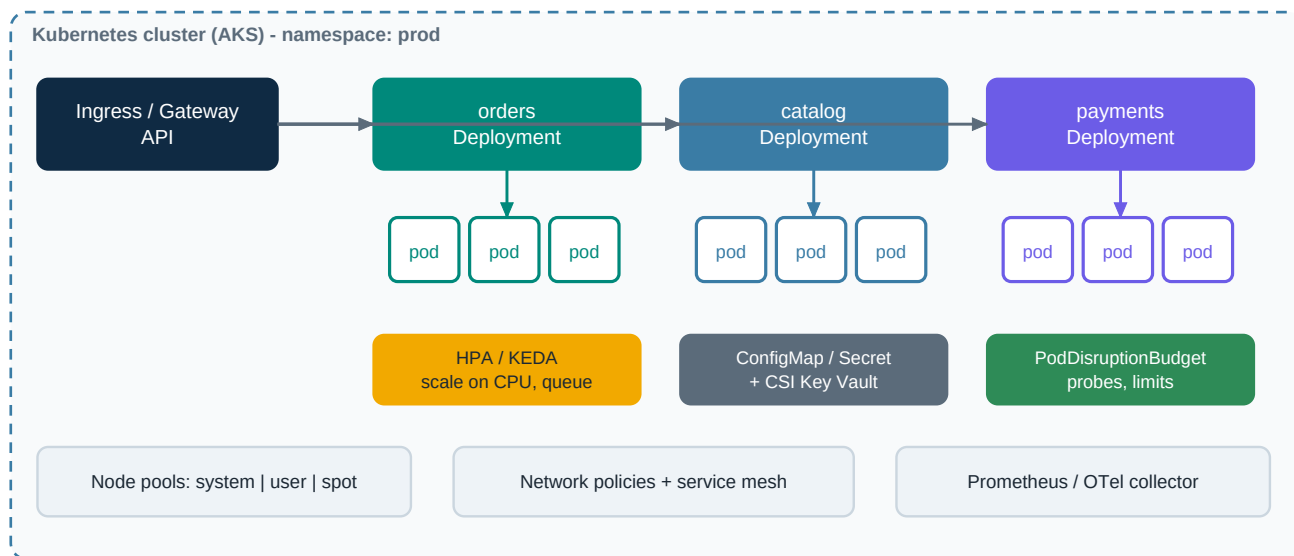


Figure B1. A cluster with ingress, deployments, pods, autoscaling, config and operations concerns.

K8s object	Role
Pod	Smallest unit; one or more containers sharing network/storage
Deployment / ReplicaSet	Declarative stateless replicas, rolling updates and rollback
StatefulSet	Stable identity and storage for stateful workloads (prefer managed PaaS for data)
Service / Ingress / Gateway API	Stable virtual IP and DNS; L7 routing and TLS
ConfigMap / Secret	Configuration (Secrets are only base64 by default; use CSI driver + Key Vault, enable encryption at rest)
HPA / VPA / KEDA / Cluster Autoscaler	Scale pods on metrics; scale on events (queue depth); add or remove nodes
PodDisruptionBudget, probes, affinity, topology spread	Keep availability during maintenance and zone loss
NetworkPolicy, RBAC, Pod Security Standards	Segmentation and least privilege
Helm / Kustomize / GitOps (Flux, Argo CD)	Packaging, environment overlays, pull-based delivery

```

apiVersion: apps/v1
kind: Deployment
metadata: { name: orders, namespace: prod }
spec:
  replicas: 3
  strategy: { type: RollingUpdate, rollingUpdate: { maxUnavailable: 0, maxSurge: 1 } }
  selector: { matchLabels: { app: orders } }
  template:
    metadata: { labels: { app: orders } }
    spec:
      topologySpreadConstraints:
        - { maxSkew: 1, topologyKey: topology.kubernetes.io/zone, whenUnsatisfiable: ScheduleAnyway,
            labelSelector: { matchLabels: { app: orders } } }
      containers:
        - name: api
          image: myacr.azurecr.io/orders@sha256:3f9a... # deploy by digest
          ports: [{ containerPort: 8080 }]
          resources: { requests: { cpu: 250m, memory: 256Mi }, limits: { memory: 512Mi } }
          readinessProbe: { httpGet: { path: /health/ready, port: 8080 }, periodSeconds: 5 }
  
```

```

livenessProbe: { httpGet: { path: /health/live, port: 8080 }, periodSeconds: 10 }
startupProbe: { httpGet: { path: /health/live, port: 8080 }, failureThreshold: 30 }
securityContext: { runAsNonRoot: true, readOnlyRootFilesystem: true, allowPrivilegeEscalation: false }

```

Platform	Operational effort	Control	Good for
Azure Kubernetes Service (AKS)	High (even managed control plane)	Maximum	Many services, custom networking, service mesh, platform team exists
Azure Container Apps	Low (K8s hidden; KEDA and Dapr built in)	Medium	Microservices without a platform team; scale-to-zero
App Service (containers)	Very low	Low-Medium	A few web APIs, lift-and-shift, simple scaling
Azure Functions	Very low	Low	Event glue, spiky jobs
Service Fabric	High, legacy direction	High	Existing SF estates; new work usually elsewhere
VMs / VM Scale Sets	High	Full OS	Legacy Windows apps (WPF/IIS), lift-and-shift

Interviewer lens

"Do you actually need Kubernetes?" is a favourite trap. A strong answer: "Only if we have enough services and teams to amortise the platform cost, need its extensibility (operators, custom scheduling, service mesh), or have portability requirements. Otherwise Container Apps or App Service give most benefits at a fraction of the operational load."

B7. Frontend, mobile and hybrid

Option	Strengths	Weaknesses	Fits
Angular	Opinionated, TypeScript-first, enterprise structure, DI	Heavier, steeper learning	Large enterprise SPAs and teams
React (+ Next.js)	Huge ecosystem, SSR/SSG	Many choices, ecosystem churn	Product web apps, content sites
Blazor (Server / WASM / Auto, Web App)	C# end to end, share models	WASM payload size; Server needs persistent connection	.NET-heavy teams, internal tools
MAUI / .NET MAUI Blazor Hybrid	Cross-platform native with C#	Smaller ecosystem than React Native/Flutter	Line-of-business mobile and desktop for .NET shops
React Native / Flutter	Cross-platform, large communities	Different language/toolchain	Consumer mobile with shared codebase
Native (Swift / Kotlin)	Best platform UX and performance	Duplicate effort	Performance or platform-feature intensive apps
PWA	One codebase, installable, offline via service workers	Limited device APIs on iOS	Light mobile needs

Backend for Frontend (BFF): one backend per client type (web, mobile, partner) owns aggregation, shaping and client-specific auth (for SPAs prefer BFF with cookie sessions and tokens kept server-side, avoiding tokens in browser storage). **Mobile-specific architecture:** offline-first with local store and sync (conflict policy: last-writer-wins, merge, or CRDT), push notifications, token refresh via PKCE, certificate pinning for high-risk apps, small payloads and delta sync, backward-compatible APIs because old app versions live for years.

B8. Optimising for cloud-native, mobile and hybrid

Target	Architectural emphasis
Cloud-native	Stateless, 12-factor, containers, IaC (Bicep, Terraform), managed services over self-hosting, autoscale, observability, immutable infrastructure, GitOps
Mobile-first	Latency and bandwidth-aware APIs, caching/ETags, pagination, offline sync, versioning tolerance, push, secure token storage, battery-friendly
Hybrid (on-prem + cloud)	Connectivity (ExpressRoute / VPN), identity federation, data gravity and residency, Azure Arc, edge processing, consistent tooling, async buffering for unreliable links (store-and-forward)
Edge / on-site systems (e.g. hospital sites)	Local autonomy during WAN loss, local cache and queue, eventual sync to central, privacy-by-design at the site

Key idea

Twelve-factor essentials for .NET services: config in environment, stateless processes, backing services as attached resources, port binding, disposability (fast start, graceful stop), dev/prod parity, logs as streams, admin tasks as one-off processes. Anything that stores state in memory or on local disk blocks horizontal scaling.

B9. Cost, performance and scalability trade-off cheat sheet

Choice	Performance	Cost	Scalability	Dev speed
Monolith on App Service	High (in-process)	Low	Scale up/out as one	Highest
Microservices on AKS	Network hops add latency	Higher (platform + people)	Per-service, very high	Slower initially, faster at team scale
Serverless functions	Cold starts; variable	Lowest at low volume, can exceed at steady high volume	Automatic, huge	High for glue
Managed PaaS DB	Good	Higher unit price, lower ops	Tier-limited	High
Self-managed DB on VMs	Tunable	Lower license/unit, higher people	Manual	Low
gRPC vs REST internally	gRPC 2-10x less overhead (binary, HTTP/2)	Lower bandwidth	Good	Needs tooling for contracts
Reserved instances / savings plans	n/a	30-60% saving for steady load	Commitment risk	n/a

Part C. System Design and Modeling

C1. High-level versus low-level design

	High-Level Design (HLD)	Low-Level Design (LLD)
Question answered	What are the parts and how do they interact?	How exactly does each part work internally?
Artefacts	Context and container diagrams, deployment view, data-flow, technology choices, NFR mapping, ADRs	Class and sequence diagrams, API schemas, DB schema, state machines, algorithms, error handling
Audience	Architects, stakeholders, ops, security	Developers, testers
Stability	Changes slowly	Changes with every sprint
Typical failure	Hand-waving boxes with no NFR numbers	Over-specifying up front; docs drift from code

A complete design walk-through in an evaluation follows a rhythm: **clarify requirements and scale -> estimate -> draw the HLD -> deep-dive the riskiest component -> data model and APIs -> failure handling and scaling -> trade-offs and alternatives**. Spending the first five minutes on assumptions and numbers separates architects from coders.

C2. Core components, data flows and integration points

Capture every design in three views: **structure** (components), **behaviour** (how they collaborate over time), and **deployment** (where they run). Then list each **integration point** with: protocol, contract owner, authentication, expected volume, failure behaviour, and data classification. Integration points are where most incidents are born.

Integration pattern (EIP)	Use	Example
Content-based router	Route by message content	Route imaging orders by modality to different processors
Message translator / canonical model	Convert between formats	HL7 v2 to FHIR resource
Aggregator / scatter-gather	Combine replies from many services	Price comparison across providers
Claim check	Pass reference instead of big payload	Message holds a blob URL, not 200 MB of data
Competing consumers	Scale processing of a queue	Many worker pods drain one queue
Anti-corruption layer	Isolate foreign model	Adapter around a legacy SOAP service
Gateway aggregation / BFF	Fewer round trips for clients	One mobile call replaces six

C3. API design

REST design checklist

Topic	Good practice
Resources	Nouns, plural, hierarchical only where ownership is real: <code>/orders/{id}/lines</code>
Methods	GET safe and idempotent; PUT replace (idempotent); PATCH partial; POST create/non-idempotent action; DELETE idempotent
Status codes	200/201 with <code>Location</code> /202 for async/204; 400 vs 422 validation; 401 vs 403; 404; 409 conflict; 412 precondition failed; 429 with <code>Retry-After</code> ; 5xx only for server faults
Errors	RFC 9457 problem details: <code>application/problem+json</code> with <code>type</code> , <code>title</code> , <code>status</code> , <code>detail</code> , <code>traceId</code>
Pagination	Cursor/keyset for large or changing sets (stable, fast); offset only for small static data; return <code>next</code> link
Filtering/sorting	Whitelist fields; bound page size; never allow arbitrary expression injection
Idempotency	<code>Idempotency-Key</code> header on POST that creates money-affecting resources; store key + result for 24h
Concurrency	ETag + <code>If-Match</code> for optimistic concurrency on updates
Versioning	URL (<code>/v2</code>) or header; additive changes are non-breaking; deprecate with <code>Sunset</code> header; support N-1
Long-running ops	202 Accepted + status resource (<code>/operations/{id}</code>) or webhook; do not hold HTTP connections
Caching	<code>Cache-Control</code> , ETag, conditional GET; CDN for public GETs
Documentation	OpenAPI document generated from code and linted; examples; <code>Microsoft.AspNetCore.OpenApi</code> on .NET 9+

POST /v1/orders HTTP/1.1
 Idempotency-Key: 6f1c9a40-6c1b-4a5f-b0d2-93a7a2d1e0aa
 Authorization: Bearer eyJ...
 Content-Type: application/json

```
{ "customerId": "c-77", "lines": [ { "sku": "A-100", "qty": 2 } ] }
```

HTTP/1.1 201 Created
 Location: /v1/orders/9b2e... ETag: "1"

HTTP/1.1 422 Unprocessable Entity
 Content-Type: application/problem+json
 { "type": "https://errors.example/out-of-stock", "title": "Out of stock", "status": 422,
 "detail": "SKU A-100 has 1 unit available", "traceId": "00-4bf92f35..." }

	REST	gRPC	GraphQL	Async messaging
Contract	OpenAPI	Protobuf (strict, codegen)	Schema (SDL)	AsyncAPI / schema registry
Payload	JSON text	Binary, compact	JSON, client-shaped	JSON/Avro/Protobuf
Transport	HTTP/1.1 or 2	HTTP/2 (browser needs gRPC-Web)	HTTP	AMQP / Kafka
Strength	Ubiquitous, cacheable, tooling	Speed, streaming, deadlines, strong typing	Flexible reads, fewer round trips	Decoupling, buffering, fan-out
Weakness	Over/under-fetching	Not browser-native, harder to debug by hand	N+1, complexity, caching, authZ per field	Eventual consistency, debugging
Use for	Public/partner APIs	Internal service-to-service	BFF for rich UIs	Integration and workflows

.NET note

gRPC in .NET: always set **deadlines** (`CallOptions(deadline:)`) and propagate cancellation; use `GrpcChannel` as a long-lived singleton (HTTP/2 multiplexing) and client-side load balancing on Kubernetes (headless services or a mesh) because L4 load balancers pin one connection to one pod.

C4. UML essentials

UML is a **communication tool**, not a deliverable. Draw the smallest diagram that answers a question. Evaluators expect fluency in three:

- **Class diagram** (structure): classes, attributes, operations, associations, multiplicity, inheritance, composition (strong ownership, diamond filled) versus aggregation, interfaces and dependencies. See Figure A6 for a DDD-flavoured example.
- **Sequence diagram** (interaction over time): who calls whom, sync vs async messages, activation bars, `alt/opt/loop` fragments.
- **Activity diagram** (workflow): actions, decisions, forks/joins, swimlanes for responsibilities.

Also useful: **state machine** (order lifecycle), **component** and **deployment** diagrams.

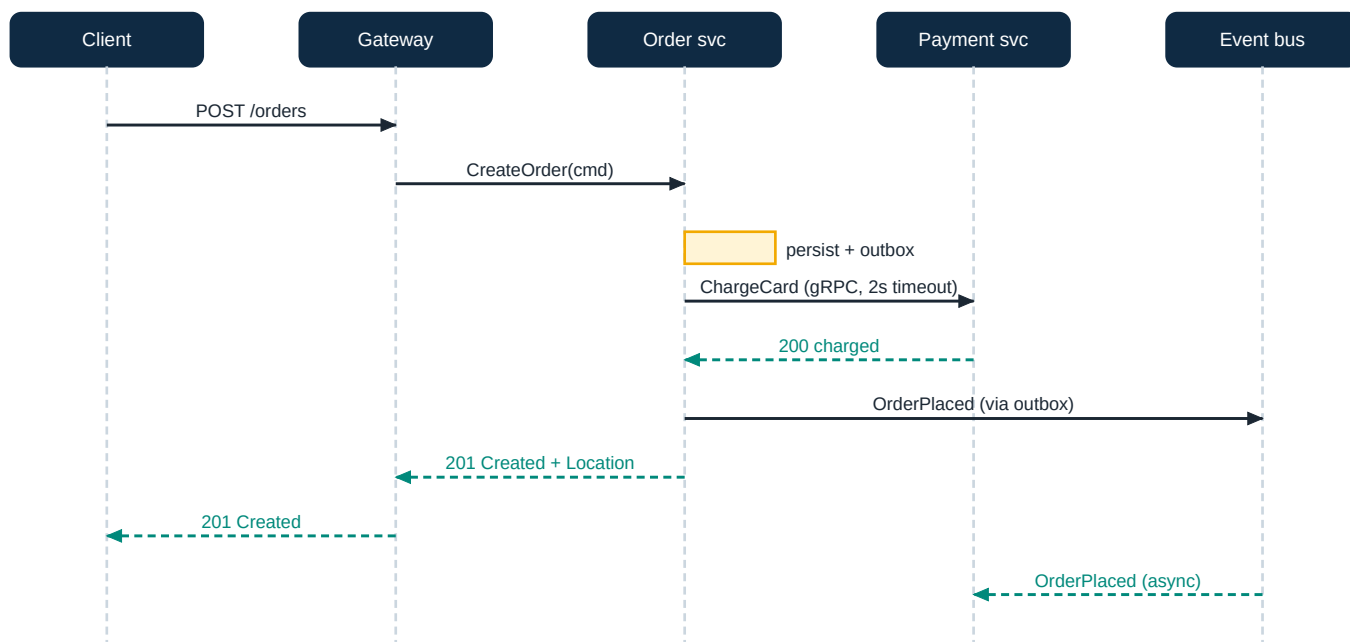


Figure C1. Sequence diagram: placing an order (solid = request, dashed = reply/async). Note the outbox write before publishing and the timeout on the payment call.

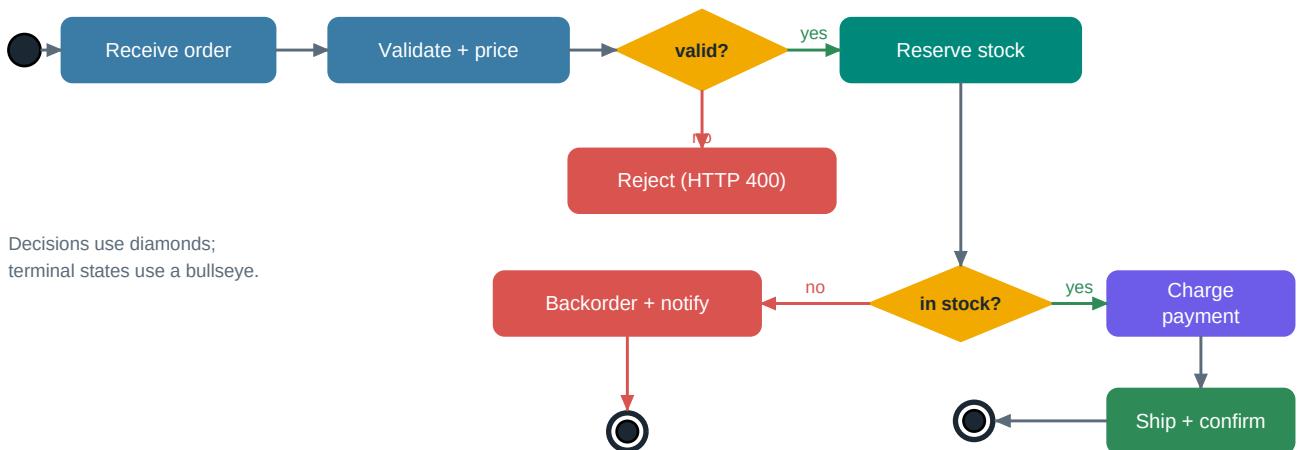


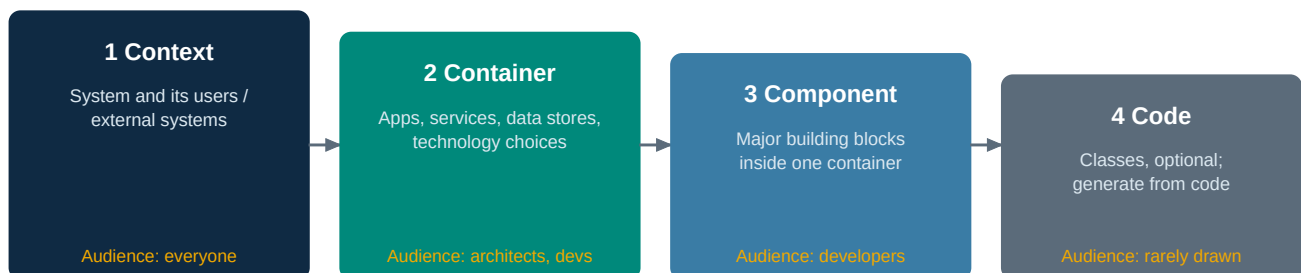
Figure C2. Activity diagram: order fulfilment with decisions and a rejection path.

Writing diagrams as code keeps them versioned and reviewable: Mermaid (renders in GitHub/Azure DevOps wikis), PlantUML, or Structurizr DSL (C4).

```

sequenceDiagram
    Client->>Gateway: POST /orders
    Gateway->>Orders: CreateOrder
    Orders->>Payments: Charge (timeout 2s)
    alt payment ok
        Payments-->>Orders: 200
        Orders->>Bus: OrderPlaced
    else payment failed
        Payments-->>Orders: 402
        Orders-->>Gateway: 422 problem+json
    end
  
```

C5. The C4 model



Zoom in one level at a time. Each diagram answers one question.

Figure C3. The four C4 zoom levels (Simon Brown).

C4 Level 1: System Context

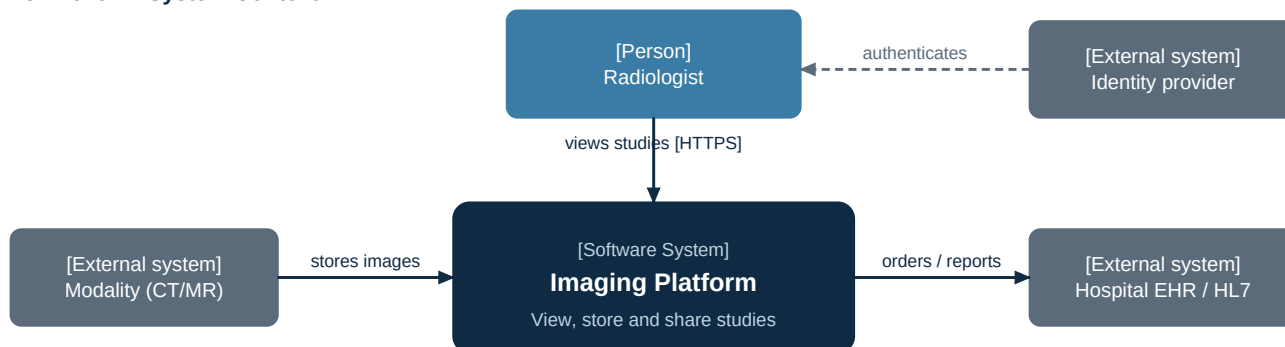


Figure C4. A C4 Level 1 System Context diagram for a medical imaging platform.

Level	Shows	Notation rules	Common mistake
1 Context	The system as one box, users and external systems	Title, type tags, short description per element, labelled relationships with technology	Putting internals on it
2 Container	Separately runnable/deployable units: web app, API, DB, queue	Technology on each box; arrows show protocol	"Container" confused with Docker container (it is any deployable unit)
3 Component	Groupings of code inside one container (controllers, handlers, repositories)	Only for important containers	Drawing for every service
4 Code	Classes/UML	Generate from code when needed	Hand-maintained and stale

Supplementary C4 views: **System landscape**, **dynamic** (numbered interactions, like a sequence), and **deployment** (containers mapped to infrastructure nodes). Every diagram needs a **key** and unambiguous labels; no unlabelled arrows. Tools: Structurizr, draw.io, PlantUML-C4, Mermaid C4.

C6. Capacity planning and performance modeling

Back-of-envelope estimation

Work from users to requests per second, then to storage, bandwidth and instances. Always state assumptions and add headroom.

Step	Worked example: imaging study list API
Users	20,000 clinicians, 25% concurrently active at peak = 5,000
Request rate	Each active user makes ~1 request per 20 s: $5,000 / 20 = 250 \text{ req/s average}$; peak factor 3 gives ~750 req/s
Latency budget	Target p95 = 300 ms; mean service time $S = 60 \text{ ms}$
Concurrency (Little's Law)	$L = \lambda \times W = 750 \times 0.06 = 45 \text{ requests in flight}$ (on average)
Instances	If one pod comfortably handles 8 concurrent requests at 60 ms (about 130 req/s), need $750/130 = 5.8$, round to 6; add N+1 for zones and 30% headroom: 9 pods
Storage	1.2M new studies/year x 150 MB avg = 180 TB/year; tiering: hot 90 days, cool, then archive
Bandwidth	Viewer streams 40 MB/s peak to 50 concurrent readers = 2 GB/s: CDN / regional caches, compression (JPEG-LS / HTJ2K) matter more than API CPU

Step	Worked example: imaging study list API
Database	750 req/s x 4 queries = 3,000 qps; check connection pool (e.g. 9 pods x 50 = 450 connections), indices, and read replicas

Key idea

Little's Law $L = \lambda \times W$ (average items in system = arrival rate x average time in system) holds for any stable system regardless of distribution; it links throughput, latency and concurrency and sizes thread pools, connection pools and instance counts.

Why latency explodes near saturation

M/M/1 queue: $R = S / (1 - \rho)$. Latency explodes near saturation.

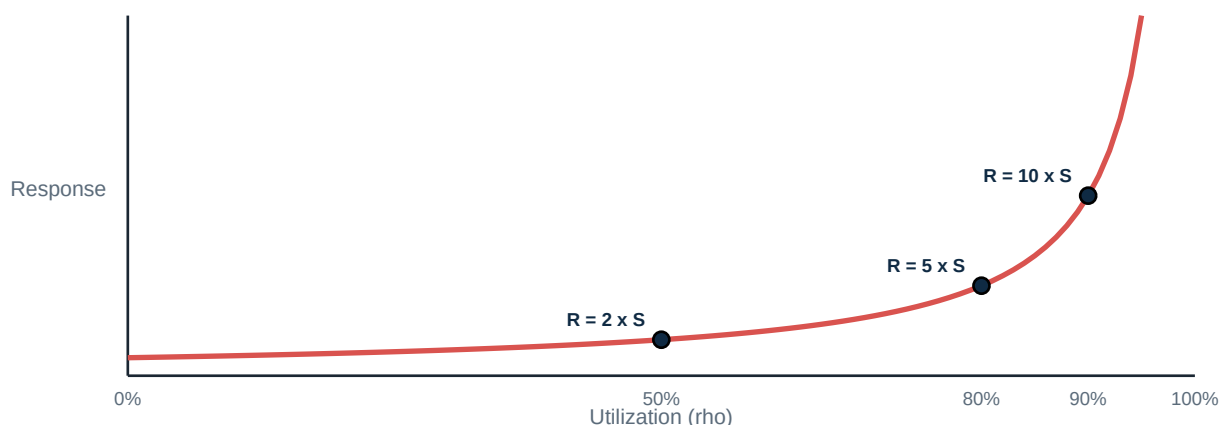


Figure C5. Response time versus utilisation for a single queue. Plan to run steady-state at 50-70% utilisation, never near 100%.

Universal Scalability Law (USL): throughput $X(N) = \lambda N / (1 + \sigma(N-1) + \kappa N(N-1))$, where σ is contention (serialisation) and κ is coherency (crosstalk) cost. Adding nodes yields diminishing returns, then **negative returns** if κ is not zero. **Amdahl's Law:** speed-up is bounded by the serial fraction: if 10% is serial, max speed-up is 10x.

Load-testing methodology

Test type	Goal	Shape
Smoke	Does the script and system work?	1-5 users, short
Load	Behaviour at expected peak	Ramp to target, hold 30-60 min
Stress	Find breaking point and failure mode	Increase until errors/latency exceed SLO
Spike	Sudden surge (marketing event, shift change)	Instant jump, watch autoscale lag
Soak / endurance	Leaks, fragmentation, connection exhaustion	Moderate load for 8-24 h
Capacity / scalability	Throughput versus resources	Stepwise increases with fixed instance counts

Tool	Model	Strengths	Notes
JMeter	GUI + XML test plans, Java threads	Protocol breadth (HTTP, JDBC, JMS), plugins, reporting	Heavier per virtual user; run non-GUI in CI; distributed mode
Locust	Python code, gevent users	Scenarios as code, easy custom logic, web UI, distributed	Python GIL per worker; scale with more workers
k6	JavaScript scripts, Go engine	Efficient, thresholds in CI, Grafana integration	No full browser unless k6 browser

Tool	Model	Strengths	Notes
NBomber / Crank / BenchmarkDotNet	.NET native	Same language as the app; micro-benchmarks	BenchmarkDotNet for code-level, not system-level
Azure Load Testing	Managed JMeter / Locust	Scale-out, Azure Monitor correlation	Cost per VUH

```
# locustfile.py: realistic mix with think time, auth and a failure threshold
from locust import HttpUser, task, between, events
import random

class Clinician(HttpUser):
    wait_time = between(5, 15)          # think time: models real users, not a stress loop
    def on_start(self):
        self.client.headers["Authorization"] = "Bearer " + get_token(self.environment.host)

    @task(8)
    def search_studies(self):
        self.client.get(f"/v1/studies?patientId=P{random.randint(1, 50000)}&limit=25", name="/v1/studies")
    @task(2)
    def open_study(self):
        self.client.get(f"/v1/studies/{random.choice(STUDY_IDS)}/metadata", name="/v1/studies/{id}/metadata")

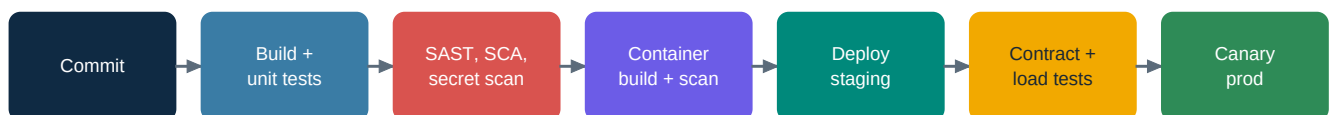
@events.quitting.add_listener
def check(environment, **kw):          # make CI fail when SLO breached
    if environment.stats.total.get_response_time_percentile(0.95) > 300: environment.process_exit_code = 1

# JMeter in CI (non-GUI) then HTML report
jmeter -n -t plan.jmx -l results.jtl -e -o report/ -Jthreads=200 -Jrampup=300 -Jduration=1800
```

Watch out

Load-test pitfalls: testing from one laptop (client becomes the bottleneck); no think time (unrealistic concurrency); unrealistic data (cache hits 100%); ignoring **coordinated omission** (a stalled generator hides tail latency; use open-model arrival rates, e.g. Locust constant throughput or k6 arrival-rate executors); testing a tiny environment and extrapolating linearly; only averages (always report p50/p95/p99 and error rate).

C7. Containers and orchestration for operational efficiency



Shift-left security and quality gates in the delivery pipeline (DevSecOps)

Figure C6. A DevSecOps delivery pipeline with quality and security gates.

Delivery practice	What it gives
Infrastructure as Code (Bicep, Terraform, Pulumi)	Repeatable environments, reviewable changes, drift detection
GitOps (Flux / Argo CD)	Git is the source of truth; pull-based, auditable, easy rollback
Progressive delivery: blue/green, canary, feature flags	Limit blast radius; decouple deploy from release
Ephemeral preview environments	Test on production-like stacks per pull request
Database migrations (expand/contract)	Zero-downtime schema changes: add nullable column -> dual write -> backfill -> switch reads -> drop later
Platform engineering / paved road	Templates (service skeleton, pipelines, dashboards) that give each team security and observability by default

Watch out

Zero-downtime deployment with a shared database: a rolling update means old and new versions run simultaneously, so every schema and message change must be **backward compatible for one version**. This is the reason for the expand/contract pattern and tolerant readers.

C8. Observability and operability design

Signal	Question	Tools	Design rule
Logs	What happened in this request?	Serilog / ILogger -> Log Analytics, Seq, ELK	Structured, include trace id, no PHI/PII, sampled in noisy paths
Metrics	Is it healthy right now and trending?	OpenTelemetry metrics, Prometheus, Azure Monitor	RED (rate, errors, duration) per service; USE (utilisation, saturation, errors) per resource
Traces	Where did the time go across services?	OpenTelemetry, Jaeger, App Insights	Propagate traceparent over HTTP and message headers
Profiles / dumps	Why is this process slow or leaking?	dotnet-trace, dotnet-counters, dotnet-dump, Profiler	Capture on demand in production safely

Define **SLIs** (measurements, e.g. proportion of requests under 300 ms), **SLOs** (targets, 99.5% over 30 days), and an **error budget** (0.5%). When the budget is spent, reliability work outranks feature work. Alert on **symptoms** that burn the budget (multi-window burn-rate alerts), not on every cause (CPU 80%).

Part D. Non-Functional Requirements as Architecture

This part turns NFRs into concrete mechanisms. For each area: the principle, the mechanism, the .NET/Azure implementation, and the trade-off. Examples lean towards regulated, data-sensitive domains such as healthcare imaging, where confidentiality, integrity and auditability are first-class drivers.

D1. Security architecture

Principles

Principle	In practice
Defence in depth	Several independent controls: WAF, gateway, service-level authZ, network policy, encryption, monitoring
Least privilege	Minimal roles and scopes; managed identities scoped per resource; just-in-time admin access (PIM)
Zero trust	Never trust the network: authenticate and authorise every call, mTLS between services, continuous verification
Secure by default / fail closed	Deny unless allowed; <code>FallbackPolicy = RequireAuthenticatedUser</code>
Minimise attack surface	Expose only the gateway; private endpoints for data stores; remove unused features
Privacy by design	Collect least data, classify, encrypt, pseudonymise, retention limits, audit access

Authentication and authorisation

Flow	Use	Notes
Authorization Code + PKCE	Browser SPAs, mobile apps, web apps (users)	Current best practice; implicit flow is deprecated
Client Credentials	Service-to-service, daemons	Prefer managed identity / workload identity to avoid secrets
On-Behalf-Of	API calls downstream preserving user identity	Token exchange; keep scopes narrow
Device Code	Input-constrained devices, CLIs	Phishing risk; restrict
mTLS / certificate-bound tokens	High-assurance service calls	Operational cost of cert lifecycle (use mesh)

OIDC adds identity (ID token) on top of **OAuth 2.0** (delegated authorisation, access token). Validate on every API: signature, `iss`, `aud`, `exp`, `nbf`, scopes/roles; keep access tokens short-lived (5-15 min) and use refresh token rotation. Never accept tokens in query strings; never treat the gateway as the only validator.

Role-based access control (RBAC) and beyond

RBAC assigns permissions to roles and roles to users. It is simple and auditable, but suffers **role explosion** and cannot express contextual rules ("a radiologist may view studies only for patients at their own site"). Layer **ABAC / policy-based** and **resource-based** checks on top (claims, attributes, ownership), and consider relationship-based access (ReBAC, e.g. OpenFGA/Zanzibar style) for sharing-heavy domains.

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o => {
        o.Authority = cfg["Auth:Authority"]; o.Audience = "api://orders";
        o.TokenValidationParameters.ValidateLifetime = true;
```

```

        o.TokenValidationParameters.ClockSkew = TimeSpan.FromMinutes(1); });
builder.Services.AddAuthorizationBuilder()
    .SetFallbackPolicy(new AuthorizationPolicyBuilder().RequireAuthenticatedUser().Build()) // secure by default
    .AddPolicy("orders:write", p => p.RequireClaim("scp", "orders.write"))
    .AddPolicy("SameSite", p => p.AddRequirements(new SiteAccessRequirement()));
builder.Services.AddSingleton<IAuthorizationHandler, SiteAccessHandler>();

app.MapPost("/orders", Create).RequireAuthorization("orders:write");

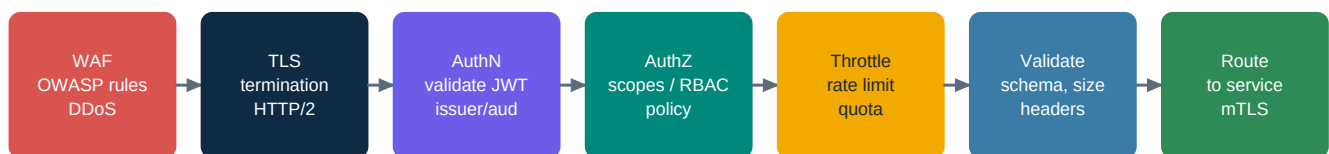
// Resource-based, contextual check inside the use case (never rely on route-level auth alone)
var result = await authz.AuthorizeAsync(user, study, "SameSite");
if (!result.Succeeded) return Results.Forbid();

```

Watch out

Broken access control is the number-one web risk because developers authorise the endpoint but not the object (IDOR/BOLA: `GET /orders/123` returns another tenant's order). Enforce **tenant and ownership filters centrally** (EF Core global query filters, row-level security) and test with negative cases.

Secure API gateway design



Fail early and cheaply: reject at the edge before work reaches a service. Log + trace every decision.

Figure D1. Gateway policy pipeline: cheap rejections first, then identity, authorisation, throttling, validation, routing.

Gateway concern	Design
Edge protection	WAF (OWASP core rule set, bot rules), DDoS protection, geo/IP rules, TLS 1.2+ only (prefer 1.3), HSTS
Identity	Validate JWT (issuer, audience, signature via JWKS cache), optionally exchange for an internal token; mTLS client auth for partners
Authorisation	Coarse scopes at gateway; fine-grained decisions in services (defence in depth)
Traffic management	Rate limits and quotas per client/key (token bucket / sliding window), request size limits, timeouts, concurrency limits, response caching
Validation	Schema validation against OpenAPI, content-type enforcement, header sanitation, reject unknown fields where appropriate
Observability	Correlation id injection, structured access logs without sensitive payloads, metrics per route
Resilience	Retries only for idempotent calls, circuit breakers per backend, multi-instance + zone redundancy, no single instance
Keep thin	No business logic; avoid becoming a new ESB. Separate public, partner and internal gateways when trust levels differ

```

// YARP with built-in rate limiting and a safe default
builder.Services.AddReverseProxy().LoadFromConfig(cfg.GetSection("ReverseProxy"));
builder.Services.AddRateLimiter(o => {
    o.RejectionStatusCode = 429;
    o.AddPolicy("per-client", ctx => RateLimitPartition.GetTokenBucketLimiter(
        ctx.User.FindFirst("azp")?.Value ?? ctx.Connection.RemoteIpAddress?.ToString() ?? "anon",
        _ => new TokenBucketRateLimiterOptions { TokenLimit = 100, TokensPerPeriod = 50,
            ReplenishmentPeriod = TimeSpan.FromSeconds(1), QueueLimit = 0 }));
});
app.UseRateLimiter(); app.UseAuthentication(); app.UseAuthorization();
app.MapReverseProxy().RequireAuthorization().RequireRateLimiting("per-client");

```

Encryption: in transit, at rest, in use

Layer	Mechanism	Detail
In transit (north-south)	TLS 1.2/1.3 at the edge	Strong cipher suites, HSTS, certificate automation (Key Vault, cert-manager, Let's Encrypt)
In transit (east-west)	TLS or mTLS between services	Service mesh (Istio, Linkerd, Open Service Mesh successors) or app-level Kestrel client certificates
At rest: storage and DB	AES-256, platform-managed or customer-managed keys (CMK)	Azure Storage SSE, SQL TDE, Cosmos encryption; CMK in Key Vault / Managed HSM for compliance
At rest: column level	Always Encrypted (client-side keys), Data Protection API, application-level envelope encryption	Protects from DBAs and backups; limits server-side query ability
In use	Confidential computing (enclaves)	Protects data while processed; niche but growing
Key management	Key Vault / HSM, rotation, separation of duties, envelope encryption (data key wrapped by key-encryption key)	Never store keys with data; audit key use
Secrets	Managed identity, Key Vault references, workload identity federation	No secrets in repos, images or env files; rotate automatically

```
// Envelope encryption: encrypt data with a random DEK, then wrap the DEK with the KEK in Key Vault.
var dek = RandomNumberGenerator.GetBytes(32);
using var aes = new AesGcm(dek, 16);
var nonce = RandomNumberGenerator.GetBytes(12); var cipher = new byte[plain.Length]; var tag = new byte[16];
aes.Encrypt(nonce, plain, cipher, tag, associatedData: ownerIdBytes); // AEAD: confidentiality + integrity
var wrapped = (await cryptoClient.WrapKeyAsync(KeyWrapAlgorithm.RsaOaep256, dek)).EncryptedKey;
// Store: wrapped DEK + nonce + tag + cipher.
// Decrypt = unwrap DEK (needs Key Vault permission), then AES-GCM decrypt.
```

.NET note

`DefaultAzureCredential` chain: managed identity in Azure, developer credentials locally; one code path, zero secrets. Prefer **workload identity** on AKS. Use ASP.NET Core **Data Protection** with keys persisted to Blob/Key Vault when running several instances, otherwise cookies and anti-forgery tokens break across pods.

Threat modeling

Threat modeling asks four questions (Shostack): **What are we building? What can go wrong? What are we going to do about it? Did we do a good job?** Do it at design time on the data-flow diagram, and repeat when architecture changes.

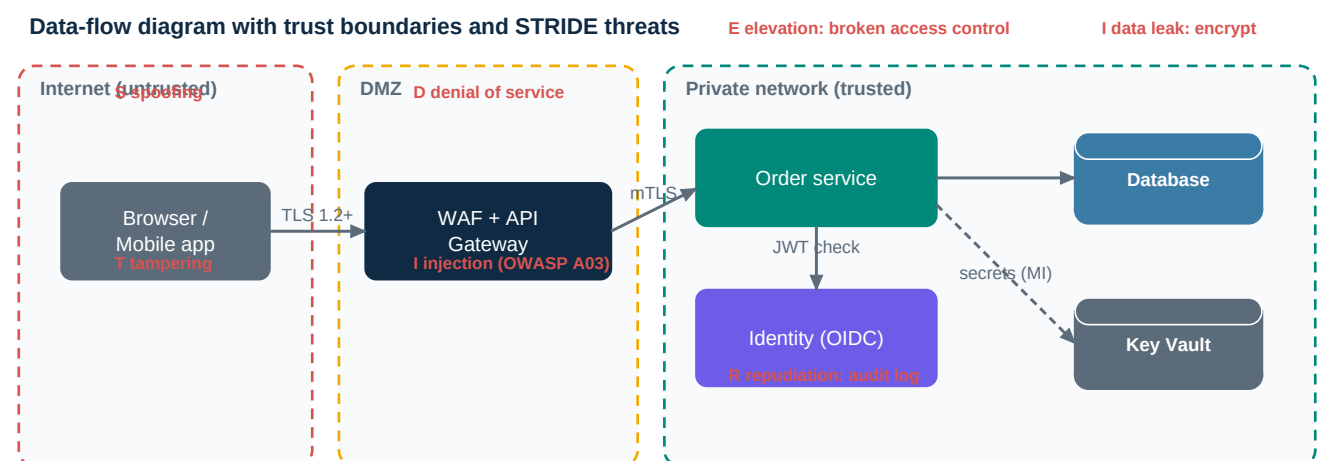


Figure D2. Data-flow diagram with trust boundaries; each crossing is a place to ask STRIDE questions.

STRIDE	Threat	Typical mitigation
Spoofing	Pretending to be someone/something	Strong authN, MFA, mTLS, signed tokens, managed identity
Tampering	Altering data in flight or at rest	TLS, integrity checks (HMAC, signatures), immutable logs, DB permissions
Repudiation	Denying an action	Audit trail with user, time, action; tamper-evident storage
Information disclosure	Leaking data	Encryption, authZ, data minimisation, error hygiene, masking in logs
Denial of service	Exhausting resources	Rate limits, autoscale, quotas, timeouts, WAF, backpressure
Elevation of privilege	Gaining rights not granted	Least privilege, input validation, patching, sandboxing, RBAC tests

Prioritise with **DREAD** or CVSS-style scoring or simple likelihood x impact, record decisions (mitigate, accept, transfer, avoid), and convert mitigations into backlog items and tests. Other methods: PASTA (risk-centric), LINDDUN (privacy), attack trees. Tool: Microsoft Threat Modeling Tool, OWASP Threat Dragon, IriusRisk.

OWASP Top 10 and .NET mitigations

OWASP risk (2021 list)	.NET mitigation
A01 Broken Access Control	Fallback authorisation policy, resource-based checks, global query filters, deny by default, negative tests
A02 Cryptographic Failures	TLS everywhere, AES-GCM, Rfc2898 /Argon2 for passwords (or delegate to IdP), no custom crypto, Key Vault
A03 Injection (SQL, command, LDAP, XSS)	Parameterised queries/EF Core, never concatenate SQL, FromSql interpolated safely, Razor/Blazor encoding, CSP header
A04 Insecure Design	Threat modeling, abuse cases, secure patterns, rate limits on business flows
A05 Security Misconfiguration	Hardened images, no dev pages in prod, disable verbose errors, security headers, policy as code
A06 Vulnerable and Outdated Components	SCA (Dependabot, dotnet list package --vulnerable), SBOM, pinned versions, NuGetAudit
A07 Identification and Authentication Failures	Delegate to IdP, MFA, logout, secure session, refresh token rotation
A08 Software and Data Integrity Failures	Signed artefacts, protected pipelines, safe deserialisation (never BinaryFormatter), verify webhooks
A09 Security Logging and Monitoring Failures	Audit events, SIEM (Sentinel), alerting, tamper-resistant retention
A10 Server-Side Request Forgery	Allow-list outbound hosts, block metadata endpoints, validate URLs, private egress rules

Interviewer lens

The OWASP Top 10 is revised periodically and a newer edition has been published after the 2021 list used above (with supply-chain concerns more prominent). In an evaluation, say you map to the **current** edition and be ready to discuss the shift; verify the latest list on owasp.org. Also know **OWASP API Security Top 10** (BOLA, broken authentication, excessive data exposure, unrestricted resource consumption, BFLA, SSRF, mass assignment).

Vulnerability management and DevSecOps

Practice	Tool examples	Stage
SAST (code analysis)	CodeQL, SonarQube, Roslyn security analyzers, Semgrep	PR / CI
SCA / dependency scan, SBOM	Dependabot, OWASP Dependency-Check, Snyk, Trivy, CycloneDX	CI + scheduled
Secret scanning	GitHub secret scanning, gitleaks	Pre-commit + CI
Container and IaC scan	Trivy, Defender for Containers, Checkov, tfsec	Build / registry
DAST	OWASP ZAP, Burp, Nuclei	Staging
Pen test / red team	Third-party, annual and after major change	Pre-release
Runtime protection	Defender for Cloud, WAF, Falco	Production
Patch SLAs	e.g. critical 7 days, high 30 days; measure time to remediate	Process

D2. Reliability and availability

Nines and composite availability

Availability	Downtime / year	Downtime / month	Typical meaning
99%	3.65 days	7.3 h	Internal tools, batch
99.9%	8.76 h	43.8 min	Standard business apps
99.95%	4.38 h	21.9 min	Important services; zone-redundant PaaS
99.99%	52.6 min	4.38 min	Critical; multi-region, automated failover
99.999%	5.26 min	26 s	Extreme; very costly, usually not worth it

Serial dependencies multiply: a service at 99.9% calling five dependencies at 99.9% each has a ceiling of about $0.999^5 = 99.4\%$. **Parallel redundancy** improves: two independent 99.9% instances give $1 - 0.001^2 = 99.9999\%$ (only if failures are truly independent and failover works). Hence: fewer synchronous hard dependencies, redundancy, and graceful degradation. Your SLO can never exceed the weakest critical dependency unless you decouple it.

Resilience patterns

Pattern	Idea	Caveat
Timeout	Bound waiting; every remote call needs one	Set from the SLO budget; propagate remaining deadline downstream
Retry with exponential backoff + jitter	Handle transient faults	Only idempotent ops; cap attempts; retry budget; avoid retry at multiple layers (multiplicative)
Circuit breaker	Stop calling a failing dependency, probe later	Tune thresholds; surface state in metrics
Bulkhead	Isolate resources per dependency (pools, queues, pods)	Slightly lower utilisation
Fallback / graceful degradation	Serve cached or partial result	Make stale data visible; do not hide failures
Load shedding & rate limiting	Reject early to protect core	Prioritise critical traffic
Queue-based load leveling	Buffer spikes with a queue	Monitor queue age; need DLQ policy

Pattern	Idea	Caveat
Health endpoint monitoring	Liveness (restart me) vs readiness (send traffic) vs startup	Do not check dependencies in liveness
Leader election / competing consumers	Coordinate singletons, scale workers	Use leases; fence stale leaders
Idempotent receiver	Safe duplicates	Needs stable ids

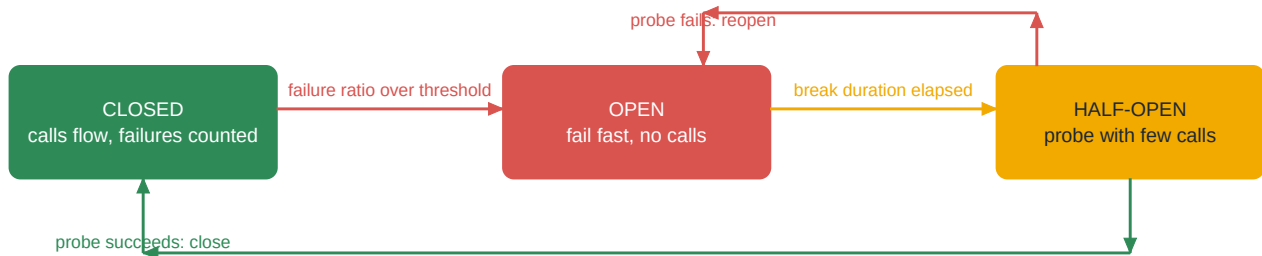


Figure D3. Circuit breaker state machine.

```

builder.Services.AddHttpClient<IPaymentsClient, PaymentsClient>(c => c.BaseAddress = new("https://payments"))
    .AddResilienceHandler("payments", pipeline =>
    {
        pipeline.AddTimeout(TimeSpan.FromSeconds(5)); // outer: total budget
        pipeline.AddRetry(new HttpRetryStrategyOptions {
            MaxRetryAttempts = 3, BackoffType = DelayBackoffType.Exponential, UseJitter = true,
            ShouldHandle = args => ValueTask.FromResult(
                args.Outcome.Result?.StatusCode is >= HttpStatusCode.InternalServerError
                || args.Outcome.Exception is HttpRequestException));
        pipeline.AddCircuitBreaker(new HttpCircuitBreakerStrategyOptions {
            FailureRatio = 0.5, MinimumThroughput = 20, SamplingDuration = TimeSpan.FromSeconds(30),
            BreakDuration = TimeSpan.FromSeconds(15));
        pipeline.AddTimeout(TimeSpan.FromSeconds(1.5)); // inner: per attempt
    });
  
```

High availability and disaster recovery

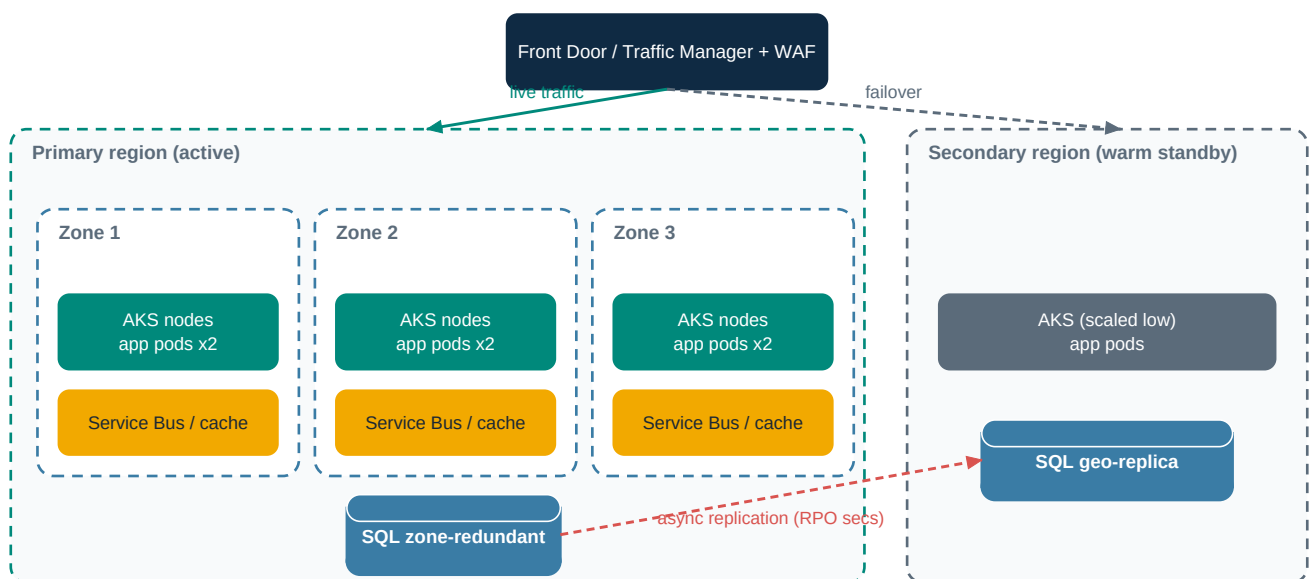


Figure D4. Zone-redundant primary region with a warm-standby secondary region and front-door traffic steering.

DR strategy	RTO	RPO	Cost	How
Backup and restore	Hours-days	Hours	\$	Backups copied to another region; rebuild IaC on demand
Pilot light	Tens of minutes-hours	Minutes	\$\$	Data replicated; minimal compute running; scale up on disaster
Warm standby	Minutes	Seconds-minutes	\$\$\$	Scaled-down full stack in second region
Active-active (multi-region)	Near zero	Near zero	\$\$\$\$	Both regions serve; conflict handling, global data tier

RTO (recovery time objective) is how long you can be down; **RPO** (recovery point objective) is how much data you can lose. Derive them from business impact analysis, then pick the cheapest strategy that meets both. HA (surviving component or zone failure automatically) is not DR (surviving loss of a region or data corruption). **Backups defend against corruption and ransomware; replication does not** (it replicates the damage). Use immutable, versioned, tested backups; practise restores and **game days**.

Interviewer lens

Evaluators often ask: "Your DR plan says RTO 30 min. When did you last prove it?" An untested DR plan is a hope. Name your drill cadence, the evidence, and what the last drill taught you.

Fault tolerance and chaos engineering

Design **failure domains**: process, node, zone, region, provider, and human (bad deployment). Use **blast-radius containment** (cell-based architecture, per-tenant shards, canary releases, feature flags with kill switches). **Chaos engineering** injects faults (kill pods, add latency, drop a dependency) in a controlled way to verify assumptions; tools: Azure Chaos Studio, Chaos Mesh, Litmus, Polly chaos strategies. Start with hypotheses and steady-state metrics.

D3. Performance engineering

Area	Techniques	.NET specifics
Network and API	Fewer round trips, batching, compression, HTTP/2/3, pagination, CDN, conditional GET	<code>MapGroup</code> + response compression; <code>HttpClientFactory</code> (never <code>new HttpClient()</code> per call); gRPC
Async and concurrency	Non-blocking I/O, avoid thread-pool starvation, bounded parallelism, backpressure	<code>async/await</code> end to end (no <code>.Result.Wait()</code>), <code>Channel<T></code> , <code>SemaphoreSlim</code> , <code>Parallel.ForEachAsync</code> with <code>MaxDegreeOfParallelism</code>
Data access	Right indexes, projection, avoid N+1, batching, read replicas, connection pooling, keyset pagination	EF Core <code>AsNoTracking</code> , <code>Select</code> projections, <code>AsSplitQuery</code> , compiled queries, bulk ops (<code>ExecuteUpdate/ExecuteDelete</code>), <code>DbContext</code> pooling
Caching	Cache-aside, write-through, TTL + invalidation by event, stampede protection	<code>HybridCache</code> (L1 + L2 with stampede protection, .NET 9+), <code>IMemoryCache</code> , Redis, output caching
Memory and GC	Reduce allocations, pooling, avoid large object heap churn	<code>Span<T></code> , <code>ArrayPool<T></code> , <code>ValueTask</code> , <code>System.Text.Json</code> source generators, Server GC vs Workstation, container memory limits
Startup and footprint	<code>ReadyToRun</code> , trimming, Native AOT, tiered PGO	Faster cold start for serverless, smaller images
Measure first	Profile before tuning; set budgets	<code>BenchmarkDotNet</code> , <code>dotnet-counters/trace</code> , PerfView, Application Insights profiler

Watch out

Cache invalidation and stampede: a popular key expiring causes many concurrent recomputes. Use stale-while-revalidate, jittered TTLs, single-flight (**HybridCache**, **SemaphoreSlim** per key), and invalidate by event. Never cache user-specific data under shared keys; include tenant and permission context in the key.

D4. Cost optimisation and resource efficiency (FinOps)

Lever	Action	Typical effect / trap
Right-size	Measure actual CPU/memory; set requests close to usage; reduce over-provisioned SKUs	20-40% saving; do not cut headroom for p99 and failover
Autoscale	HPA/KEDA, scale-to-zero for non-prod and bursty workloads, schedule dev environments off	Cold-start and scaling lag; set min replicas for critical paths
Commitments	Reserved instances, savings plans, Azure Hybrid Benefit for existing licences	30-60% off steady baseline; commit only to the stable floor
Spot / low-priority nodes	Batch, stateless workers, CI agents	Eviction handling required
Storage tiering	Hot -> cool -> archive lifecycle policies; compress; delete derived data	Retrieval latency and fees; compliance retention minimums
Data transfer	Keep chatty services in the same zone/region; CDN; avoid cross-region egress	Egress is a hidden major cost
Observability spend	Sample traces, drop noisy logs, tier retention, metrics over logs	Log ingestion often rivals compute cost
Architecture	Do not use microservices where a modular monolith suffices; managed PaaS where ops cost dominates	People cost often exceeds cloud bill
Governance	Tags, budgets, anomaly alerts, showback/chargeback, unit-cost metrics (cost per study / per 1,000 requests)	What gets measured gets managed

Key idea

Track **unit economics** (cost per transaction, per active user, per study stored). A bill that grows slower than usage is healthy; one that grows faster signals an architectural problem. Include **cost as an NFR** with a number in each ADR.

D5. Compliance-aware design (regulated data)

Concern	Design response
Audit and traceability	Append-only audit log of who accessed which record and why ("break-the-glass" flows); time-synchronised; retained per policy
Data residency	Region-pinned storage and processing; avoid replicating PHI to disallowed regions; data classification tags
Minimum necessary / least access	Attribute-based access by role, site and care relationship; masked views
Pseudonymisation and de-identification	Tokenise identifiers for analytics/AI; keep re-identification keys separate; handle burned-in pixel annotations in imaging
Retention and deletion	Policy-driven lifecycle; crypto-shredding for erasure; legal hold support
Third-party risk	Business associate / DPA agreements, vendor security reviews, private endpoints, customer-managed keys
Validated change control	Traceable pipeline (requirements -> tests -> release), signed artefacts, rollback evidence

Part E. The Question Bank

Questions are grouped by theme and tagged **Foundation / Senior / Tough / Architect-level**. Answers are model answers: concise, with the reasoning that scores. Practise by covering the answer, speaking yours aloud for 60-90 seconds, then comparing. For Tough and Architect-level items, rehearse the trade-off sentence: "I would choose X because of driver Y; I accept cost Z; I would revisit if W changes."

E1. Principles and quality attributes

Q1. What is the difference between functional requirements and quality attributes, and which drives architecture more? [Foundation]

Functional requirements describe behaviour; quality attributes (performance, availability, security, modifiability) describe how well. Architecture is shaped mostly by the **architecturally significant requirements**, the few quality attributes with hard targets, because they constrain structure (e.g. 99.99% availability forces redundancy; strict latency forces caching and locality). Any component structure can deliver functions; only some deliver the qualities.

Q2. How do you turn "the system must be scalable" into something actionable? [Senior]

Write a **quality attribute scenario**: source, stimulus, environment, artefact, response, response measure. Example: "When daily active users grow 10x over 6 months (stimulus), the system adds capacity without redesign (response), keeping p95 under 300 ms and cost per request within 20% (measure)." Then derive tactics (stateless services, partitioning, autoscaling), a load test that proves it, and an SLO that monitors it.

Q3. Name three quality attributes that conflict and show how you resolve one conflict. [Senior]

Security vs performance (encryption, authZ checks add latency), availability vs consistency (CAP), modifiability vs performance (indirection layers). Resolution example: for security vs performance, terminate TLS at the edge with hardware offload, cache JWKS and token validation results for the token lifetime, keep authZ in-process, and measure the added cost against the latency budget. The point is **explicit budgeting**, not eliminating the conflict.

Q4. What is a sensitivity point versus a trade-off point in ATAM? [Tough]

A **sensitivity point** is a design decision that strongly affects one quality attribute (e.g. thread-pool size affecting throughput). A **trade-off point** affects more than one attribute in opposing directions (e.g. replication factor improves availability but hurts write latency and cost). Trade-off points are where architectural debate should concentrate.

Q5. What is an architectural fitness function? Give a .NET example. [Senior]

An automated, objective check that an architectural characteristic is preserved as the system evolves. Examples: NetArchTest rule that Domain does not reference Infrastructure; CI gate failing when p95 exceeds budget in a load test; test that no public API change breaks the OpenAPI contract; a build that fails on a critical CVE in a container image.

Q6. What does an ADR contain and why keep it in the repository? [Foundation]

Title, status, context, options considered, decision, consequences. In the repo it is versioned with the code, reviewed in pull requests, discoverable for new joiners, and prevents re-litigating settled decisions. Supersede rather than edit: the history of reasoning is valuable.

Q7. Explain "last responsible moment" and when it is dangerous. [Senior]

Delay irreversible decisions until you have the most information without blocking progress. It is dangerous when the decision affects long-lead items (data residency, identity provider, cloud region, core data model) or when delay causes teams to build on divergent assumptions. Prefer making decisions **reversible** (abstraction at the right seam, feature flags), and decide early on one-way doors.

Q8. State SOLID and give one violation seen in microservice code. [Foundation]

Single responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion. Violation example: a **OrderService** that validates, prices, persists, sends email and publishes events (SRP); a repository interface with 40 methods forcing mocks of everything (ISP); controllers that **new** their **SqlConnection** (DIP).

Q9. Is DRY always right in microservices? [Tough]

No. DRY applies to **knowledge**, not text. Sharing code across services via a common library couples release cycles and forces coordinated upgrades. Duplicating a small mapping or DTO is usually cheaper than coupling. Share stable, versioned contracts (Protobuf, OpenAPI) and infrastructure libraries (logging, auth), not business logic.

Q10. How would you reduce coupling between two modules in a modular monolith? [Senior]

Define a **public contract assembly** per module (interfaces, DTOs, integration events); keep implementation internal; forbid direct references across module internals via fitness tests; communicate through in-process events or the contract; give each module its own schema (or at least own tables) and forbid cross-module joins; expose only what is needed. This also pre-pays the cost of later extraction.

Q11. You are asked to review an architecture with no NFR numbers. What do you do first? [Architect-level]

Stop the design review and elicit **measurable** NFRs: availability, latency, throughput, data volume, RPO/RTO, security classification, compliance, cost envelope, growth. Run a short quality-attribute workshop (utility tree) to rank them. Without numbers you cannot judge any option, and every pattern looks justifiable. Document assumptions explicitly and mark the riskiest as items to validate by spike or load test.

Q12. How do you handle technical debt at the architecture level? [Tough]

Make it visible (debt register with impact and interest: what it slows or risks), tie each item to a quality attribute, allocate a steady capacity (e.g. 15-20%) rather than "later", prefer incremental strangler refactors with fitness functions to prevent regression, and retire debt that sits on the critical path of upcoming features first. Deliberate, prudent debt (documented, time-boxed) is acceptable; reckless debt is not.

Q13. What is Conway's Law and how do you use it? [Senior]

Organisations produce designs that mirror their communication structure. Use the **inverse Conway manoeuvre**: arrange teams around desired service boundaries (stream-aligned teams owning a bounded context end to end), minimise cross-team dependencies, and provide a platform team to cut cognitive load. A microservice boundary that crosses team boundaries will leak.

Q14. Why do 12 teams on one account need an architecture governance model, and what does a lightweight one look like? [Tough]

Without it, 12 teams diverge on auth, observability, messaging and data contracts, making integration expensive and security uneven. Lightweight governance: a small architecture guild, published reference architectures and golden-path templates, ADR review for cross-cutting decisions, automated checks (fitness functions, policy as code), and **guardrails not gates**: teams may deviate with a recorded rationale. Governance should reduce decision cost, not centralise it.

E2. Microservices

Q15. What defines a microservice and what does it not? [Foundation]

A small, independently deployable service owning a single business capability and its data, communicating through well-defined interfaces. It is **not** defined by lines of code, by being behind HTTP, or by running in a container. If it cannot be deployed alone, or shares a database with others, it is not independent.

Q16. How do you decide microservice boundaries? [Senior]

Combine: business capabilities and bounded contexts (DDD), event storming to find aggregates and hot spots, data ownership (what must change transactionally together), team ownership and change cadence, and non-functional differences (one part needs different scaling or security). Validate by checking that common use cases involve one service and that chatty calls between two services indicate they should merge.

Q17. Two services need the same data frequently. What are your options? [Tough]

(1) Merge them if they change together. (2) Make one the owner and let the other call its API (with caching and a timeout). (3) **Replicate** read-only data via events (event-carried state transfer) into a local store, accepting eventual consistency and a data-freshness SLO. (4) Introduce a read model/composition layer. Avoid a shared database. Choose by staleness tolerance and call volume.

Q18. Why is the "database per service" rule important and when can you bend it? [Tough]

It enforces encapsulation and independent deploy/scale/evolve. Bending: a shared **database server** with separate schemas and credentials per service is fine operationally. Sharing tables between services is not. During migration from a monolith, a temporary shared DB with an ACL or views is acceptable if a removal date is recorded.

Q19. Synchronous REST vs asynchronous messaging between services: how do you choose? [Senior]

Use sync when the caller needs an immediate answer to proceed (queries, validation) and the dependency is highly available; use async when work can be deferred, fan-out is needed, or the callee may be down. Sync chains multiply latency and couple availability; async adds eventual consistency and operational surface (DLQ, ordering). A mixed model is normal: commands/queries sync at the edge, state-change propagation async.

Q20. Service A calls B, which calls C, which calls D synchronously. What goes wrong and what do you do? [Tough]

Availability multiplies (0.999^4), latency adds, a slow D exhausts threads up the chain (cascading failure), and retries amplify load. Fixes: reduce depth (merge or aggregate in a BFF), replace with async events or a read model, add timeouts with deadline propagation, circuit breakers and bulkheads, cache results, and make callers degrade gracefully.

Q21. What is the API gateway pattern and its main risk? [Senior]

A single entry point handling routing, authentication, rate limiting, TLS termination and aggregation. Risk: it becomes a bottleneck, a single point of failure, or an ESB-like place for business logic. Keep it thin, run multiple instances across zones, and use BFFs per client type instead of one general gateway holding all composition logic.

Q22. Gateway vs BFF vs service mesh: how do they differ? [Senior]

Gateway: north-south edge concerns for all clients. BFF: client-specific backend that aggregates and shapes data for one experience (web, mobile). Service mesh: east-west infrastructure (mTLS, retries, traffic splitting, telemetry) via sidecars or ambient mode, without code changes. They complement each other.

Q23. How does service discovery work on Kubernetes? [Senior]

Each Service gets a stable ClusterIP and DNS name (`orders.prod.svc.cluster.local`); kube-proxy/eBPF routes to ready pods. No client-side registry needed. For gRPC, use headless services or a mesh for per-request load balancing because a long-lived HTTP/2 connection otherwise sticks to one pod.

Q24. How do you manage distributed configuration and secrets? [Tough]

Environment-specific config via App Configuration/ConfigMaps with feature flags; secrets in Key Vault accessed through managed or workload identity; one immutable image across environments; reload on change where safe. Never commit secrets; rotate automatically; audit access. In .NET use configuration providers and the options pattern ([IOptionsMonitor](#)).

Q25. How do you version APIs between independently deployed services? [Senior]

Make changes backward compatible (additive fields, tolerant readers), run **consumer-driven contract tests** (Pact) in the producer pipeline, version explicitly (URL or header) only for breaking changes, support N-1 for a defined window, and announce deprecation with [Sunset](#) headers and telemetry on remaining consumers. For messages use schema registry compatibility rules.

Q26. What is the "distributed monolith" and how do you detect it? [Tough]

Services that cannot deploy independently: shared DB, lockstep releases, deep synchronous chains, shared domain library. Detect: release-coupling metric (percentage of releases requiring another service), dependency graph from traces, shared schema ownership, incident blast-radius analysis, and the question "can I deploy this alone today?".

Q27. When would you advise AGAINST microservices? [Architect-level]

Small team or domain not understood; need for strong ACID across the model; low operational maturity (no CI/CD, observability); uniform scaling; tight latency with chatty interactions; limited budget. Recommend a **modular monolith** with enforced boundaries and an extraction path. Microservices solve organisational scaling and independent evolution problems; do not adopt them to solve a code-quality problem.

Q28. How do you test microservices without a giant integration environment? [Tough]

Unit tests on domain logic; component tests of each service with real dependencies via Testcontainers and [WebApplicationFactory](#); consumer-driven contract tests for APIs and messages; a thin layer of end-to-end tests on critical journeys; production **synthetic monitoring**, canary releases, and feature flags. Keep E2E tests few and stable.

Q29. How do you implement correlation and tracing across services? [Senior]

Adopt W3C Trace Context ([traceparent](#)); in .NET, [Activity](#) propagates automatically over [HttpClient](#); add OpenTelemetry instrumentation for ASP.NET Core, [HttpClient](#), gRPC, SQL, and the messaging library; propagate context in message headers; export to a backend (Jaeger, Tempo, App Insights); put the trace id in logs and in error responses.

Q30. What is the sidecar pattern and what are its downsides? [Tough]

Co-deploying a helper container with the app to provide cross-cutting capabilities (proxy, logging agent, Dapr). Pros: language-agnostic, independent upgrades. Cons: resource overhead per pod, extra latency, lifecycle coupling, operational complexity. Mesh "ambient" modes and node-level agents reduce overhead.

Q31. How would you handle a long-running operation in a REST API? [Senior]

Return [202 Accepted](#) with a [Location](#) of an operation resource, process asynchronously via a queue, let the client poll (with [Retry-After](#)), or notify through webhook/SignalR. Make the operation idempotent and cancelable, and persist its state to survive restarts (Durable Functions, a saga, or a job table).

Q32. How do you prevent one tenant from affecting others in a multi-tenant microservice platform? [Tough]

Per-tenant rate limits and quotas, bulkheads (separate queues/pools for heavy tenants), partition keys including tenant id, resource governance, noisy-neighbour monitoring, option for a dedicated deployment cell for large tenants, and tenant-scoped authorisation via global query filters or row-level security. Decide the isolation model (shared, schema, database, stamp) from compliance and cost.

Q33. Design the service decomposition for a hospital imaging platform (study ingestion, viewing, reporting, orders, AI). [Architect-level]

Start with capabilities: **Ingestion** (receives DICOM via STOW-RS/DIMSE, validates, stores objects), **Archive/Catalogue** (metadata index, QIDO), **Image delivery** (WADO-RS, rendering, caching), **Worklist/Orders** (HL7/FHIR integration), **Reporting**, **Identity & access**, **Audit**, and **AI orchestration**. Data ownership: Archive owns pixel storage and metadata; Orders owns scheduling. Use events (**StudyReceived**, **ReportSigned**) to trigger AI and notifications; keep image bytes on blob storage with claim-check references. Start as a modular monolith for Orders/Reporting if team size is small; separate Ingestion and Delivery early because their scaling and failure profiles differ (bursty bulk write vs latency-sensitive read).

Q34. What are the typical failure modes in a microservice POC that surface in production? [Tough]

Missing timeouts and retries with no jitter, unbounded queues, no idempotency, schema changes breaking consumers, config drift, N+1 remote calls, connection pool exhaustion, thread-pool starvation from blocking calls, log volume explosion, secrets in env files, health checks that restart healthy pods, no backpressure, and single instances hidden behind "HA" labels.

Q35. What do liveness, readiness and startup probes do? [Senior]

Startup: delays other probes until the app has started (slow init). **Readiness**: removes the pod from load balancing when it cannot serve (dependencies, warm-up). **Liveness**: restarts a stuck process. Do not include remote dependency checks in liveness: a database blip would restart every pod and worsen the outage.

Q36. How do you roll out a breaking change to a service with 8 consumers? [Tough]

Use **expand/contract**: add the new contract alongside the old; migrate consumers one by one with telemetry showing remaining usage; use contract tests; keep both until usage is zero; remove old after the deprecation window. For data, add the new column, dual-write, backfill, switch reads, then drop. Never require lockstep deployment.

Q37. What are the main costs of microservices that teams underestimate? [Senior]

Distributed debugging, testing complexity, versioning, data consistency work, platform engineering, observability spend, cognitive load, duplicated cross-cutting code, and organisational coordination. Budget a platform team and tooling before splitting.

Q38. What is the twelve-factor app and which factors do POCs usually violate? [Foundation]

A methodology for portable cloud services. Commonly violated: config in files/code (III), stateful processes holding session in memory (VI), logs written to files (XI), dev/prod divergence (X), and slow startup/ungraceful shutdown (IX).

E3. Event-driven architecture and messaging

Q39. Difference between a command, an event and a query message? [Foundation]

Command: an instruction to one handler, may be rejected (**ReserveStock**). **Event**: an immutable fact that already happened, published to any number of subscribers (**StockReserved**). **Query**: a request for data. Naming and routing differ: commands to a queue, events to a topic.

Q40. What delivery guarantees do brokers offer and which do you design for? [Senior]

At-most-once, at-least-once, and limited exactly-once within a boundary. Design for **at-least-once** with idempotent consumers; true end-to-end exactly-once across a DB and broker is not achievable without cooperation (outbox + inbox gives effectively-once processing).

Q41. Explain the dual-write problem and the outbox pattern. [Tough]

Updating the DB and publishing a message are two separate operations; if one succeeds and the other fails, state diverges. Outbox: write the event to an outbox table in the **same local transaction** as the business change; a relay (poller or CDC such as Debezium) publishes and marks it sent. Consumers dedupe via an inbox or idempotent handlers. Result: at-least-once publication with no lost events.

Q42. How do you guarantee ordering? [Tough]

Global order is unscalable. Provide ordering **per key**: Kafka partition by aggregate id; Service Bus sessions with session id = aggregate id; single-writer per key. Make handlers tolerate out-of-order across keys; include version numbers so stale events are ignored.

Q43. What is a poison message and how do you handle it? [Senior]

A message that repeatedly fails processing and blocks or loops. Handle: bounded retries with backoff, then move to a **dead-letter queue** with the failure reason; alert on DLQ depth; provide tooling to inspect, fix and replay; separate transient from permanent errors (do not retry validation failures).

Q44. How do you design an idempotent consumer? [Tough]

Use a stable message id; in one transaction record it in an inbox table (unique key) and apply the effect; on duplicate, skip. Or make the operation naturally idempotent (upsert, **SET** not **INCREMENT**), or use version checks. Beware of external side effects (emails, payments): use idempotency keys with the provider.

Q45. Choreography vs orchestration for a business process. [Senior]

Choreography: services react to events; loose coupling but hidden process and cyclic risk. Orchestration: a coordinator drives the steps; explicit, testable, observable, but a central component. Rule of thumb: choreography for simple, few-step flows; orchestration (saga/state machine/workflow) once there are compensations, timeouts, or more than about 3-4 participants.

Q46. How do you evolve event schemas safely? [Tough]

Additive changes only; tolerant readers; schema registry with compatibility checks in CI; never repurpose a field; deprecate then remove across versions; for breaking changes publish a new event type; use upcasters for replay. Consumers must ignore unknown fields. Keep events small and focused on the business fact.

Q47. Event notification vs event-carried state transfer vs event sourcing? [Senior]

Notification: tiny event, consumers call back for details (low payload, higher coupling and chatty). Event-carried state transfer: event includes the data consumers need (autonomy, bigger payloads, replicas). Event sourcing: events are the persistent source of truth for one aggregate. They are independent choices.

Q48. Kafka vs Azure Service Bus: how do you choose? [Tough]

Kafka/Event Hubs: high-throughput log, replay, stream processing, multiple independent readers by offset; ordering per partition; consumer manages position. Service Bus: enterprise messaging, per-message settlement, sessions, DLQ, scheduling, transactions, lower throughput. Choose Kafka for analytics/streaming and event backbone with replay needs; Service Bus for workflow and command-style integration.

Q49. Consumer lag is growing. How do you diagnose and fix? [Tough]

Check processing time per message versus arrival rate (Little's law), partition skew (hot key), failing retries, downstream bottleneck (DB locks, rate limits), GC or thread-pool starvation, and consumer count versus partitions. Fix: scale consumers (up to partition count), batch processing, optimise handler, increase partitions with care (reorders keys), move slow work off the hot path, apply backpressure, or shed non-critical messages.

Q50. What is backpressure and how is it handled in .NET? [Senior]

Signalling producers to slow when consumers cannot keep up. Techniques: bounded queues/`Channel<T>` with `BoundedChannelFullMode.Wait`, prefetch limits on consumers, rate limiting, `await` on bounded `SemaphoreSlim`, and for HTTP, 429/503 with `Retry-After`. Unbounded queues only move the failure to out-of-memory later.

Q51. How do you test event-driven flows? [Tough]

Unit-test handlers; test publishers and consumers against **schema/contract** (Pact messages or schema registry); integration tests with an in-memory or containerised broker; saga tests with a test harness (MassTransit test harness / NServiceBus testing); end-to-end with correlation ids and polling assertions with timeouts; chaos tests dropping and duplicating messages.

Q52. What is the claim-check pattern? [Senior]

Store a large payload (image, document) in blob storage and send only a reference in the message. Keeps messages small (broker size limits, throughput), lets consumers fetch on demand, and supports access control and lifecycle on the blob. Needs secure short-lived URLs and cleanup.

Q53. Your event-driven system has become impossible to reason about. What do you do? [Architect-level]

Introduce visibility and ownership: an event catalogue (AsyncAPI), end-to-end tracing and a "business process" view, naming conventions, an owner per event type and per cross-service process; replace implicit choreography for complex flows with explicit orchestration; reduce event granularity noise; add contract tests; and document sequence diagrams for key flows. EDA needs governance proportional to its coupling-in-the-shadows.

Q54. What is the difference between a queue and a topic with subscriptions? [Senior]

A queue delivers each message to one of several competing consumers (load balancing). A topic delivers a copy to each subscription (pub/sub); each subscription then behaves like a queue for its consumers. Use queues for work distribution and topics for fan-out of facts.

Q55. What is Change Data Capture and where does it fit? [Tough]

CDC reads the database transaction log to emit row changes (Debezium, SQL Server CDC/Change Tracking). Fits outbox relays, cache and search index sync, replication to analytics. Caveats: schema coupling to table shape (so project through an outbox or a translation layer), ordering guarantees, operational complexity, and handling of snapshots.

E4. SOA

Q56. What is SOA and what was the role of the ESB? [Foundation]

Architecture of loosely coupled, coarse-grained, contract-based services (typically SOAP/WSDL), integrated and mediated by an Enterprise Service Bus providing routing, protocol bridging, transformation and orchestration. The ESB centralised integration logic and governance.

Q57. How are microservices different from SOA, honestly? [Senior]

Both are service-oriented; microservices emphasise small autonomous services, decentralised governance and data, lightweight protocols and smart endpoints, and independent deployment, versus SOA's enterprise reuse, shared canonical models, central ESB and heavier standards. Microservices can be seen as SOA done with DevOps and bounded contexts. Many "microservices" fail by recreating an ESB in a gateway or a shared library.

Q58. What are the downsides of an ESB and how do modern systems avoid them? [Senior]

Single point of failure and bottleneck, logic hidden in the bus, vendor lock-in, slow change through central teams. Modern approach: smart endpoints and dumb pipes (brokers with minimal logic), decentralised ownership, consumer-driven contracts, and cloud-native integration (Service Bus, Event Grid, Logic Apps, API Management) used with discipline.

Q59. A bank has 200 WCF services. Migration plan to modern .NET? [Tough]

Inventory and classify by usage (call telemetry), consumers (internal vs external SOAP clients), and bindings. For external SOAP consumers consider CoreWCF behind the same contract; for internal callers move to gRPC or REST with contract-first design; front all with a facade (YARP) and apply the strangler pattern; extract shared data access behind services; retire unused services first (often 30%). Measure traffic shift; keep the old stack until zero traffic.

Q60. What is canonical data model and why is it risky? [Senior]

One enterprise-wide model that all systems translate to and from. Reduces point-to-point mappings but creates a coupling hotspot and a lowest-common-denominator model. Prefer per-context models with explicit translation (ACL) and published language only at real integration boundaries.

Q61. Explain orchestration vs choreography in the SOA context. [Senior]

Orchestration (BPEL, BizTalk, Logic Apps, Durable Functions) has a controller that invokes services in sequence with business-process visibility. Choreography: services react to events without a central controller. SOA historically favoured orchestration; modern microservices choose per workflow complexity.

E5. Domain-Driven Design

Q62. What is a bounded context and how does it relate to a microservice? [Foundation]

A boundary within which a domain model and its language are consistent and unambiguous. A microservice often implements one bounded context (or part of one). A bounded context is a conceptual boundary; a microservice is a deployment boundary. One context may contain several services, but a service should not span contexts.

Q63. Entity vs value object vs aggregate? [Foundation]

Entity: has identity and lifecycle (Order). Value object: immutable, defined by values, no identity (Money, Address). Aggregate: a consistency boundary around entities and value objects with a single root through which all changes pass, enforcing invariants.

Q64. How big should an aggregate be? [Senior]

As small as possible while protecting true invariants within a single transaction. Large aggregates cause contention, lock conflicts and heavy loads. Reference other aggregates by id and use domain events for eventual consistency across aggregates. If you need two aggregates updated atomically, reconsider the boundary or accept a saga.

Q65. How do you handle an invariant that spans two aggregates? [Tough]

Question whether it is a real invariant needing immediate consistency; most are not. Options: merge into one aggregate (if small), enforce with eventual consistency via domain events and compensation, use a domain service that reads both and relies on optimistic concurrency, or a database constraint as a backstop. Document the acceptable inconsistency window.

Q66. What is an anti-corruption layer and when do you add one? [Senior]

A translation boundary that maps a foreign or legacy model into your domain language so it cannot pollute your model. Add when integrating with legacy systems, third-party APIs, or a context with an incompatible model. In .NET: an adapter implementing your domain interface, with mapping and defensive validation.

Q67. Core, supporting and generic subdomains: how does classification drive investment? [Senior]

Core (competitive advantage): invest in rich DDD, best engineers, custom code. Supporting: simpler implementation, maybe CRUD. Generic (auth, billing, email): buy or use SaaS/library. This avoids over-engineering low-value areas and prevents buying what should be differentiating.

Q68. Where do domain events get dispatched and what are the transactional pitfalls? [Tough]

Aggregates record events; after the unit of work commits, handlers run. **Pre-commit** dispatch lets in-process handlers join the same transaction (consistent but couples); **post-commit** dispatch is safer for external effects but can lose events on crash unless using an **outbox** written in the same transaction. In EF Core, collect events in `SaveChanges` override and write to the outbox table in the same save.

Q69. Repository vs EF Core DbContext directly? [Senior]

`DbContext` already implements Unit of Work and repository-like `DbSet`. A repository per aggregate root is still useful to express aggregate-oriented loading (includes), hide persistence from the domain, and ease testing, but generic repositories over EF Core are mostly noise. Use specific repositories for aggregate roots; use direct queries (read models) for reads.

Q70. How do you persist an aggregate with EF Core cleanly? [Tough]

Map the root and owned entities (`OwnsMany`, `OwnsOne` for value objects), use private setters/backing fields, `HasConversion` for strongly typed ids, keep the constructor for EF private, concurrency token (`rowversion`) on the root, and avoid navigation across aggregates (use ids). Load whole aggregates and save once per transaction.

Q71. What are strongly typed ids and why use them? [Senior]

`record struct OrderId(Guid Value)` instead of raw `Guid` prevents mixing ids of different aggregates at compile time and documents intent. Needs converters for EF Core and `System.Text.Json`; minor overhead.

Q72. What is a context map relationship "conformist" and why avoid it? [Tough]

Downstream team adopts the upstream model without translation because it cannot influence it. Cheap short term; long term it couples your model to someone else's changes. Prefer an ACL when the upstream model is poor or volatile; use conformist only when the upstream model is good and stable.

Q73. How would you introduce DDD to a team used to CRUD and a database-first approach? [Architect-level]

Start strategic: event storming workshop to find the ubiquitous language and contexts; choose one **core** subdomain to apply tactical patterns; leave CRUD areas as simple transaction scripts. Coach with code reviews focused on invariants and naming; make business experts reviewers of the model. Avoid a big-bang mandate and avoid ceremonial layers.

Q74. What is "eventual consistency" from the user's perspective and how do you design UX for it? [Tough]

After a command the read side may lag. Design: return the new state in the command response or an optimistic UI update; show "processing" states; use read-your-writes via session consistency or version tokens (client sends the version it expects); poll or push (SignalR) for completion; set and monitor a staleness SLO.

Q75. What are specification and factory patterns used for in DDD? [Senior]

Specification encapsulates a business rule or query criterion as an object (composable and reusable in domain and persistence). Factory encapsulates complex creation to guarantee invariants from the first moment. Both keep rules out of controllers and handlers.

Q76. What does "persistence ignorance" mean and what is its limit? [Tough]

The domain model has no dependencies on persistence technology. It improves testability and focus, but perfect ignorance is unrealistic: EF Core needs private constructors, backing fields and mapping config, and performance may require aggregate shape compromises. Aim for pragmatic ignorance: no persistence attributes or base classes in the domain, mapping in infrastructure.

E6. CQRS, Event Sourcing and Sagas

Q77. What problem does CQRS solve? [Foundation]

Different shapes and loads for reads and writes: write models enforce invariants and are normalised; read models are denormalised for queries and scale independently. It also removes the need for the domain model to serve every UI view.

Q78. Does CQRS require two databases or event sourcing? [Senior]

No. Levels: (1) separate command/query handlers over one DB; (2) separate read views/tables; (3) separate stores with async projections. Event sourcing is an independent choice and almost always paired with CQRS because event streams are poor for queries.

Q79. What are the consistency and operational consequences of CQRS with separate stores? [Tough]

Eventual consistency between write and read, projection lag, rebuild procedures, idempotent projectors with checkpoints, duplicated schema evolution, and monitoring of projection lag. Benefits only pay off where read/write scaling or model differences are real.

Q80. How do you do read-your-writes after a command in a CQRS system? [Senior]

Return the resulting resource/version in the response; have the client pass a minimum version; the query side waits briefly or falls back to the write store when version is behind; or read from the primary for the session window; or push updates via SignalR when projection catches up.

Q81. When is event sourcing a good idea? [Senior]

When audit and history are first-class (finance, healthcare provenance), when temporal queries matter, when many downstream projections are needed, or when the domain is naturally event-based. Poor fit: simple CRUD, heavy ad hoc reporting without projections, teams without experience, or data with strict erasure needs unless you design crypto-shredding.

Q82. How do you handle event schema changes in an event-sourced system? [Tough]

Never mutate stored events. Use upcasters (convert old versions on read), add new event types, keep event contracts minimal and semantic, and version through type names or explicit version fields. Test replay of production-like histories in CI. Consider the weak schema (JSON) with explicit migrations when necessary (copy-transform to a new stream).

Q83. What are snapshots and when do you need them? [Tough]

A stored aggregate state at version N to avoid replaying the whole stream. Optimisation only; needed when streams are long (thousands of events). Design aggregates to have short lifetimes (e.g. one stream per period) before reaching for snapshots.

Q84. How do you rebuild a projection without downtime? [Tough]

Build a new projection version alongside the old by replaying events from the beginning into a new store; run until caught up; switch reads (feature flag) once lag is near zero; keep the old version for rollback; then delete it. Projectors must be idempotent and rate-limited to avoid overloading the event store.

Q85. How does GDPR "right to be forgotten" work with event sourcing? [Tough]

Do not store personal data directly in immutable events if erasure applies. Options: **crypto-shredding** (encrypt PII with a per-subject key stored separately; delete the key), store only references to PII in a mutable store, or use "forgettable payloads". Document legal basis and retention; test that projections purge personal data too.

Q86. What is a saga and how is it different from a distributed transaction? [Senior]

A saga coordinates a business transaction across services as a series of local transactions with compensating actions for failure. Unlike 2PC it does not hold locks across services and does not provide isolation; intermediate states are visible, so design with semantic locks and idempotency.

Q87. What makes a good compensating action? [Tough]

Idempotent, retryable, commutative with other steps where possible, and semantically meaningful (refund, release, cancel) rather than literal rollback. Compensation can fail; design retries, alerts, and manual intervention queues. Order steps so non-compensable actions come last (pivot transaction).

Q88. Orchestrator crashed in the middle of a saga. What happens? [Tough]

The orchestrator persists its state (state machine instance) after each step in a durable store; on restart it resumes from the last recorded state, re-sends pending commands (idempotent receivers handle duplicates). Use timeouts to detect stuck steps and schedule compensation. MassTransit saga state in SQL/Mongo/Redis, Durable Functions event history, or Temporal provide this.

Q89. Design the order-placement saga for an e-commerce platform including failure at payment.

[Architect-level]

Steps: create order (Pending) -> reserve stock -> authorise payment -> confirm order -> ship. If authorisation fails: publish **PaymentFailed**; orchestrator sends **ReleaseStock**, marks order **Cancelled**, notifies the customer. Use idempotency keys for payment, outbox in every service, timeouts per step (e.g. 30 s) with escalation, and a status API so users see Pending/Confirmed/Failed. Capture state in a saga state machine; test each failure path including duplicate replies and late responses after timeout.

Q90. How do you query across services (reporting) without joins? [Senior]

API composition in a BFF for small/real-time needs; **CQRS read models** populated from events for a denormalised cross-service view; a data lake/warehouse for analytical queries (ETL/ELT or streaming); search index for text queries. Never join across service databases.

Q91. Is two-phase commit ever acceptable? [Tough]

Within a single database cluster or a small, tightly controlled set of resource managers (MSDTC, XA) it works, but it reduces availability, couples services, and is unsupported by most cloud messaging. For microservices across teams, prefer sagas/outbox. Legacy brownfield may keep 2PC temporarily with clear boundaries.

Q92. What is the difference between optimistic and pessimistic concurrency, and which do you pick?

[Senior]

Optimistic: no lock; detect conflict at write via version/ETag (**rowversion**, **If-Match**) and retry or reject. Pessimistic: lock rows/aggregates while working. Optimistic suits low-conflict, distributed, long-think-time scenarios (web); pessimistic only for short critical sections with high conflict.

E7. Anti-patterns and system health

Q93. How do you measure the health of an architecture? [Senior]

Leading and lagging indicators: DORA metrics (deployment frequency, lead time, change failure rate, time to restore), dependency graph metrics (cycles, fan-in/out), coupling indicators (co-change analysis from git), build/test time, escaped-defect rate, SLO attainment and error budget, incident blast radius, cost per unit, and onboarding time for new developers.

Q94. How do you find the real dependency structure of a running system? [Tough]

From distributed traces (service graph), mesh telemetry, network flow logs, and message topology (who subscribes to what). Compare with intended C4 diagrams; drift is a finding. Static analysis (project references, NetArchTest) for in-process structure.

Q95. Describe a retry storm and how you prevent it. [Tough]

When a dependency slows, callers retry immediately, multiplying load (e.g. 3 attempts x 3 layers = 27x) and preventing recovery. Prevent: retries at one layer only, exponential backoff with jitter, retry budgets (cap retry ratio), circuit breakers, timeouts shorter than the caller's, load shedding at the dependency, and honouring **Retry-After**.

Q96. What is the "golden hammer" anti-pattern and how do ADRs counter it? [Senior]

Applying a favourite technology to every problem. ADRs require listing alternatives and criteria, which exposes mismatches (e.g. Kafka for 10 messages/minute). Peer review of ADRs and a technology radar help.

Q97. What is a "chatty interface" and how do you detect and fix it? [Tough]

A client makes many fine-grained calls to complete one task, typically N+1 across services. Detect via traces showing fan-out patterns and call-count per request. Fix: coarser-grained endpoints, batch APIs, BFF composition with parallel calls, GraphQL with data loaders, replicated read models, caching.

Q98. What is "shared kernel" and when is a shared library acceptable? [Senior]

A small, jointly owned part of the model shared by two contexts with coordinated changes. Acceptable for stable primitives (Money, standard contracts, infrastructure helpers). Not acceptable for business rules that evolve: it re-couples deployment and forces upgrades across teams.

Q99. A client says microservices gave them slower delivery. How do you investigate? [Architect-level]

Measure lead time and where it accrues (queueing for environments, cross-team coordination, flaky tests, manual release). Examine boundaries (features touching several services), shared DB/library coupling, missing platform automation, and team topology. Remedies may be merging services, introducing a golden path and paved-road CI/CD, contract tests to replace E2E, or moving back to a modular monolith for chatty parts. Be willing to recommend consolidation.

E8. Technology stack selection

Q100. How do you justify a technology choice to a sceptical architecture board? [Foundation]

Tie it to quality attribute scenarios and constraints, present at least two alternatives scored in a weighted matrix, show evidence (spike results, benchmarks, reference customers), state costs (licence, skills, operations, exit), the risks and mitigations, and the reversal plan. Provide an ADR. Avoid appeals to popularity.

Q101. Why choose .NET for new backend services in a Microsoft-centred enterprise? [Senior]

High performance and low overhead on modern runtime, excellent async model, strong typing and tooling, first-class Azure SDKs and managed identity, LTS support, mature ecosystem for enterprise (EF Core, gRPC, SignalR, Identity), good container support, and skill reuse from existing .NET Framework teams. Honest trade-offs: smaller ML ecosystem than Python, occasional commercial licences in third-party libraries, and runtime memory footprint above Go/Rust.

Q102. Compare Minimal APIs and controllers for a microservice. Which would you standardise on? [Tough]

Minimal APIs: lower ceremony and overhead, route groups, endpoint filters, typed results, easy source-generated JSON; fits small focused services. Controllers: conventions, filters, model binding richness, large teams used to them, API behaviours. Standardise on Minimal APIs with a shared endpoint-module convention for new services, but allow controllers where an existing team has a large surface; the pipeline is the same so the decision is about team ergonomics, not performance.

Q103. When do you choose gRPC over REST? [Senior]

Internal service-to-service calls needing low latency, strongly typed contracts, streaming and deadlines. Prefer REST for public/partner APIs, browser clients without gRPC-Web, caching and human-debuggability. In .NET, gRPC requires HTTP/2 and good load balancing; keep Protobuf contracts in a versioned shared package.

Q104. EF Core or Dapper? [Senior]

EF Core for domain-heavy write models (change tracking, migrations, aggregates, global filters). Dapper (or raw SQL) for read-heavy hot paths, complex reports, or bulk operations where you want exact SQL. Mixing them under CQRS is common and sound; ensure both use the same connection/transaction when needed.

Q105. SQL or NoSQL for a given service? Walk through your reasoning. [Tough]

Start from access patterns and consistency needs: relational integrity, ad hoc queries, transactions -> SQL. Known key-based access at massive scale, flexible schema, global distribution -> Cosmos DB/DynamoDB. Evaluate partition key cardinality, hot partitions, cost model (RU), consistency level. Default to relational unless a clear requirement exceeds it; polyglot adds operational cost.

Q106. How do you choose a Cosmos DB partition key? [Tough]

High cardinality, even distribution of both storage and request volume, present in most queries (to avoid cross-partition fan-out), and bounded partition size (20 GB logical limit). Consider synthetic or hierarchical keys (e.g. tenantId + date bucket). A poor key causes hot partitions and throttling that cannot be fixed without data migration.

Q107. Azure vs AWS vs GCP: how do you pick? [Senior]

Weigh existing licences and identity (Entra ID, Hybrid Benefit), team skills, required services and regional availability, compliance and data residency, pricing at expected scale, support relationship and lock-in tolerance. For a .NET/Windows-heavy enterprise Azure usually wins on integration and cost; for data/ML or Kubernetes-centric strategies GCP can be strong; AWS for breadth. Run a cost model and a spike rather than guess.

Q108. What is the real cost of multi-cloud? [Tough]

Duplicate skills and tooling, lowest-common-denominator services, double security and compliance work, cross-cloud egress charges and latency, complexity in identity and networking, and slower use of managed features. Benefits are narrow (regulation, M&A, specific best-of-breed). A multi-region single-cloud design usually gives better resilience per dollar.

Q109. What is lock-in and how do you manage it rationally? [Senior]

Cost to switch. Treat it as a spectrum and price it: use open standards where cheap (PostgreSQL, Kafka protocol, OpenTelemetry, containers, Terraform), accept managed proprietary services where they deliver value (Service Bus, Cosmos), isolate behind ports/adapters at strategic seams, keep data export paths, and record exit costs in ADRs. Total avoidance is more expensive than managed lock-in for most firms.

Q110. Docker vs VMs: why containers? [Senior]

Containers share the host kernel, start in seconds, are lightweight, image-based and reproducible, enabling density and CI/CD parity. VMs offer stronger isolation and full OS control (needed for some Windows legacy, kernel modules, strong multi-tenant isolation). Combine: containers in VMs (AKS nodes) with extra isolation (Kata, confidential containers) when required.

Q111. AKS vs Container Apps vs App Service vs Functions: decision rule? [Senior]

App Service: few web apps, minimal ops. Functions: event glue, bursts. Container Apps: microservices without a platform team, KEDA scaling, Dapr, scale to zero. AKS: many services, custom networking/operators/mesh, portability, dedicated platform team. Choose the simplest that meets requirements; migrate up when limits bite.

Q112. Argue for and against Kubernetes for a team of 8 with 6 services. [Tough]

For: standard deployment model, portability, autoscaling, ecosystem. Against: operational and cognitive load (networking, upgrades, security, cost), likely overkill at that size. Recommendation: Container Apps or App Service first; adopt AKS when scale, compliance (network control), or multi-team platform needs justify a platform team. Keep containers and Helm/Kustomize skills transferable.

Q113. Which frontend would you choose for an enterprise imaging or line-of-business web app: Angular, React, or Blazor? [Senior]

Angular suits large teams: opinionated structure, TypeScript, DI, long-term support, good for complex enterprise SPAs and existing Angular skills. React offers flexibility and ecosystem but needs more governance. Blazor shares C# models and fits .NET-heavy teams for internal tools, but heavy client-side graphics (e.g. WebGL viewers) often favour TypeScript ecosystems. Decide by team skills, UX complexity, and library needs (e.g. cornerstone.js for imaging).

Q114. How do you design for mobile clients differently? [Senior]

Assume unreliable, slow networks and long-lived old app versions: small payloads, pagination, delta sync, ETags, offline-first with conflict policy, idempotent writes with client-generated ids, PKCE auth with secure token storage, push notifications, backward-compatible API versions, and a BFF to shape responses.

Q115. Design choices for a hybrid cloud + on-prem deployment (e.g. hospital sites). [Tough]

Reliable connectivity (ExpressRoute/VPN, redundancy), identity federation, local autonomy during WAN loss (local cache, store-and-forward queue), data gravity decisions (what stays on site vs central), consistent tooling (Arc, GitOps, containers at the edge), staged sync with conflict rules, and security parity (encryption, patching). Define explicit degraded-mode behaviour.

Q116. How do you assess whether a library or framework is safe to adopt? [Senior]

Maintenance activity and bus factor, licence (and licence changes), security history, community size, documentation, compatibility with your runtime version, upgrade path, transitive dependencies, and ease of replacement. Wrap risky dependencies behind an interface; add SCA scanning; avoid abandoned packages.

Q117. MediatR/MassTransit licensing changes: how should an architect respond? [Tough]

Treat library licence changes as supply-chain risk. Options: pay for licences if the value is clear, pin the last open version (with security-patch risk), migrate to alternatives (Wolverine, Rebus, NServiceBus, Brighter, plain handler interfaces, native broker SDKs), or abstract at a seam. Verify the current licence terms before deciding. A good design keeps the messaging library at the edge so replacing it is a contained change.

Q118. Native AOT or JIT for a microservice? [Senior]

AOT improves startup, memory and image size (serverless, scale-to-zero, edge) but restricts reflection, dynamic code and some libraries (EF Core support is limited; Minimal APIs and source-generated serialisers work well). JIT with tiered PGO often wins peak throughput for long-running services. Choose AOT for functions and small gateways; JIT for general services.

Q119. Present a technology stack recommendation for a new multi-team clinical platform on Azure.

[Architect-level]

Backend: .NET LTS, ASP.NET Core Minimal APIs, gRPC internally, EF Core + Dapper; messaging: Service Bus (commands/workflow) plus Event Hubs for telemetry/streams; data: Azure SQL (transactional), Blob (images/documents), Redis (cache), AI Search (search); compute: Container Apps for most services, AKS only if a platform team exists; edge: Front Door + WAF, API Management for external APIs, YARP BFFs; identity: Entra ID with PKCE and managed identities; observability: OpenTelemetry to Azure Monitor/Grafana; delivery: GitHub/Azure DevOps with Bicep, container scanning, GitOps. Justify each against NFRs (security, availability, compliance, cost), and list exit costs and the first three decisions to validate by spike.

E9. System design and modeling

Q120. Difference between HLD and LLD artefacts? [Foundation]

HLD: context/container/deployment diagrams, technology choices, NFR mapping, integration points. LLD: class and sequence diagrams, API definitions, database schema, algorithms, error handling, state machines. HLD guides stakeholders; LLD guides implementation.

Q121. What are the four levels of the C4 model and who is each for? [Senior]

Context (everyone), Containers (architects, developers, ops), Components (developers for important containers), Code (rarely; generated). Principle: zoom in only as far as needed; each diagram has a clear scope, titles, key, and labelled relationships with technology.

Q122. What is a C4 "container"? [Senior]

A separately runnable or deployable unit that stores or processes data: web application, API, database, message queue, console app. It is not a Docker container, although it may be deployed as one.

Q123. When do you use sequence versus activity diagrams? [Senior]

Sequence: interactions between participants over time, with sync/async messages and failure branches; ideal for API and integration design. Activity: workflows and decision logic, with parallel flows and swimlanes; ideal for business processes and algorithms. State machine diagrams for lifecycle (Order: Draft, Confirmed, Shipped, Cancelled).

Q124. Show how a sequence diagram reveals a design problem. [Tough]

Long synchronous chains appear as deep stacks of nested calls; a missing timeout shows as an unbounded arrow; a dual write is visible as DB update followed by publish without an outbox; chatty N+1 as repeated loops; and circular calls as arrows returning to earlier participants. Drawing flows with failure branches (alt blocks) exposes missing compensation.

Q125. How do you decide what to put on a class diagram? [Senior]

Only what answers the current question: key aggregates, relationships, invariants and interfaces. Avoid listing every property; show multiplicity, composition vs association, and stereotypes (`<<aggregate root>>`). Keep diagrams small and purpose-specific, and generate detailed ones from code if needed.

Q126. How do you document an integration point? [Senior]

Name, direction, protocol and format, contract location and owner, authentication/authorisation, volume and SLA, error handling and retry policy, idempotency, data classification, versioning, and monitoring. Present as a table and as an interaction in a dynamic diagram.

Q127. Design a URL shortener. State your approach in 3 minutes. [Tough]

Clarify: 100M new URLs/month, 10:1 read:write, 5-year retention. Estimates: ~40 writes/s, 400 reads/s average (peaks 5x), ~6B records, ~500 bytes each = 3 TB. API: `POST /urls`, `GET /{code}` 301/302. Key generation: base62 of a distributed counter or pre-generated key ranges per node (avoid hashing collisions), 7 chars = 3.5T keys. Store: key-value/partitioned NoSQL keyed by code; cache hot codes in Redis/CDN; read path cache-aside; analytics async via events. Availability: stateless API behind a load balancer, multi-zone store. Trade-offs: 301 vs 302 (analytics), custom aliases, abuse/rate limiting, expiry cleanup.

Q128. Design a rate limiter for an API gateway. [Tough]

Algorithms: fixed window (simple, burst at edges), sliding window log/counter (accurate), token bucket (allows bursts, common), leaky bucket (smooth). Key by client/API key/IP. Distributed state: Redis with atomic Lua scripts or a gateway-local approximate limiter with periodic sync; local in-memory limiters give lower latency but are approximate across instances. Return 429 with `Retry-After` and `RateLimit-*` headers; fail-open vs fail-closed policy documented; per-tier quotas. .NET: built-in `AddRateLimiter` for per-instance limits plus Redis for global quotas.

Q129. Design a system to ingest and serve 2 million medical images per day with sub-second first-image display. [Architect-level]

Ingest: authenticated STOW-RS/DIMSE gateway, validate and write objects to blob storage (hot tier), emit `InstanceStored` events via outbox; metadata indexer writes to SQL/search for QIDO queries; idempotent by SOP Instance UID. Serve: WADO-RS with frame-level access, progressive/compressed transfer syntaxes, regional cache/CDN, prefetch of the current study and priors; metadata in cache. Scale: queue-based load leveling for bursts, autoscale on queue age and request rate; storage tiering to cool/archive by age with rehydration policy. Security: PHI encryption with CMK, audit every access, tenant/site-scoped authorisation. Reliability: zone-redundant storage, geo-replication with defined RPO, health-based routing. Test with realistic load (large objects, many small frames) and measure time-to-first-image at p95.

Q130. How do you model data flows and where do you validate and secure them? [Senior]

Draw a data-flow diagram with trust boundaries; label each flow with data classification and protocol; validate at every trust boundary (input validation, schema, size); authorise at each service; encrypt crossing boundaries; log with correlation but without sensitive payloads. This is also the base for threat modeling.

Q131. REST maturity model: what do levels 0-3 mean and does HATEOAS matter? [Senior]

L0 single endpoint (RPC over HTTP), L1 resources, L2 HTTP verbs and status codes, L3 hypermedia controls. Most production APIs live at L2; hypermedia helps discoverability and workflow evolution (links for next actions) but adds client complexity; use selectively (pagination links, long-running operations).

Q132. How do you design pagination for a changing dataset with millions of rows? [Tough]

Keyset (cursor) pagination: order by a stable unique key (e.g. `(createdAt, id)`), and use `WHERE (createdAt, id) > (@lastCreatedAt, @lastId) LIMIT n`. It is $O(\log n)$ with an index and stable under inserts, unlike offset which is $O(\text{offset})$ and skips/duplicates under changes. Return an opaque cursor token; bound page size; avoid total counts unless needed (estimate).

Q133. How do you make a POST endpoint idempotent? [Senior]

Require an `Idempotency-Key`; store key, request hash and response for a TTL in a table with a unique constraint; on repeat return the stored response; on concurrent duplicates lock or return 409/in-progress; scope key to the caller. Provide the same for payment calls to third parties.

Q134. Design the error model for a public API. [Tough]

RFC 9457 problem details with stable machine-readable `type` URIs, `status`, `title`, `detail`, `instance/traceId`, and extension members for validation errors (field paths); consistent mapping of domain exceptions to 4xx, never leaking stack traces or internals; correlation id in header; documented error catalogue; separate client errors (4xx, do not retry) from server errors (5xx, retry with backoff). ASP.NET Core: `AddProblemDetails()` and an `IExceptionHandler`.

Q135. How would you design API versioning for mobile clients that update slowly? [Senior]

Treat released APIs as long-lived; additive evolution; explicit major versions only for breaking changes; telemetry of version usage; minimum supported version policy with forced upgrade messaging; server-driven feature flags; avoid removing fields; test old clients against new servers in CI (contract tests).

Q136. What is a "bounded context map" for the hospital platform and where are the ACLs? [Tough]

Contexts: Ordering/Scheduling, Imaging Archive, Image Viewing, Reporting, Identity & Access, Billing, AI Services. ACLs at: HL7/FHIR integration with the EHR (translate foreign models), modality DICOM gateway (translate DIMSE/DICOM to internal events), billing system adapter, and third-party AI vendors. Upstream-downstream: Archive upstream of Viewing and AI (published language: DICOM metadata projections).

Q137. What is the difference between component, deployment and container diagrams? [Senior]

Component: logical building blocks within a container. Deployment: mapping of containers to infrastructure nodes (regions, clusters, VMs, networks). Container: the application and data-store units of the system with their technology and communication. Together they cover logical, runtime and physical views.

Q138. How do you keep architecture documentation from rotting? [Tough]

Docs-as-code in the repository (Markdown, Mermaid, Structurizr DSL), generated diagrams from code or IaC where possible, ADRs, review documentation changes in pull requests, automate checks (broken links, diagram builds), assign owners, and keep documents few and decision-focused. Delete docs nobody reads.

Q139. What belongs in a Definition of Done for architecture-significant stories? [Senior]

Observability (metrics, traces, alerts), security review (threat model delta), performance budget check, resilience (timeouts, idempotency), contract updated and compatible, ADR if a decision was made, runbook updated, and infrastructure as code changes included.

E10. Capacity planning and performance modeling

Q140. State Little's Law and use it. [Foundation]

$L = \lambda \times W$: average number in system equals arrival rate times average time in system. With 500 req/s and 200 ms average latency, concurrency is 100 in flight. It sizes thread/connection pools, instance counts and queue depth.

Q141. How do you size a service from requirements? [Senior]

Estimate peak requests/s (users x actions/min / 60 x peak factor), measure per-instance throughput at the target latency from a load test (not guess), divide for baseline, add N+1 for failure domains and 20-30% headroom, then validate dependencies (DB connections, broker throughput, downstream rate limits). Re-run after each major change.

Q142. Why target 60-70% utilisation instead of 90%? [Senior]

Queueing theory: latency grows hyperbolically as utilisation approaches 100% ($R = S/(1-\rho)$), so small load spikes cause large latency spikes, and failover capacity is needed when instances fail. Higher utilisation is acceptable for batch workloads without latency targets.

Q143. Explain coordinated omission. [Tough]

A load generator that waits for a slow response before sending the next request under-samples the period of slowness, hiding tail latency. Use open-model workloads with fixed arrival rates (k6 constant-arrival-rate, Locust constant throughput, wrk2) or correct the histogram (HdrHistogram expected-interval correction) so p99 reflects what users would experience.

Q144. JMeter vs Locust vs k6: when to choose each? [Senior]

JMeter: broad protocol support (JDBC, JMS, SOAP), GUI-based plans, existing enterprise skills, Azure Load Testing. Locust: Python-code scenarios, flexible logic, distributed. k6: JavaScript, efficient, thresholds as pass/fail in CI, great Grafana integration. For .NET shops NBomber is an option. Choose by team skills, protocol needs and CI integration; avoid GUI-recorded scripts as the only artefact.

Q145. Your load test shows throughput plateaus while CPU is at 30%. What do you check? [Tough]

Something else is saturated: thread-pool starvation (blocking calls, `ThreadPool.GetAvailableThreads`), connection pool limits (DB, HttpClient `MaxConnectionsPerServer`), lock contention, downstream rate limiting or latency, load generator limits, network bandwidth, GC pauses, single-threaded hot path, or database locks/IO. Use `dotnet-counters`, traces, and DB wait stats to find the bottleneck; do not add instances until the constraint is known.

Q146. What is the Universal Scalability Law telling you? [Tough]

Throughput scales sub-linearly due to contention (serialisation, sigma) and coherency cost (crosstalk, kappa), and may decline beyond a peak N. Fit the model to load-test points to find the saturation node count; reduce contention (partitioning, lock-free, async) and coherency (less shared state, fewer cross-node chats).

Q147. How do you capacity-plan storage growth for an imaging archive? [Senior]

Studies/day x average study size x retention, add replication factor and overhead; model compression ratios and modality mix; tier (hot/cool/archive) with lifecycle policies; account for priors prefetch and backups/geo-replication; forecast with growth rate and review quarterly; compute cost per TB per tier and egress for retrieval.

Q148. How do you validate autoscaling works? [Tough]

Spike and step tests measuring time-to-scale (metric lag + pod start + warm-up), ensure scale triggers on a leading indicator (queue depth/requests per pod, not CPU lag), configure min replicas for critical paths, pre-warm, check downstream capacity scales too (DB, caches), and test scale-in (graceful drain). Document the maximum surge the system can absorb during scale-out lag.

Q149. What metrics do you collect during a load test? [Senior]

Client side: throughput, p50/p95/p99 latency, error rate per endpoint. Server: CPU, memory, GC, thread pool, connection pool, request queue, DB (CPU, IO, waits, locks), cache hit ratio, broker lag, network. Correlate with traces. Record test conditions (data volume, version, config) for comparability.

Q150. The business expects 5x growth next year. How do you plan? [Architect-level]

Convert to quantified scenarios; identify the first bottleneck by load testing at 1x, 2x, 5x; model cost; classify limits (stateless scale-out is easy; database, hot partitions, single queues and licensing are hard); fix the structural ones first (partitioning, read replicas, caching, async), plan capacity increments with triggers (utilisation 60%), and review monthly. Present the plan as milestones with costs and risk.

Q151. What is the difference between performance, load, stress and soak tests? [Senior]

Performance: general measurement against targets. Load: expected peak. Stress: beyond capacity to find failure behaviour. Soak: long duration for leaks and degradation. Spike: sudden surge. Each answers a different risk.

E11. Containers and orchestration

Q152. What is the difference between an image and a container? [Foundation]

An image is an immutable, layered template; a container is a running (or stopped) instance of an image with a writable layer and runtime config. Many containers can run from one image.

Q153. How do you write a good Dockerfile for .NET? [Senior]

Multi-stage build (SDK for build, ASP.NET runtime for final), restore in a separate cached layer, publish with `--no-restore`, run as non-root, minimal/chiseled base, expose a non-privileged port (8080), health checks via orchestrator, no secrets, `.dockerignore`, pin base image versions/digests, and SBOM/scan in the pipeline.

Q154. How does the .NET runtime respond to container memory and CPU limits? [Senior]

GC respects cgroup memory limits (heap hard limit defaults to 75% of the limit) and `ProcessorCount` reflects CPU limit rounding; thread pool and Server GC heap count follow the limit. Set requests/limits consciously: too small CPU leads to throttling and latency spikes; memory limits too close to working set cause OOMKills. Prefer memory limits with headroom and measure.

Q155. Pods keep getting OOMKilled. How do you investigate? [Tough]

Compare container working set to limit, check GC settings and `GCHepHardLimit`, capture memory dumps (`dotnet-dump`), look for large object heap growth, unbounded caches/queues, native memory (e.g. image libraries), many simultaneous requests buffering bodies, and unbounded `Channels`. Fix root cause; adjust limits and concurrency limits; consider Server vs Workstation GC and `DOTNET_gcConcurrent`.

Q156. Deployment vs StatefulSet vs DaemonSet vs Job/CronJob? [Senior]

Deployment: stateless replicas with rolling updates. StatefulSet: stable identity and ordered storage (databases, brokers; prefer managed services). DaemonSet: one pod per node (log/metrics agents). Job/CronJob: run-to-completion and scheduled tasks.

Q157. How do you achieve zero-downtime rolling deployments? [Tough]

Readiness probes, `maxUnavailable: 0`, `maxSurge`, graceful shutdown (handle `SIGTERM`, stop accepting, drain within `terminationGracePeriodSeconds`, `preStop` delay for endpoint propagation), `PodDisruptionBudgets`, backward-compatible schema and contracts (expand/contract), and canary analysis with automated rollback.

Q158. What do requests and limits mean in Kubernetes? [Senior]

Requests are used by the scheduler and guaranteed share; limits are enforced ceilings (CPU throttling, memory OOMKill). Quality of Service classes: Guaranteed, Burstable, BestEffort. A common stance: set CPU requests (limits optional to avoid throttling) and memory requests = limits.

Q159. HPA vs KEDA vs VPA vs Cluster Autoscaler? [Senior]

HPA scales pod count on CPU/memory/custom metrics. KEDA scales on event sources (queue length, lag, cron) including to zero. VPA adjusts resource requests (restarts pods unless in-place). Cluster Autoscaler/Karpenter adds nodes when pods are unschedulable. Use them together with limits to avoid oscillation.

Q160. How do you secure a Kubernetes cluster? [Tough]

Private API server, Entra ID RBAC, namespaces with least privilege, Pod Security Standards (restricted), network policies (default deny), workload identity instead of secrets, Key Vault CSI, image signing and admission policy (Azure Policy/Gatekeeper/Kyverno), vulnerability scanning, node patching, runtime threat detection, audit logs, and encrypted etcd. Limit exposed ingress and use WAF.

Q161. What is a service mesh and do you need one? [Senior]

An infrastructure layer (sidecars or ambient) providing mTLS, traffic management (canary, retries, timeouts), and telemetry for all service-to-service traffic. Needed when you require uniform mTLS, advanced traffic shifting, and policy without code changes across polyglot services. Otherwise it adds overhead and complexity; many .NET shops use app-level resilience and network policies.

Q162. What is GitOps and what are its advantages and risks? [Tough]

Desired state stored in Git; an in-cluster operator (Flux/Argo CD) pulls and reconciles. Advantages: audit trail, easy rollback, drift correction, no cluster credentials in CI. Risks: secret management (use sealed/external secrets), repo structure sprawl, slow feedback for emergency fixes, and environment promotion conventions that must be designed.

Q163. Design a CI/CD pipeline for 12 teams delivering to a shared platform with security and compliance gates. [Architect-level]

Standardised pipeline templates (paved road) per stack: build, unit tests, SAST, SCA, secret scan, container build and scan, SBOM, sign; deploy to ephemeral/test environments via GitOps; contract and integration tests; performance smoke gate; manual or automated approval for regulated environments with evidence stored; progressive delivery with automated rollback. Policy as code (OPA/Azure Policy) enforces mandatory controls; platform team owns templates; teams customise within guardrails. Track DORA metrics per team.

Q164. What is the difference between blue/green and canary? [Senior]

Blue/green: two full environments, switch traffic at once (fast rollback, double capacity). Canary: shift a small percentage of traffic gradually, observing metrics (lower risk, needs good observability and routing). Feature flags decouple release from deploy and complement both.

Q165. How do database migrations fit zero-downtime deployments? [Tough]

Use expand/contract: deploy backward-compatible schema (add nullable columns/new tables), deploy app that writes both/reads old, backfill, switch reads, then remove old structures in a later release. Avoid long locks (online index builds, batched updates). Run migrations as a separate controlled step, not on app start in multi-instance deployments.

E12. Security, RBAC, encryption and threat modeling

Q166. Difference between authentication and authorisation? OAuth2 vs OIDC? [Foundation]

Authentication proves who you are; authorisation decides what you may do. OAuth 2.0 is a delegated authorisation framework issuing access tokens; OpenID Connect adds an identity layer (ID token, userinfo) on top for authentication.

Q167. Which OAuth flow for a SPA, a mobile app, a daemon, and an API calling another API on behalf of the user? [Senior]

SPA and mobile: Authorization Code with PKCE (SPA ideally behind a BFF holding tokens server-side). Daemon: Client Credentials, preferably managed/workload identity or certificate credentials. API to API on behalf of user: On-Behalf-Of (token exchange), keeping scopes minimal. Implicit and ROPC flows are deprecated.

Q168. What exactly must an API validate in a JWT? [Senior]

Signature against the issuer's keys (JWKS, cached with rotation), **iss**, **aud**, **exp**, **nbf**, algorithm allow-list (reject **none**), required scopes/roles/claims, and optionally token binding/**azp**. Treat the token as untrusted until validated; do not rely solely on gateway validation.

Q169. RBAC vs ABAC vs ReBAC? [Senior]

RBAC: permissions via roles; simple, auditable, role-explosion prone. ABAC: decisions from attributes of subject, resource, action, environment; expressive but harder to govern. ReBAC: access from relationships in a graph (owner, member, shares); suited for sharing-heavy domains. Real systems combine RBAC for coarse access with attribute/relationship checks for fine-grained rules.

Q170. How do you prevent IDOR/BOLA in a multi-tenant API? [Tough]

Never trust ids from the client alone: always scope queries by the authenticated tenant/user (EF Core global query filters, SQL row-level security), use authorisation handlers on the resource, prefer unguessable ids as defence in depth (not as the control), and include automated negative tests (user A requests user B's resource) in CI. Log denied attempts.

Q171. Where do you enforce authorisation: gateway or service? [Senior]

Both. Gateway: coarse checks (valid token, scope, rate limits) to reject early. Service: fine-grained, resource-aware decisions (the service owns the data and rules). Relying only on the gateway fails when a service is reachable another way or when business rules depend on data the gateway lacks.

Q172. Encryption in transit, at rest and in use: explain and give Azure examples. [Senior]

Transit: TLS 1.2+/1.3 at the edge and mTLS inside. At rest: storage service encryption, SQL TDE, Cosmos encryption, with platform or customer-managed keys in Key Vault/HSM; optionally column-level (Always Encrypted). In use: confidential computing (SGX/SEV enclaves). Key management with rotation and separation of duties matters as much as the algorithm.

Q173. Explain envelope encryption and why it is used. [Tough]

Data is encrypted with a data encryption key (DEK), and the DEK is encrypted (wrapped) by a key encryption key (KEK) held in an HSM/Key Vault. Benefits: the KEK never leaves the vault, rotation re-wraps small DEKs instead of re-encrypting data, per-object/tenant keys give crypto-shredding, and calls to the vault are infrequent. Use AEAD (AES-GCM) with associated data.

Q174. How do you manage secrets for 12 teams across environments? [Tough]

No secrets where possible: managed/workload identities for Azure resources. Where unavoidable, Key Vault per environment/team with RBAC, secrets accessed by identity, automated rotation, short-lived credentials, secret scanning in repos and pipelines, and break-glass procedures. Platform templates make the secure path the default.

Q175. What is mTLS and when do you use it? [Senior]

Mutual TLS: both client and server present certificates, giving strong service identity and encrypted transport. Use for east-west traffic in zero-trust networks, B2B partner APIs, and device authentication. Operational cost: certificate issuance, rotation, revocation; a mesh or Key Vault + cert-manager automates it.

Q176. What is the Confused Deputy problem in microservices? [Tough]

A service with high privilege is tricked by a lower-privileged caller into performing an action the caller could not do directly. Mitigate: propagate the **end-user identity** (token exchange/OBO) and authorise against it, validate audience on tokens, avoid blanket service-to-service trust, and use fine-grained service scopes.

Q177. Walk through STRIDE on an API that uploads patient images. [Senior]

Spoofing: strong token validation, mTLS for modality gateways. Tampering: TLS, hash/integrity check of the object, immutable storage. Repudiation: audit log of upload with user and device. Information disclosure: encryption, scoped SAS tokens, no PHI in logs/URLs. DoS: size limits, rate limits, queue buffering. Elevation: least-privilege identities, validate content-type/format, malware scanning, no path traversal on filenames.

Q178. How do you protect against SQL injection in .NET? [Senior]

Parameterised queries everywhere (EF Core LINQ, `FromSql` with interpolation which parameterises, `FromSqlRaw` only with parameters, Dapper parameters), no string concatenation, least-privilege DB accounts, input validation as a secondary control, WAF, and static analysis rules (CA2100).

Q179. What are the top API security risks beyond the classic OWASP Top 10? [Tough]

OWASP API Top 10: broken object-level authorisation, broken authentication, broken object property-level authorisation (excessive data exposure/mass assignment), unrestricted resource consumption, broken function-level authorisation, unrestricted access to sensitive business flows (abuse), SSRF, security misconfiguration, improper inventory management (shadow/zombie APIs), unsafe consumption of third-party APIs. Mitigate with DTOs (no entity binding), authorisation per object, quotas, API inventory and gateway governance.

Q180. How do you secure service-to-service calls in Azure without secrets? [Senior]

Managed identity (or workload identity federation on AKS) obtains Entra tokens for the target API/resource; the callee validates `aud` and app roles. Azure SDKs use `DefaultAzureCredential`. No client secrets, no rotation burden, auditable role assignments.

Q181. Design RBAC for a platform with clinicians, radiologists, admins, external partners and AI services. [Tough]

Define roles per capability (Viewer, Reporter, SiteAdmin, PlatformAdmin, PartnerReader, AIProcessor) mapped to scopes; add attribute checks for site/tenant and care relationship; break-glass role with reason capture and enhanced audit; service principals for AI with least-privilege per resource and time-bound tokens; admin actions require MFA and just-in-time elevation (PIM); regular access reviews. Avoid role explosion by combining a small role set with attributes/relationships; test a permission matrix automatically.

Q182. How does a WAF differ from an API gateway and a network firewall? [Tough]

Network firewall: L3/L4 filtering. WAF: L7 inspection for common web attacks (OWASP rules, bots) at the edge. API gateway: API-aware policy (auth, quotas, transformation, routing). They are complementary layers; each blocks different classes of attack.

Q183. What are secure defaults for an ASP.NET Core public API? [Senior]

HTTPS redirection and HSTS, fallback authorisation policy, anti-forgery for cookie sessions, CORS allow-list (no wildcard with credentials), security headers (CSP, X-Content-Type-Options, frame-ancestors), request size and timeout limits, rate limiting, problem-details errors without stack traces, structured audit logging, dependency scanning, and disabled developer exception pages in production.

Q184. A pen test found that any authenticated user can read any patient record by changing an id. What is your response plan? [Architect-level]

Immediate: containment (hotfix with ownership check, block exploit paths at WAF, rotate if needed), assess exposure by reviewing audit logs for abuse, notify per incident process and regulations if exploited. Root cause: authorisation enforced at endpoint but not object; fix systemically with global query filters/policy handlers, authorisation tests for every endpoint in CI, and code review checklist. Prevent: threat model update, architectural guardrail (repository requires a scope), SAST rule, periodic DAST. Share lessons across the 12 teams via the guild.

Q185. How do you design audit logging that is trustworthy? [Tough]

Log who, what, when, from where, on which resource, outcome and reason; time-synchronised; written append-only to a separate store with restricted write and no delete (immutability policies, hash chaining/WORM); avoid PHI payloads in log text (log ids); protect log pipelines; alert on anomalies; retain per regulation; make logging non-bypassable (middleware/interceptor), not developer-optional.

Q186. What are the privacy-by-design implications for an AI pipeline processing patient data? [Tough]

Minimise and de-identify data before leaving the trust boundary, handle identifiers inside image metadata and burned-in pixels, purpose limitation and consent tracking, keep training and inference data segregated, restrict vendor retention (no training on your data without agreement), log model inputs/outputs by id, run models inside private networks/regions, and apply DPIA and model governance.

Q187. What is a supply-chain attack and how do you defend in a .NET pipeline? [Senior]

Compromise via dependencies, build tools or artefacts. Defend: lock files and pinned versions, private feed with upstream controls, **NuGetAudit** and SCA, SBOM generation, signed packages/images, protected branches and pipeline permissions, ephemeral build agents, secret hygiene, verify provenance (SLSA), and review new dependencies.

Q188. How would you apply zero trust to an existing VM-hosted .NET application estate? [Tough]

Identity-centric controls first: central IdP with MFA, remove shared local accounts; segment networks and place apps behind a gateway/WAF; require TLS everywhere and mTLS where feasible; replace connection-string secrets with managed identities/Key Vault; add device and conditional access for admin; centralise logging and detection; adopt just-in-time access; progressively move toward per-service authN/authZ. It is an incremental programme, not a product.

E13. Reliability, availability, HA and DR

Q189. Define availability, reliability, RTO and RPO. [Foundation]

Availability: proportion of time the service is usable. Reliability: probability of performing correctly over time (failure rate, MTBF). RTO: maximum tolerable downtime after a disaster. RPO: maximum tolerable data loss measured in time. MTTR matters as much as MTBF for availability: $A = \text{MTBF} / (\text{MTBF} + \text{MTTR})$.

Q190. Compute the availability of a system with three serial components at 99.9% and a redundant pair at 99.9%. [Senior]

Serial: $0.999^3 = 99.70\%$. A redundant pair in parallel: $1 - (0.001)^2 = 99.9999\%$. Combined chain of the three serial plus the pair behaving as one component at 99.9999%: about 99.70%. The serial weak links dominate, so reduce the number of hard dependencies or make them redundant. Note the independence assumption: shared failure domains (same zone, same bad deploy) invalidate the arithmetic.

Q191. Difference between HA and DR? [Senior]

HA keeps the service running through component or zone failures automatically (redundancy, failover within a region). DR restores service after a catastrophic event (region loss, data corruption, ransomware) using backups/replicas in another location, usually with defined RTO/RPO and some manual or orchestrated steps.

Q192. Which DR strategy would you choose for RTO 30 min, RPO 5 min? [Tough]

Warm standby in a paired region: continuously replicated data (async geo-replication with RPO in seconds-minutes), scaled-down compute ready to scale, traffic steering via Front Door/Traffic Manager, IaC for scale-out, automated runbooks, and regular failover drills. Pilot light might miss 30 min if scale-out is slow; active-active exceeds the requirement at higher cost.

Q193. Why are replicas not backups? [Tough]

Replication copies corruption and deletions (accidental or malicious) instantly. Backups provide point-in-time recovery, immutability against ransomware, and separate failure domain. Keep both: replication for availability, versioned/immutable backups for recovery, and test restores.

Q194. How do you design for zone failure in AKS with a stateful dependency? [Tough]

Spread pods/nodes across zones (topology spread constraints), zone-redundant data services (zone-redundant SQL, ZRS storage, Service Bus premium zone redundancy), enough capacity to lose one zone (N+1: with 3 zones keep 50% extra headroom or 33% per-zone loss tolerance), PDBs, and test by zone-failure chaos experiments. Check cross-zone latency and egress cost.

Q195. What makes a good health check? [Senior]

Fast, cheap, dependency-aware for readiness (can I serve?), dependency-agnostic for liveness (am I wedged?), authenticated or restricted when verbose, versioned output for diagnostics, with timeouts. Avoid cascading checks that overload dependencies; use cached results for deep checks.

Q196. Design graceful degradation for a viewer when the AI service is down. [Tough]

Treat the AI as an optional enhancement: circuit breaker and short timeout on AI calls, show cached or "AI results unavailable" state, queue jobs for later processing, keep core viewing/reporting independent, flag degraded mode in UI and telemetry, and ensure no PHI is lost. Define which features are tier-1 (must work) vs tier-2 (may degrade).

Q197. Explain bulkhead with a .NET example. [Senior]

Isolate resources so one failing dependency cannot exhaust shared capacity. Examples: separate `HttpClient` instances/connection limits per downstream, a `SemaphoreSlim` or Polly bulkhead limiting concurrent calls to a slow API, separate thread/consumer pools or queues per tenant, separate pods per workload class.

Q198. What is the difference between retries, circuit breakers and timeouts, and in which order do you compose them? [Tough]

Timeout bounds each attempt, retry handles transient faults, circuit breaker stops hammering an unhealthy dependency. Typical pipeline (outer to inner): overall timeout -> retry -> circuit breaker -> per-attempt timeout (-> bulkhead). Retry only idempotent operations and keep total time within the caller's deadline.

Q199. How do you test your DR plan? [Tough]

Scheduled game days: failover to the secondary region in a production-like or production environment, restoring from backups into a clean environment, measuring RTO/RPO vs targets, validating runbooks, DNS/traffic switches, data integrity checks, dependency readiness (identity, keys, third-party allow-lists), and fail-back. Record findings and fix gaps.

Q200. Leadership asks for 99.999% availability. How do you respond? [Architect-level]

Quantify: 5.26 minutes of downtime per year, which includes deployments, dependency incidents and human error; cost grows exponentially. Establish the business impact per minute of downtime, define critical user journeys and tiers; propose 99.95-99.99% for the critical path with multi-zone/multi-region design, graceful degradation for the rest, and an error-budget policy. Show that dependencies (identity provider, DNS, cloud region SLAs) cap the achievable figure. Offer options with costs and let the business choose.

Q201. What is chaos engineering and how do you introduce it safely? [Senior]

Controlled experiments that inject failures to test resilience hypotheses. Steps: define steady-state metrics, hypothesise, limit the blast radius, run in non-production first, automate abort conditions, monitor, and fix what you find. Tools: Azure Chaos Studio, Chaos Mesh, Litmus. Culture and observability are prerequisites.

Q202. How do you handle partial failure in a request that fans out to five services? [Tough]

Set per-call timeouts and a global deadline, call in parallel, use hedged requests sparingly, return partial results with explicit markers where acceptable, cache fallbacks, circuit-break chronic failures, and shed low-priority calls. Decide per field/feature whether it is required or optional and encode this in the aggregator.

Q203. What is graceful shutdown in .NET hosting and why does it matter? [Senior]

On SIGTERM `IHostApplicationLifetime` triggers `ApplicationStopping`; Kestrel stops accepting new connections and drains in-flight requests up to `HostOptions.ShutdownTimeout` (and Kubernetes `terminationGracePeriodSeconds`). Hosted services should honour the cancellation token and complete or requeue work. Without it, rolling updates drop requests and leave messages in flight.

E14. Performance, caching and cost

Q204. A .NET API is slow under load, CPU low. Likely causes? [Senior]

Thread-pool starvation from sync-over-async or blocking calls, connection pool exhaustion, slow downstream or DB waits, lock contention, excessive allocations causing GC pauses, large payload serialisation, N+1 queries, or synchronous logging. Diagnose with `dotnet-counters` (thread pool queue length), traces, DB wait stats and allocation profiling; then fix the specific constraint.

Q205. Why avoid `.Result` and `.Wait()` in ASP.NET Core? [Senior]

They block threads while waiting on async work. Under load the thread pool starves (it grows slowly), latency increases and timeouts cascade. Async all the way, use `ConfigureAwait(false)` in libraries, and use `await` with cancellation tokens.

Q206. What are common EF Core performance pitfalls? [Senior]

N+1 lazy loading, loading whole entities for read-only views (use projections and `AsNoTracking`), cartesian explosion with multiple includes (`AsSplitQuery`), tracking large result sets, `Contains` with huge lists, per-row updates instead of `ExecuteUpdate/ExecuteDelete`, missing indexes, and creating `DbContext` per call without pooling at high rates. Inspect generated SQL and query plans.

Q207. Explain cache-aside, write-through and write-behind and their pitfalls. [Tough]

Cache-aside: app reads cache, on miss loads DB and populates (simple; stale risk, stampede). Write-through: writes go through the cache to the DB (consistent reads; write latency). Write-behind: writes buffered then flushed asynchronously (fast; risk of loss, ordering complexity). Mitigate stampede with single-flight and jittered TTLs; invalidate by event.

Q208. What is HybridCache and why use it? [Tough]

A .NET caching abstraction (from .NET 9) combining in-process L1 with distributed L2 (Redis), providing stampede protection (single-flight per key), tag-based invalidation and serialisation handling through `GetOrCreateAsync`. It reduces boilerplate and prevents duplicate concurrent factories. Still design keys and TTLs deliberately.

Q209. How do you reduce tail latency (p99)? [Tough]

Eliminate queueing (right-size, load shedding), avoid long GC pauses (allocation reduction, appropriate GC mode), timeouts and hedged requests, avoid head-of-line blocking (split slow and fast work), warm caches/JIT (ReadyToRun/tiered PGO), avoid cold starts via min replicas, optimise slow queries, and isolate noisy neighbours. Measure with percentiles and traces, not averages.

Q210. Where does the cost of a cloud-native system usually hide? [Senior]

Idle over-provisioned compute, data egress and cross-zone/region traffic, log/trace ingestion volume, premium database tiers, storage never tiered or deleted, non-production environments running 24x7, and the people cost of operating complex platforms. Use tags, budgets, anomaly alerts and unit cost metrics.

Q211. Reserved instances vs savings plans vs spot: how do you combine them? [Tough]

Commit (reserved/savings plan) to the stable baseline you are confident of running for 1-3 years; use on-demand autoscaling for variable load; use spot for fault-tolerant batch and workers; apply Hybrid Benefit where licences exist. Reassess commitments quarterly as architecture changes (e.g. migration to Container Apps) to avoid stranded commitments.

Q212. Reduce the cloud bill by 30% without hurting reliability. Your plan? [Architect-level]

Baseline and attribute costs per service; eliminate waste (unused resources, idle environments, oversized SKUs); right-size using utilisation data; schedule non-prod shutdown; storage lifecycle tiering; reduce observability noise; evaluate architecture-level changes (merge chatty services, cache, compress, regional placement to cut egress); commit to savings plans for baseline; verify with SLO dashboards that reliability is unchanged; set ongoing FinOps governance with unit cost KPIs.

Q213. What is "performance budget" and how is it enforced? [Senior]

An allocated share of the latency/resource goal to each component (e.g. 300 ms p95: gateway 20, service 80, DB 100, network 50, margin 50). Enforced via load tests in CI with thresholds, tracing-based SLO dashboards, and rejection of changes exceeding allocations.

E15. .NET deep-dive for senior architects

Q214. Explain the ASP.NET Core request pipeline and middleware ordering. [Senior]

A chain of middleware delegates built in `Program.cs`; each can short-circuit. Order matters: exception handling, HTTPS/HSTS, static files, routing, CORS, authentication, authorisation, rate limiting, then endpoints. Authentication must precede authorisation; CORS must precede both for preflight handling; exception handler first to catch everything downstream.

Q215. Singleton, scoped, transient: pitfalls? [Senior]

Capturing a scoped service in a singleton (captive dependency) causes stale or thread-unsafe state; scope validation in development catches it. `DbContext` is scoped and not thread-safe; do not share across parallel tasks. Transient `IDisposable`s resolved from the root provider are held until app end. Use `IServiceScopeFactory` in background services.

Q216. Why must you not create `new HttpClient()` per request, and what does `HttpClientFactory` give? [Tough]

Per-request clients exhaust sockets (`TIME_WAIT`) and ignore DNS changes when reused forever. `HttpClientFactory` pools and rotates `HttpMessageHandlers` (handler lifetime), centralises config, named/typed clients, and integrates with resilience handlers and telemetry. In .NET 5+ `SocketsHttpHandler` with `PooledConnectionLifetime` is also an option.

Q217. What is `async/await` really doing, and when is `ValueTask` appropriate? [Senior]

The compiler turns an async method into a state machine; `await` yields the thread back until the awaited operation completes, resuming via synchronisation context/thread pool. `ValueTask` avoids allocation when results are often available synchronously (cached paths); do not await it twice or store it. Default to `Task` unless profiling shows allocation pressure.

Q218. Describe the .NET garbage collector aspects an architect should know. [Tough]

Generational (0, 1, 2) plus LOH/POH; Workstation vs Server GC (Server uses per-core heaps and more throughput, more memory); concurrent/background GC reduces pauses; container limits shape the heap; allocation rate dominates pause frequency. Reduce allocations (pooling, `Span<T>`), avoid LOH churn (>85 KB), consider `GCSettings.LatencyMode` and `DOTNET_gcServer` per workload.

Q219. How do you implement background processing in .NET? [Senior]

`BackgroundService/IHostedService` for in-process loops (consume from `Channel<T>` or a broker); queue-triggered Functions or workers for scale-out; Hangfire/Quartz for scheduling; Durable Functions/Temporal for workflows. Ensure idempotency, cancellation, scoped services via scope factory, and graceful shutdown. For horizontally scaled instances, use leases/leader election for singleton jobs.

Q220. What is the options pattern and `IOptionsMonitor`? [Senior]

Strongly typed configuration binding with validation (`ValidateDataAnnotations`, `ValidateOnStart`). `IOptions<T>` is a singleton snapshot; `IOptionsSnapshot<T>` is per request; `IOptionsMonitor<T>` supports change notifications for singletons. Validate at startup to fail fast.

Q221. How do you implement multi-tenancy in EF Core? [Tough]

Tenant id on every row with a global query filter using the current tenant from `ITenantProvider`; set the tenant in a `SaveChanges` interceptor; consider separate schemas or databases for isolation (connection per tenant, dynamic `DbContext` configuration); add DB-level row-level security as a backstop; test that queries without tenant context fail.

Q222. What does `Microsoft.Extensions.Http.Resilience` add over raw Polly? [Tough]

Opinionated, pre-tuned pipelines (`AddStandardResilienceHandler` with rate limiter, total timeout, retry, circuit breaker, attempt timeout) integrated with `IHttpClientFactory` and telemetry. Built on Polly v8 `ResiliencePipeline`. Use custom pipelines for non-HTTP calls or special policies.

Q223. What is Aspire (formerly .NET Aspire) and what problem does it solve? [Senior]

An opinionated stack for cloud-ready distributed apps: an app host describing resources and projects (databases, queues, services), service discovery and configuration wiring, default telemetry (OpenTelemetry), health checks, a dev dashboard, and deployment manifests for targets like Azure Container Apps or Kubernetes. It reduces local-dev and boilerplate friction; it is not a runtime orchestrator in production by itself. Check current docs because the product has evolved quickly.

Q224. How do you handle long-running work and timeouts in ASP.NET Core minimal APIs? [Tough]

Keep requests short; enqueue work (202 + status endpoint); set request timeouts middleware (`AddRequestTimeouts`), propagate `HttpContext.RequestAborted` cancellation; for streaming use `IAsyncEnumerable` or SSE/SignalR; for big uploads stream to blob rather than buffering and configure Kestrel limits.

Q225. How would you design strongly consistent counters under concurrency in SQL? [Senior]

Use atomic updates (`UPDATE ... SET Count = Count + 1`), optimistic concurrency with `rowversion` retries, or sequence/identity for ids. Avoid read-modify-write in the app. For hot rows use sharded counters aggregated later, or push to Redis `INCR` and persist asynchronously.

Q226. Explain optimistic concurrency in EF Core and how to expose it via API. [Tough]

Mark a `rowversion`/token property as `IsRowVersion()` or `IsConcurrencyToken()`; `SaveChanges` throws `DbUpdateConcurrencyException` when the row changed. Expose the token as an ETag; require `If-Match` on PUT/PATCH; map the exception to `412 Precondition Failed` (or 409), and let the client re-fetch and retry.

Q227. How do you design the outbox in EF Core in practice? [Tough]

Add `OutboxMessage` table in the same DbContext; in a `SaveChanges` interceptor convert domain events to outbox rows (same transaction); a hosted service polls with `SELECT ... WITH (UPDLOCK, READPAST)` batches, publishes, then marks processed; use unique message ids, exponential retry, and clean-up of old rows. Alternatively CDC via Debezium. Monitor outbox age.

Q228. What are the main ways to scale SignalR? [Senior]

Use a backplane (Redis or Azure SignalR Service) to share messages across instances, sticky sessions if not using the managed service, and consider connection limits per node. Azure SignalR Service offloads connections for large scale. Design clients to reconnect and resync state.

Q229. Source generators, minimal reflection and AOT: why should architects care? [Tough]

They cut startup time and memory (serverless/scale-to-zero), enable trimming and smaller images, and improve throughput by removing runtime reflection (System.Text.Json source gen, regex gen, logging gen `LoggerMessage`). Choose libraries compatible with trimming/AOT when targeting such deployments.

Q230. How do you structure a .NET solution for multiple services and shared contracts? [Senior]

Repo strategy: mono-repo with per-service folders and independent pipelines (path filters) or poly-repo per service. Shared: a small `Contracts` package (events/Protobuf) with semantic versioning, and `BuildingBlocks` for infrastructure helpers (logging, auth, health). Use central package management (`Directory.Packages.props`), analyzers, and a service template (`dotnet new`) to keep conventions.

Q231. Describe how you would migrate an ASP.NET 4.8 / WCF / WPF estate to modern .NET incrementally. [Tough]

Assess with .NET Upgrade Assistant and API portability analyzer; stabilise tests; extract shared libraries to .NET Standard 2.0 then multi-target; put YARP in front of web apps and strangle endpoints to ASP.NET Core; replace WCF with CoreWCF/gRPC per consumer needs; keep WPF on .NET (WPF is supported on modern .NET on Windows) after migrating libraries; move to containers and Azure PaaS in stages; keep CI/CD, monitoring and security parity throughout. Measure by traffic and component retirement.

Q232. What are common pitfalls in using `System.Text.Json` for APIs between services? [Senior]

Case sensitivity defaults, missing/unknown fields behaviour, `DateTime` vs `DateTimeOffset` and time zones, enum as string vs number (version drift), polymorphism requires explicit config, large numbers (`long`) to JS clients, and nullable annotations. Use source generation, shared naming policy, and contract tests.

Q233. How do you manage time and ordering in distributed systems? [Tough]

Do not rely on wall-clock ordering across machines (skew). Use logical versions per aggregate, sequence numbers, vector clocks for conflict detection, hybrid logical clocks, or database-assigned sequences. Use `TimeProvider` for testability and store timestamps in UTC (`DateTimeOffset`).

E16. Scenario and case-study questions

These open-ended cases mirror how senior evaluators probe judgement. Use the structure: **clarify -> estimate -> design -> deep-dive risk -> failure modes -> trade-offs**. Model answers are condensed; expect follow-ups after each paragraph.

Q234. Case 1: Your POC splits an order system into 9 services. In staging, p95 latency is 2.4 s for "place order". What is your investigation and redesign? [Architect-level]

Trace a single request to see the span tree: count hops, serial vs parallel calls, time in each. Typical findings: synchronous chain of 5 services, repeated authorisation lookups, N+1 calls to Catalog, no connection reuse, and cold caches. Remedies in order: (1) parallelise independent calls; (2) replace calls for reference data with local replicas or caching; (3) move non-essential work (email, analytics, loyalty points) behind events after commit; (4) merge services with chatty interaction; (5) optimise slow queries and add timeouts. Re-test against the 300 ms budget and keep a per-hop latency budget in the design.

Q235. Case 2: Two teams disagree: one wants Kafka, the other Service Bus. You decide. [Architect-level]

Elicit requirements instead of picking by preference: need for replay and stream processing, throughput (events/s), ordering granularity, per-message settlement/DLQ/scheduling, team skills, ops capacity and cost. If the primary use is workflow commands with retries and DLQ at modest volume, Service Bus; if the platform needs an event backbone with replay and multiple analytical consumers, Kafka/Event Hubs. Allowed hybrid: Service Bus for commands, Event Hubs for telemetry/event streams, with an outbox-to-bridge. Record the ADR, define the boundary and the revisit trigger (e.g. need for >10x throughput or replay).

Q236. Case 3: A critical downstream partner API is unreliable (5% timeouts). Your checkout depends on it. Design for resilience. [Architect-level]

Timeout (2 s) and limited retries with jitter for idempotent calls; circuit breaker; bulkhead to protect your thread pools; idempotency keys for payment-like calls; **asynchronous path**: accept the order as Pending, queue the partner call with retries, notify when done; cache stable partner data; fallbacks (alternative provider, degrade feature); monitor partner SLO and enforce contractual SLAs; synthetic probes and dashboards. Make user experience explicit: "we are confirming your order".

Q237. Case 4: You must prove the platform can handle a hospital group's 3x volume at month-end. What do you deliver? [Architect-level]

A capacity model (arrival rate, concurrency by Little's law, storage and bandwidth), a production-like environment, realistic load profiles (open model, think time, real data distribution, large payload mix), scenarios for load, stress, spike, soak and failure injection (kill a zone), SLO thresholds, a bottleneck analysis report with remediations, autoscaling validation, and a go/no-go summary with headroom statement and risk register. Include cost at 3x.

Q238. Case 5: Security asks for end-to-end encryption of PHI with customer-managed keys, and Operations asks to keep full-text search. Reconcile. [Architect-level]

Classify fields: encrypt sensitive fields at the application layer with envelope encryption (CMK in Key Vault/HSM) and search on **tokenised or blind-index** values for equality (HMAC of normalised value) while keeping non-sensitive fields searchable; for free text, either limit indexed content, search within an encrypted enclave/confidential compute boundary, or restrict to a de-identified index. Be explicit about the residual risk, key rotation, and performance impact. Always Encrypted with enclaves is an option for SQL; evaluate limits.

Q239. Case 6: A single "Customer" service has grown into a god service used by 25 callers. Plan the decomposition. [Architect-level]

Analyse call graph and data usage per consumer; identify distinct bounded contexts hidden inside (Identity, Profile, Preferences, Billing contact, Consent); introduce a facade, then extract the most independent capability first via strangler with events keeping data in sync; give each new service its own store; use consumer-driven contracts to migrate callers incrementally; track by caller count and shared-table usage; stop when change-coupling drops. Avoid creating nano-services by merging items that always change together.

Q240. Case 7: The same domain event is consumed by 7 services and a bug in the producer sent a wrong payload for two hours. Recover. [Architect-level]

Stop the bleeding (fix or disable the producer), identify the affected event range using ids/time, decide per consumer whether it is idempotent and whether corrections can be reapplied; publish **corrective events** (not edit history) or replay corrected events with a flag; coordinate with consumer owners; verify with reconciliation reports; add schema validation, contract tests and a canary consumer to detect payload anomalies; improve observability on event quality.

Q241. Case 8: Migrate a VM-hosted ASP.NET 4.8 application with a shared SQL database into the cloud with minimal downtime. [Architect-level]

Phase 1 rehost to Azure VMs/Managed Instance with Azure Migrate and DMS (online migration with continuous sync, cutover window short); put the app behind Front Door/App Gateway; introduce CI/CD and monitoring. Phase 2 replatform: App Service or containers for stateless web, Key Vault, Service Bus for async. Phase 3 refactor to .NET LTS and modular components; strangle with YARP. Validate with performance baselines before and after; keep rollback (reverse sync) until stable; plan DR in the new region.

Q242. Case 9: Design multi-region active-active for a stateful web application. [Architect-level]

Choose a data strategy first: globally distributed DB with multi-write (Cosmos) and conflict resolution, or region-partitioned ownership (each tenant has a home region, async replication for DR, route by tenant). Stateless compute per region behind global load balancing; session state in cookies/distributed cache per region; eventual-consistency semantics and conflict policy (LWW, merge, domain rules); cross-region messaging with idempotency; failover tests; cost and complexity justify only if RTO/RPO or latency require it. Often **warm standby + regional read replicas** is the pragmatic answer.

Q243. Case 10: The CTO asks "what is our architecture's biggest risk?" in 5 minutes. How do you answer? [Architect-level]

Give a ranked risk view tied to business impact: identify the top 3 from evidence (single points of failure, untested DR, shared database coupling, absence of observability, security debt, key-person dependency), with likelihood, impact, mitigation and cost/time, and one decision you need from leadership. Offer to back with data (incident history, dependency graph, SLO attainment). Concrete beats comprehensive.

E17. Rapid-fire round (one-liners)

Aim to answer each in one or two sentences. These are the questions evaluators use to test breadth quickly.

#	Question	Crisp answer
R1	CAP theorem in one line?	During a partition choose consistency or availability; partitions are not optional in distributed systems.
R2	What is eventual consistency?	Replicas converge if no new updates occur; reads may be stale meanwhile.
R3	Idempotency?	Applying an operation many times has the same effect as once.
R4	Difference between 401 and 403?	401: not authenticated (or invalid credentials). 403: authenticated but not allowed.
R5	What does 409 mean?	Conflict with current resource state (duplicate, version mismatch).
R6	PUT vs PATCH?	PUT replaces the whole resource idempotently; PATCH applies partial modification.
R7	Sync-over-async?	Blocking on async code with <code>.Result/.Wait()</code> ; risks deadlocks and thread-pool starvation.
R8	What is a captive dependency?	A longer-lived service (singleton) holding a shorter-lived one (scoped).
R9	Saga compensation is...	A semantic undo action, not a rollback.
R10	Why not share a database?	It couples schema, deploys and ownership; breaks encapsulation.
R11	Strangler fig pattern?	Incrementally route functionality to a new system behind a facade until the old one is retired.
R12	Circuit breaker states?	Closed, open, half-open.
R13	What is jitter for?	Randomises retry delays to avoid synchronised retry waves.
R14	What is a DLQ?	Holding queue for messages that cannot be processed, for inspection and replay.
R15	At-least-once means...	No loss but possible duplicates; consumers must be idempotent.
R16	Claim check?	Send a reference to a large payload stored elsewhere.
R17	BFF?	Backend per frontend type that aggregates and shapes data for it.
R18	Sidecar?	Helper container deployed with the main app in the same pod.
R19	Blue/green vs canary?	Switch whole environments vs shift a small traffic share gradually.
R20	Liveness vs readiness?	Restart me vs send me traffic.
R21	RTO vs RPO?	Time to recover vs amount of data you can lose.
R22	Backup vs replica?	Backup is point-in-time and isolated; a replica copies changes including damage.
R23	Little's Law?	$L = \lambda \times W$.
R24	Why p99, not average?	Averages hide the slow tail that users feel and that drives timeouts.
R25	What is TLS termination?	Decrypting HTTPS at an edge component (gateway, load balancer), then forwarding on an internal hop (re-encrypt for zero trust).
R26	What is PKCE?	Proof key for code exchange: protects authorisation code flow for public clients against code interception.
R27	JWT vs opaque token?	JWT is self-contained, validated locally; opaque requires introspection but is revocable and leaks nothing.
R28	Salt vs pepper?	Salt: per-hash random value stored with the hash. Pepper: secret global value stored outside the DB.
R29	Hash vs encrypt?	Hash is one-way (integrity, passwords with slow KDF); encryption is reversible with a key.

#	Question	Crisp answer
R30	What is HSTS?	Header telling browsers to use HTTPS only for the domain.
R31	CORS purpose?	Browser policy letting servers allow cross-origin requests; not a server-side security control.
R32	CSRF vs XSS?	CSRF abuses a user's authenticated session to send actions; XSS injects script into pages. Different mitigations (anti-forgery tokens vs output encoding/CSP).
R33	Principle of least privilege?	Grant only the minimum access needed, for the shortest time.
R34	Defence in depth?	Multiple independent security layers so one failure is not fatal.
R35	Shift-left security?	Move security checks earlier (design, code, CI).
R36	What is SBOM?	Software bill of materials: inventory of components for vulnerability tracking.
R37	Kubernetes pod vs container?	A pod wraps one or more containers sharing network and storage.
R38	What does a Service do?	Gives pods a stable virtual IP/DNS and load balances across ready endpoints.
R39	Helm?	Package manager/templating for Kubernetes manifests.
R40	Immutable infrastructure?	Replace rather than modify servers/images; changes ship as new versions.
R41	IaC tools?	Bicep/ARM, Terraform, Pulumi, CloudFormation.
R42	What is DORA?	Four delivery metrics: deploy frequency, lead time, change failure rate, time to restore.
R43	Fan-out vs fan-in?	One producer to many consumers vs many sources aggregated into one.
R44	Event vs message?	Event is a fact; message is the transport envelope (command, event, reply).
R45	Value object equality?	By all attribute values, not identity.
R46	What is a domain service?	Stateless domain logic not belonging to one entity.
R47	Unit of Work?	Tracks changes and commits them atomically (DbContext).
R48	Specification pattern?	Encapsulated, composable business rule or query predicate.
R49	Why use ETag?	Cache validation and optimistic concurrency.
R50	What is HATEOAS?	Responses include links describing available next actions.
R51	gRPC deadlines?	Absolute time limits propagated across calls so downstream stops working when the caller gives up.
R52	Why cursor pagination?	Stable and fast on large changing data; offset is slow and inconsistent.
R53	Hot partition?	Skewed key distribution concentrating load on one partition and throttling it.
R54	Sharding vs partitioning?	Partitioning splits data within a system; sharding spreads partitions across nodes/databases.
R55	Read replica use?	Offload reads; beware replication lag and read-your-writes needs.
R56	What is a quorum?	Majority agreement ($W + R > N$) used for consistency in replicated systems.
R57	Exactly-once delivery?	Practically achieved as at-least-once plus idempotent processing.
R58	What is a tolerant reader?	Consumer ignores unknown fields and accepts optional ones, enabling additive evolution.
R59	Orchestration vs choreography?	Central controller sends commands vs services react to each other's events.
R60	What is a feature flag for?	Decouples deployment from release; enables canary, kill switches and experiments.

Part F. Cheat Sheets, Drills and Self-Assessment

F1. Pattern selection cheat sheet

Problem	Reach for	Beware
Atomically update DB and publish event	Transactional outbox + idempotent consumers	Outbox lag; relay failure monitoring
Multi-service business transaction	Saga (orchestrated if complex)	Compensation failure; lack of isolation
Slow or unreliable dependency	Timeout, retry+jitter, circuit breaker, bulkhead, fallback	Retry storms; hiding failures
Spiky load	Queue-based load leveling, autoscale on queue depth	Queue age; DLQ growth
Expensive repeated reads	Caching (HybridCache/Redis), CQRS read model	Stale data; stampede
Clients need tailored responses	BFF / GraphQL	Business logic leaking into BFF
Legacy replacement	Strangler fig + ACL	Dual maintenance; data sync
Large payloads in messages	Claim check	Dangling references; access control
Need audit and history	Event sourcing or append-only audit	Complexity; schema evolution; erasure
Different read/write shapes or scale	CQRS (levels 1-3)	Eventual consistency; operational overhead
Cross-cutting infra without code changes	Service mesh / sidecar	Overhead; complexity
Singleton job in many instances	Leader election / distributed lease	Split brain; fencing tokens
Prevent duplicate side effects	Idempotency key / inbox	Key storage and TTL
Protect against overload	Rate limiting, load shedding, backpressure	Fairness and priorities

F2. NFR to mechanism cheat sheet

NFR	Primary mechanisms	Verify with
Availability	Zones/regions, redundancy, health probes, graceful degradation, safe deploys	Chaos tests, SLO dashboards, failover drills
Scalability	Statelessness, partitioning, autoscaling, async decoupling	Load/stress/spike tests
Performance	Caching, async I/O, indexes, batching, compression, CDN	Percentile SLOs, profiling
Security	IdP + OAuth2/OIDC, RBAC/ABAC, TLS/mTLS, encryption, WAF, secrets mgmt, scanning	Threat model, pen test, DAST/SAST
Reliability	Idempotency, retries, timeouts, outbox, DLQ, replay	Fault injection, replay tests
Maintainability	Modular boundaries, tests, ADRs, fitness functions, observability	Change lead time, coupling metrics
Cost	Right-size, autoscale, commitments, storage tiers, egress control	Unit cost KPIs, budgets
Compliance	Audit trail, residency, retention, pseudonymisation, access reviews	Audits, control evidence

F3. The 10-step answer structure for any design question

- **1. Clarify** scope, users, scale, constraints, success measures.
- **2. State assumptions** and NFR targets with numbers.
- **3. Estimate** load, storage, bandwidth (back of envelope).
- **4. Draw HLD**: context, containers, data stores, flows.
- **5. Define APIs and data model** for the core use case.
- **6. Deep-dive** the riskiest part (consistency, scale, security).
- **7. Failure modes**: what breaks, how detected, how recovered.
- **8. Security and compliance** at each boundary.
- **9. Operations**: observability, deployment, cost.
- **10. Trade-offs**: alternatives rejected, costs accepted, revisit triggers.

F4. Questions you should ask the evaluator (and yourself)

Area	Question
Business	What is the cost of one hour of downtime? Which journeys are critical?
Scale	Expected peak, growth rate, data volumes, tenant count?
Consistency	Which data needs strong consistency; what staleness is acceptable?
Compliance	Data residency, retention, audit requirements, certifications?
Team	Team size, skills, operational maturity, on-call capacity?
Legacy	Which systems must we integrate with; what can we change?
Budget & time	Cost envelope and delivery milestones?
Risk	What is the biggest uncertainty to validate first?

F5. Self-assessment rubric

Score yourself 0-3 on each row (0 cannot explain, 1 can define, 2 can apply with an example, 3 can discuss trade-offs and failure modes). Target 2+ across the board and 3 in your own specialism.

Topic	0	1	2	3
Quality attributes and ATAM-style trade-offs				
Microservices boundaries and data ownership				
Messaging: outbox, idempotency, ordering, DLQ				
Sagas and eventual consistency				
DDD tactical and strategic design				
CQRS and event sourcing (when and when not)				
API design, versioning, contracts				
Cloud vendor and platform selection				
Docker, Kubernetes, delivery pipelines				
UML and C4 modeling				
Capacity planning, JMeter/Locust methods				

Topic	0	1	2	3
Security: OAuth/OIDC, RBAC, encryption, threat modeling				
HA/DR, resilience patterns, chaos				
.NET internals: async, DI, EF Core, GC, Aspire				
Cost optimisation and FinOps				

F6. Four-week preparation plan

Week	Focus	Practice
1	Principles, quality attributes, microservices and messaging	Write 3 ADRs from your POC; draw C4 L1/L2; answer E1-E3 aloud
2	DDD, CQRS, ES, sagas, anti-patterns	Model one bounded context with aggregates; implement outbox and an idempotent consumer; answer E4-E7
3	Stack selection, design/modeling, capacity, containers	Run a Locust/JMeter test and write the capacity estimate; Dockerfile and K8s manifest hardening; answer E8-E11
4	NFRs: security, HA/DR, performance, cost; mock evaluation	Threat-model your POC (STRIDE); write a DR plan with RTO/RPO; timed case studies E16; peer mock interview

F7. Glossary

Term	Meaning
ACL	Anti-corruption layer: translation boundary protecting a domain model
ADR	Architecture decision record
ATAM	Architecture trade-off analysis method
BFF	Backend for frontend
CDC	Change data capture from a database log
CQRS	Command query responsibility segregation
DLQ	Dead-letter queue
DDD	Domain-driven design
DORA	DevOps research and assessment delivery metrics
ES	Event sourcing
HPA / KEDA	Horizontal pod autoscaler / event-driven autoscaler
IaC	Infrastructure as code
mTLS	Mutual TLS
PACELC	CAP extension: else latency vs consistency
PKCE	Proof key for code exchange
RPO / RTO	Recovery point / time objective
SLI / SLO / SLA	Service level indicator / objective / agreement
STRIDE	Spoofing, tampering, repudiation, information disclosure, DoS, elevation of privilege
USL	Universal scalability law

Term	Meaning
YARP	Yet another reverse proxy (Microsoft .NET reverse proxy toolkit)

Key idea

Final advice for the evaluation. Speak in trade-offs, quantify, show failure thinking, and admit uncertainty with a plan to resolve it ("I would validate this with a 2-day spike measuring X"). Evaluators trust candidates who name what they do **not** know and how they would find out. Bring your POC evidence: traces, load-test results, ADRs, and the mistakes you corrected.